

操作系统

第一章 操作系统 概论	3
第一节 操作系统的概念	3
第二节 操作系统的发展	6
第三节 操作系统分类	7
第四节 操作系统设计	10
第五节 操作系统启动	14
第二章 操作系统的运行环境	15
第一节 计算机系统的层次结构	15
第二节 中央处理器	15
第三节 存储系统	17
第四节 中断机制	19
第五节 I/O 技术	21
第六节 时钟	22
第七节 系统调用	22
第三章 进程线程模型	23
第一节 进程基本概念	23
第二节 进程状态及状态转换	28
第三节 线程引入及基本概念	29
第四节 线程的实现和实例	31
第四章 进程线程调度	32
第一节 进程调度的基本概念	32
第二节 进程调度算法的设计思路	33
第三节 经典进程调度算法	34
第四节 其他进程调度算法	37
第五节 操作系统调度算法实例	38
第六节 多处理器调度算法	40
第七节 实时调度算法	40
第五章 存储管理	41
第一节 概述	41
第二节 分区管理方案	44
第三节 覆盖与交换技术	48
第四节 虚拟页式存储管理方案	49
第六章 文件系统	54
第一节 文件管理的基本概念	54

第二节 文件的逻辑结构和物理结构	56
第三节 文件目录	58
第四节 文件存储空间管理	61
第五节 实现文件系统的表目	64
第六节 文件及文件目录的操作	64
第七节 文件系统的性能	66
第八节 文件共享、保护	67
第七章 设备管理	68
第一节 设备管理的基本概念	68
第二节 设备硬件和设备管理软件的组成	70
第三节 I/O 设备的控制方式	72
第四节 设备分配与回收	74
第五节 磁盘驱动调度	77
第六节 缓冲技术	82
第七节 虚拟设备技术	83
第八章 进程同步机制与死锁	84
第一节 进程（线程）的同步与互斥	84
第二节 经典的进程同步问题	88
第三节 死锁	91
第四节 哲学家就餐问题	97

更多资料 自考真题考研资料库

操作系统

第一章 操作系统概论

第二章 操作系统运行环境与运行机制 第三章 进程/线程模型

第四章 进程/线程调度

第五章 存储管理 第六章 文件系统 第七章 设备管理

第八章 进程同步机制与死锁

第一章 操作系统 概论

本章重难点分析

1.操作系统的设计 2.操作系统的启动

3.操作系统的体系结构

第一节 操作系统的概念

一、计算机系统

计算机系统是一种可以按用户的要求接收和存储信息、自动进行数据处理并输出结果信息的系统

广义的计算机系统包含机械式系统和电子式系统两类。

在电子式系统中，还可以划分为模拟式和数字式两种计算机系统。我们在这里讨论的是数字电子计算机系统，简称计算机系统。

计算机系统包括硬件（子）系统和软件（子）系统。硬件系统是计算机系统赖以工作的实体。

软件系统保证计算机系统按用户指定的要求协调工作（灵魂）。

计算机系统的资源包括两大类：硬件资源和软件资源。计算机硬件系统：

中央处理器（CPU=运算器+控制器） 内存储器（又称主存）

外存储器（磁盘、磁带等）

输入输出设备（键盘、鼠标、显示器、打印机等） 软件系统=程序+数据

一、计算机系统

计算机系统	软件系统 (程序、数据)	应用软件：文字处理、图形图像处理、 科学计算、MIS...
		支撑软件：数据库、网络、多媒体...
		系统软件：操作系统、编译程序...
	硬件系统	中央处理单元（CPU）、主存储器 外存储器（磁盘、磁带...） 输入输出设备（键盘、鼠标、显示器、打印机...）

二、操作系统的定义

操作系统是计算机系统中的一个系统软件，它是这样一些程序模块的集合：它们能有效地组织和管理计算机系统中的硬件及软件资源，合理地组织计算机工作流程，控制程序的执行，并向用户提供各种服务功能，使得用户能够灵活、方便、有效地使用计算机，并使整个计算机系统能高效地运行。

三、操作系统的特征 1.并发性

并发性是指在计算机系统中同时存在若干个运行着的程序，从宏观上看，这些程序在同时向前推进。

计算机程序的并发性体现在如下具体两个方面：

- 1) 用户程序与用户程序之间并发执行；
- 2) 用户程序与操作系统程序之间并发执行。两个相似的概念：并行性以及并发性。

并行性是指两个或者多个事件在同一时刻发生，这是一个具有微观意义的概念，即在物理上这些事件是同时发生的。

而并发性是指两个或者多个事件在同一时间间隔内发生，它是一个较为宏观的概念，与所使用的时间间隔相对应的、有某种程度的统计意义。

2.共享性

共享性是指操作系统程序与多个用户程序共用系统中的各种资源。这种共享性是在操作系统控制下实现的。

资源的共享性主要针对计算机系统中的如下几项重要资源。

- 1) 中央处理器
- 2) 内存储器
- 3) 外存储器
- 4) 外部设备

PS：所有的硬件资源

资源的共享一般有两种形式：互斥共享和同时共享 3.虚拟性

虚拟性是一种管理技术，该技术把物理上的一个实体变成逻辑上的多个对应物，或把物理上的多个实体变成逻辑上的一个对应物。

4.异步性

操作系统的运行是在一种随机的环境下进行的。

这种随机环境的含义是，操作系统不能对所运行的程序的行为以及硬件设备的情况做出任何事先的假定。

随机性并不意味着操作系统不能很好地控制资源的使用和程序的运行，随机性突出强调了在进行操作系统的设计与实现时要充分考虑各种各样的可能性。

操作系统本身应该稳定、可靠、安全、高效，实现程序并发和资源共享的目的。

四、研究操作系统的观点

1、软件的观点

从软件的观点来看，操作系统是一种大型软件系统，它是多种功能程序的集合。作为一种大型软件系统，操作系统有软件的外在特性和内在特性。

1) 外在特性是指，操作系统是一种软件，它的外部表现形式。

2) 内在特性是指，操作系统既然是一种软件，它就具有一般软件的结构特点。

2、资源管理的观点

在计算机系统硬件和软件资源可以分成以下几部分：中央处理器（CPU）、存储器（内存和外存）、外部设备和信息（文件）。

3、进程的观点

用进程的观点，则把操作系统看作由多个可以同时独立运行的程序和一个对这些程序进行协调的核心所组成。

4、虚拟机的观点

虚拟机的观点是从系统功能分解的角度出发，考虑操作系统的结构。

5、服务提供者的观点

从用户的角度，站在操作系统之外观察操作系统，则可以认为该服务提供者为用户提供比裸机功能更强、服务质量更高、更方便的灵活的虚拟机器。

为了用户使用上的便利，该服务提供者提供了一组功能强大的、方便、易用的广义指令（称为系统调用）。

五、操作系统的功能

1、进程管理（处理器管理） 进程管理的主要内容：

（1）进程控制：创建进程时资源分配，进程退出回收资源。控制进程的状态。

（2）进程同步：多个进程并发执行，用同步或者互斥的方式实现同步机制。

（3）进程间通信：相互协作的进程之间交换数据和消息。

（4）调度：即处理器调度。包括线程调度、进程调度、作业调度。

2、存储管理

存储管理的任务是管理计算机内存的资源。存储管理有三个方面的任务。

1) 当多个程序共享有限的内存资源时，要考虑如何为多个程序分配有限的内存空间；

2) 存储在内存中的多个程序和数据应该彼此隔离、互不侵扰；

3) 解决内存扩充的问题，即将内存和外存结合起来管理，为用户提供一个容量比实际内存大得多的虚拟存储器；

概括地说，存储管理有三个方面的任务。

（1）内存的分配与回收

（2）存储保护

（3）内存扩充

3、文件管理

在计算机系统信息资源（如程序和数据）是以文件的形式存储在外存储器（如磁盘、磁带）上的，需要时再把它装入内存。

（1）文件存储空间管理

（2）目录管理

（3）文件系统的安全性

4、设备管理

由操作系统的设备管理功能负责外部设备的分配、启动和故障处理，用户不必详细了解设备及接口的技术细节，就可以方便地通过操作系统提供的设备管理手段，对设备进行操作。

5、用户接口

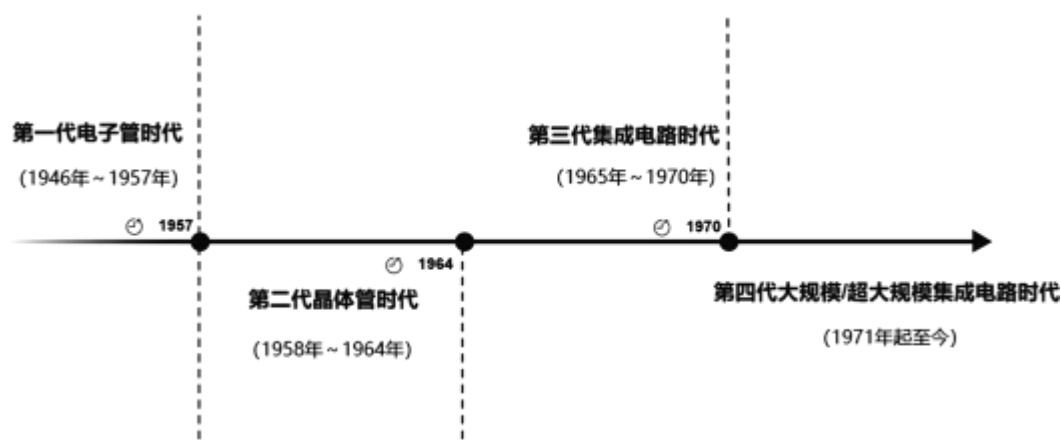
除了上述五项功能之外，操作系统还应该向用户提供使用它自己的手段，这就是用户与计算机系统之间的接口。

1) 命令接口

2) 应用程序接口

第二节 操作系统的发展

一、操作系统的发展历史



二、不同时期的操作系统

1.手工操作：二战时期对武器装备设计的需要，美国、英国和德国等国家，陆续开始使用真空管建造数字电子计算机。通过在插板上的硬连线控制计算机的基本功能，程序设计全部采用机器语言，没有程序设计语言（甚至没有汇编语言）。20世纪50年代早期，出现穿孔卡片，可以将程序写在卡片上，此阶段特点是手工操作、独占方式

2.监控程序（早期批处理）：50年代晶体管的发明后，出现了FORTRAN、ALGOL以及COBOL等高级语言要运行一个作业（JOB），程序员首先将程序写在纸上，穿孔成卡片，将卡片盒带到输入室，交给操作员，计算结果从打印机上输出，操作员到打印机上取下运算结果并送到输出室，程序员取到结果。由于处理器速度提高，导致手工操作设备输入/输出信息与计算机计算速度不匹配，人们设计了监督程序（或管理程序），来实现作业的自动转换处理。操作员将作业“成批”地输入到计算机中，由监督程序识别一个作业，进行处理后再取下

一个作业。这种自动定序的处理方式称为“批处理（Batch Processing）”方式，由于是串行执行作业，因此称为单道批处理

3.多道批处理：进入第三代计算机以后，硬件有了很大发展，通道使得输入/输出操作与 CPU 操作并行处理成为可能。与此同时软件系统也随之相应变化，实现了在硬件提供并行处理之上的多道程序设计

所谓多道是指它允许多个程序同时存在于主存之中，由中央处理器以切换方式为之服务，使得多个程序可以同时执行。计算机资源不再是“串行”地被一个个用户独占，而可以同时为几个用户共享，从而极大地提高了系统在单位时间内处理作业的能力。这时管理程序已迅速地发展成为一个重要的软件分支——操作系统

4.分时与实时系统：批处理系统不适合交互式的作业，对交互式工作方式的需求导致了分时系统（Time Sharing System）的出现。分时系统是指多个用户通过终端设备与计算机交互作用来运行自己的作业，并且共享一个计算机系统而互不干扰。由于调试程序的用户常常只发出简短的命令（如编译一个五页的源文件），而很少有长的费时命令（如上百万条记录的文件排序），所以计算机能够为许多用户提供交互式快速的服务，同时在 CPU 空闲时还能运行大的后台作业

5.UNIX 通用操作系统：20 世纪 60 年代末，贝尔实验室的 Ken Thompson 和 Dennis M. Ritchie 设计了 UNIX 操作系统，以下是 UNIX 的特点：

用 C 语言编写，可移植

世界上唯一能在笔记本计算机、PC 机、工作站直至巨型机上运行的操作系统

系统源代码非常有效，系统容易适应特殊的需求

良好的、通用的、多用户、多任务、分时操作系统 树形文件系统和一定的安全机制

简洁的字符流文件和文件保护机制

具备功能强大的 shell，它既是命令解释器，又是编程用高级语言，可用于扩充系统的功能 6.个人计算机操作系统：20 世纪 70 年代末期出现了面向个人计算机的操作系统，例如微软公司的 MS DOS 操作系统，属于单用户单任务操作系统

1984 年，苹果推出交互式图形功能的操作系统

1992 年，微软推出交互式图形功能的 Windows 3.1

1995 年，Windows 95 出现，Windows 成为个人计算机主流操作系统

1991 年，Linus 公布 Linux 操作系统，继承 UNIX 全部优点，开放源码，Linux 极其健壮 7.当代操作系统两大发展方向——宏观应用与微观应用：

大型系统：分布式操作系统、机群操作系统 微型系统：嵌入式操作系统

第三节 操作系统分类

一、批处理操作系统

1、基本工作方式

收到一定数量的用户作业后，组成一批作业，输入到计算机，在系统中形成一个连续的、自动转接的作业流。

2、特点与分类

优点：成批处理、系统资源利用率高、作业吞吐率高、作业流程自动化较高、作业吞吐量

缺点：用户不能直接与计算机交互，不适合调试程序 分类：简单批处理系统和多道批处理系统

3、设计思想

简单批处理系统在操作系统发展早期出现，被称为早期批处理系统，或监控程序。在监控程序启动前，操作员有选择地把若干作业合并成一批作业，将这批作业安装在输入设备上。然后启动监控程序，监控程序将自动控制这批作业的执行，作业的运行以及作业之间的衔接都由监控程序自动控制，有效地提高了作业运行的效率。

4、作业控制说明书

在简单批处理系统中，作业的运行步骤是由作业控制说明书传递给监控程序。作业控制说明书是由作业控制语言编写的一段程序，通常存放在被处理作业前面。在运行过程中，监控程序读入并解释作业控制说明书中的语句，以控制各个作业步的执行。

5、SPOOLing 技术

为了提高硬件资源的利用率，批处理系统发展为更加高级的多道批处理系统，关键技术就是多道程序运行、假脱机（SPOOLing）技术等。

假脱机技术，全称是“同时的外部设备联机操作”，借助硬件通道技术，实现了输入输出操作和处理器动作的自动并行处理。通道是指专门用来控制输入输出的硬件设备，可以看作是专门的 I/O 处理机。通道基本上是自主控制外设的，可以与 CPU 并行工作。假脱机技术为实现多道批处理系统中的多道程序设计思想提供了重要的基础。

二、分时操作系统

分时操作系统出现在批处理操作系统之后，弥补批处理方式不能向用户提供交互式快速服务的缺点。

1、基本工作方式

用户通过终端交互式地向系统提出命令请求，系统接受用户的命令之后，采用时间片轮转方式处理服务请求，并通过交互方式在终端上向用户显示结果。用户根据系统送回的处理结果发出下一道交互命令。

2、设计思想

将 CPU 的时间划分成若干个小片段，称时间片。操作系统以时间片为单位，轮流为每个终端用户服务。

3、特点

分时操作系统具有多路性、交互性、独占性和及时性的特点：多路性指有多个用户在同时使用一台计算机交互性指用户根据系统响应的结果提出下一个请求，用户直接干预操作每一步的进行独占性指每个用户感觉不到计算机为其他人服务

及时性指系统能够对用户提出的请求及时给予响应

通用操作系统结合了分时系统与批处理系统两种系统的特点。对于分时与批处理的处理原则是：分时优先，批处理在后。在通用操作系统中，一般把系统的分时和批处理运行状态，称为前台和后台在系统的“前台”响应需频繁交互的作业，如大量终端的处理要求；而“后台”则处理处理对时间性要求不强的作业

三、实时操作系统

实时操作系统（RTOS: Real Time Operating System）：在规定时间内，及时响应外部事件请求，完成事件处理，控制实时设备和实时任务协调一致工作 分类：

①硬实时：对关键事件的响应和处理时间严格要求，否则产生严重后果

②软实时：对事件响应和处理时间有一定要求，不满足要求会影响服务质量，但不会引发严重后果

实时系统为实现硬实时或软实时的要求，需要具备以下能力：（1）实时时钟管理

实时任务根据时间要求可以分为两类：

定时任务，它依据用户的定时启动并按照严格的时间间隔重复运行

延时任务，它非周期地运行，允许被延后执行，但是往往有一个严格的时间界限 依据任务功能的不同，可分为主动式任务和从动式任务：

主动式任务依据时间间隔主动运行，多用于实时监控 从动式任务依赖于外部事件的发生

（2）过载防护

实时任务的启动时间和数量具有很大的随机性，突发的大量实时任务极有可能超出系统的处理能力，从而发生过载。实时系统在出现过载现象时，要有能力迅速分析判断并找出最重要的实时任务，通过抛弃或者延后次要任务以保证最重要任务成功执行。

（3）高可靠性

高可靠性是实时系统的设计目标之一，实时操作系统需要有很强的健壮性和坚固性。

四、个人计算机操作系统

个人计算机操作系统（**Personal Computer Operating System**）是一种单用户的操作系统 个人计算机操作系统的主要特点：

计算机在某一时间内为单个用户服务

采用图形界面人机交互的工作方式，界面友好 使用简单方便

五、网络操作系统

网络操作系统（**Network Operating System**）是基于计算机网络的、在各种计算机操作系统之上按网络体系结构协议标准设计开发的软件

计算机网络的不同模式 集中式模式

运算处理在主计算机里发生，终端仅作为输入/输出设备使用，通过连接两台或更多主机的方式构成计算机网络 分布式模式

每台计算机都有运算处理能力，多台计算机通过网络交换数据并共享资源和服务。在分布式模式中，网络环境中的计算机不仅能共享数据、资源及服务，还能够共享运算处理能力 分布式操作系统

分布式系统（**Distributed System**）将大量的计算机通过网络连结在一起，获得极高的运算能力及广泛的数据共享

分布式操作系统（**Distributed Operating System**）是网络操作系统的更高级形式，分布式操作系统除了保持了网络操作系统的各种功能之外，还具备如下的特征：

（1）系统中的所有主机使用统一的操作系统

（2）资源深度共享。通过统一的操作系统的调度，在某台主机的一个计算任务可以迁移到另一台主机上执行

（3）透明性。用户不知道分布式系统运行在多少台计算机上，各个主机地理位置上的差异对用户透明

（4）自治性。各个主机之间没有主从关系，一个主机的失效一般不会影响整个分布式系统 六、分布式操作系统

系统

分布式系统把系统中的所有计算机构成一个完整的、功能更加强大的计算机系统，系统中的计算机相互协作，共同完成一个大型计算任务，一个计算任务可分解成若干可以并行执行的子任务，每个子任务分别在不同的计算机上执行。

分布式系统的优点：

分布式。以较低的成本获得较高的运算性能

可靠性。当一个计算机系统发生故障时，整个分布式系统仍旧能够工作。适用于高可靠的环境

机群（cluster）是分布式系统的一种，一个机群通常由一群处理器密集构成，机群操作系统专门服务于这样的机群。在机群操作系统的指挥下，可以用低成本的微型计算机和以太网设备等产品，构造出性能相当于超级计算机运算性能的机群。

七、嵌入式操作系统

嵌入式操作系统用于工业控制、交通管理、信息家电等嵌入式系统，设计紧凑、高效，

只保留运行在其上的特定应用程序所需要的功能。嵌入式操作系统的特点：

（1）系统内核小。由于资源有限，嵌入式操作系统的内核要比传统的操作系统小得多

（2）专业性强。与硬件紧密结合，针对不同的任务和设备进行定制和移植，不能通用

（3）系统精简。不需要过于复杂的功能设计和实现，降低系统成本和提高系统安全性

（4）高实时性。能够及时响应外部事件和中断，保证任务的执行顺序和完成时间

（5）多任务的操作系统。能够支持多任务的调度和管理，合理地分配和利用系统资源

八、其他类型操作系统
（1）大型机操作系统，能够处理海量的数据和事务，提供高度的安全性、可靠性和可伸缩性，通常用于银行、政府、保险等行业

（2）服务器操作系统，支持多个用户和多个任务，提供网络服务和资料共享。具有高效的内存管理、文件系统和安全机制，通常用于互联网、企业等领域

（3）多处理操作系统，利用多个处理器并行执行多个任务，提高性能和吞吐量。具有复杂的调度算法、同步机制和通信协议，通常用于科学计算、图形处理等领域

（4）移动计算操作系统，支持移动通信、定位、传感等功能，提供用户友好的界面和应用。具有低功耗、小内存占用和高可移植性，通常用于智能手机、平板电脑等设备

（5）传感器节点操作系统，支持传感器数据采集、处理和传输，提供无线网络和自组织能力。具有极低的资源需求、高度的可配置性和实时性，通常用于物联网、环境监测等领域

（6）智能卡操作系统，支持智能卡的存储、计算和安全功能，提供身份认证和支付服务。具有高度的安全性、稳定性和兼容性，通常用于金融、交通等领域

第四节 操作系统设计

一、操作系统设计难点

操作系统设计过程的主要困难：

1、设计复杂程度高

程序长、接口信息多、动态性强、并行性强，不同部分之间可以同时操作

2、正确性难以保证

操作系统包含很多功能成份，各种外部设备的接口复杂，导致操作系统源代码的正确性较难以保证。操作系统的正确性依靠软件工程的规范来保证

3、研制周期长

一般的软件研制开发周期包括：

需求分析，用户提供软件的需求，并根据需求制作软件规格说明书 设计者根据软件规格说明书进行软件设计
实现者编写程序，调试、测试并整理各种开发文档，提交给用户使用，在应用的过程中 不断改进和提高软件的品质

二、操作系统的设计过程

操作系统的设计过程可分为三个部分：功能设计、算法设计和结构设计。操作系统的三方面 设计是互相渗透的，不能截然分开

1、功能设计

功能设计指根据系统的设计目标和使用要求，确定所设计的操作系统应具备哪些功能， 以及操作系统的类型。

2、算法设计

算法设计是根据计算机的性能和操作系统的功能，选择和设计满足系统功能的算法和策略，并分析和估算其效能。

3、结构设计

结构设计是按照系统的功能和特性要求，选择合适的结构，使用相应结构设计方法将系统逐步地分解、抽象和综合，使操作系统结构清晰、简明、可靠、易读、易修改，而且使用 方便，适应性强。

三、操作系统设计目标

一个高质量的操作系统应具有可靠性、高效性、易维护性、易移植性、安全性和简明性 等特征。

1、可靠性

可靠性包括正确性和健壮性。正确性受并发、共享以及随之带来的不确定性影响，要求 系统能动态地应付随时发生的各种内部和外部事件

2、高效性

提高系统中各种资源的利用率

3、易维护性

易维护性包括易读性、易扩充性、易剪裁性、易修改性等含义。

4、可移植性

可移植性指，把一个程序系统从一个计算机系统环境中移到另一个计算机系统环境中并能正常运行的特性。

5、安全性

操作系统的安全性是整个计算机系统安全性的基础，为保护用户的程序和数据提供了最 基本的安全机制。

四、操作系统结构设计

1、操作系统结构设计的重要性

早期操作系统：结构简单，规模小，逻辑关系简单，只注重功能设计和效率，忽视结构设计
随计算机结构复杂化，应用范围扩大，要求提高，需要较强的功能、可适应性和可靠性

2、操作系统结构研究的目标

（1）系统模块化

将模块看做一组数据结构以及定义在其上的一组操作

（2）模块标准化

①标准设计，模块规格划一，遵循相同模块构造准则和模块（构件）标准

②总结、提炼基本成份并定型化

（3）通信规范化

模块之间接口清晰划一，联系方式统一 五、操作系统结构的体系结构

1、整体式结构

首先确定操作系统的总体功能，将总功能分解为若干个子功能，实现每个子功能的程序 称为模块。按照功能将每个大模块分解为若干个较小的模块，直至每个模块仅包含单一功能 或紧密联系的小功能为止，通过接口将所有模块连接起来形成一个整体，称之为模块组合结 构。

模块组合法的优点：

- 1) 结构紧密，接口简单直接，系统效率较高
- 2) 系统中的模块不是根据程序和数据本身的特性而是根据它们完成的功能来划分的，数据 基本上作为全程量使用
- 3) 不同模块的程序之间可以不加控制地互相调用和转移，信息的传递方式也可以根据需要 随意约定，因而可能造成模块间的循环调用

模块组合法的缺点：

- 1) 模块间转接随意，各模块互相牵连，独立性差，系统结构不清晰
- 2) 数据基本上作为全程量处理，系统内所有模块的任一程序均可对其进行存取和修改，从 而造成了各模块间有着更为隐蔽的关系，要更换一个模块或修改一个模块都比较困难
- 3) 模块组合结构常以大型表格为中心，为保证数据完整性，往往采用全局封中断办法，从 而限制了系统的并发性

2、层次式结构

层次式结构力求使模块间调用的无序性变为有序性。把操作系统的所有功能模块，按功 能流图的调用次序，分别将这些模块排列成若干层，各层之间的模块只能是单向依赖或单向

调用关系，结构清晰，且不构成循环 层次式结构的优点：

各模块之间的组织结构和依赖关系清晰明了。增加了系统的可读性和可适应性，每一步 都建立在可靠的基础上，只要下层的各模块设计是正确的，就为上层功能模块的设计提供了 可靠基础，从而增加了系统的可靠性 对操作系统增加或替换掉一层而不影响其他层次。层次结构法设计的操作系统具有易于 调试、易于修改、易于扩充、易于维护、易于保证正确性等优点

模块放在哪一层，共有多少层，依据功能流图和数据流图分层，原则如下：

（1）为增加可适应性，方便移植，把与机器特点紧密相关的软件，如中断处理、输入输出 管理等放在紧靠硬件的最低层。为便于修改移植，把与硬件有关和与硬件无关的模块截然分 开，把与硬件有关的BIOS（管理输入输出设备）放在最内层。当硬件环境改变时只需修改 这一层

（2）为便于一种操作方式转变到另一种操作方式，把共同使用的基本部分放在内层，把改 变部分放在外层

（3）为给进程提供环境和条件，系统调用的各功能，为进程提供服务，这些功能构成操作 系统内核，放在系统内层

3、微内核（客户/服务器）结构

客户/服务器结构的操作系统适于应用在网络环境下分布式处理的计算环境中，又称为微内 核体系结构，特点如下：

(1) 运行在内核态的内核：内核提供所有操作系统都具有的操作，如线程调度、虚拟存储、消息传递、设备驱动以及内核的原语操作集和中断处理等

(2) 运行在用户态的并以客户/服务器方式运行的进程层：除内核部分外，操作系统所有的其他部分被分成若干个相对独立的进程，每一个进程实现一组服务，称为服务进程。这些服务进程可以提供各种系统功能、文件系统服务以及网络服务等。服务进程的任务是检查是否有客户机提出要求服务的请求，并在满足客户机进程的请求后将结果返回

优点：分成若干小的自包含分支（服务进程），分支运行在独立进程中，通过规范一致方式收发消息。内核中建立最小机制，策略留给服务进程，灵活性强

(1) 可靠：分支独立自包含，某个服务器失败，不会引起其他服务器崩溃

(2) 灵活：便于新功能，接口规范，修改服务器代码不影响系统其他部分，可维护性好

(3) 适于分布式处理环境：不同服务运行在不同计算机上，具有分布式处理能力 缺点：只能通过微内核通信，通信频繁时，效率低

4、外核结构

外核结构（Exokernel）是一种操作系统结构。在这种结构中，内核负责的主要工作仅为简单的申请操作以及释放和复用硬件资源，其由以往操作系统提供的抽象全部在用户空间中运行。这种方式实现了应用级资源管理，即由应用程序而不是操作系统管理硬件资源

5、典型实例

1) Unix

Unix 是一种多用户、多任务的操作系统，具有很强的可移植性和开放性。它采用分层结构，内核负责管理硬件资源，提供系统调用接口给上层应用程序

2) Linux

Linux 是一种类似于 Unix 的操作系统，它采用了模块化的设计方法，内核负责硬件驱动、进程调度、内存管理等基本功能，而其他功能则通过模块化的方式实现

3) Windows

Windows 是微软公司推出的一种操作系统，它采用了混合内核结构，内核负责硬件驱动、进程调度、内存管理等基本功能，同时也提供了图形用户界面和丰富的应用程序接口

4) Android

Android 是谷歌公司推出的一种基于 Linux 内核的移动操作系统，它采用虚拟机技术来运行应用程序，并提供了丰富的应用程序接口和开发工具

5) 鸿蒙操作系统

鸿蒙操作系统（HarmonyOS）是华为推出的一款面向万物互联的全场景分布式操作系统，支持手机、平板、智能穿戴、智慧屏等多种终端设备运行，提供应用开发、设备开发的一站式服务的平台。它采用微内核设计，仅包括了操作系统必要的功能模块（任务管理、内存分配等）

6) OpenEuler 操作系统。

OpenEuler 是开放原子开源基金会（OpenAtom Foundation）孵化及运营的开源项目。

OpenEuler 是一款面向数字基础设施的操作系统，支持服务器、云计算、边缘计算、嵌入式等应用场景，支持多样性计算，致力于提供安全、稳定、易用的操作系统

7) 银河麒麟操作系统

银河麒麟是由天津麒麟信息技术有限公司开发和维护的一款国产操作系统。它支持以 x86、x86_64、PowerPC、SPARC 为代表的国际主流 CPU 和以飞腾为代表的国产 CPU，并已经广泛应用于国防、军工、政务、电力、航天、金融、电信、教育、大中型企业等行业或领域

第五节 操作系统启动

一、引导方式

(1) BIOS 引导。BIOS 是存储在 BIOS 芯片中的程序，当计算机启动时，BIOS 会执行硬件自检并初始化硬件配置，后根据 CMOS 中用户指定的硬件启动顺序，读取相应设备的启动或引导记录，引导相应设备上的操作系统启动

(2) UEFI 引导。UEFI 是一种新型的固件接口，采用模块化、高级语言（主要是 C 语言）构建的一个小型化系统，直接利用加载 EFI 驱动的方式，识别系统硬件并完成硬件初始化

二、操作系统的引导过程

1. 计算机开机，BIOS 自检并检测第一个能够引导系统的设备，如硬盘或光驱 2. 运行 MBR 扇区的启动程序，如 GRUB 引导程序

3. GRUB 引导程序读取配置文件获取内核和镜像文件系统的设置以及路径位置

4. 内核和镜像文件系统加载到内存 5. 加载硬件驱动程序，初始化进程 三、操作系统的启动机制

操作系统的启动过程：BIOS 自检、系统引导、启动内核、初始化系统

1. 计算机开机，BIOS 程序将存储设备的引导记录（Boot Record）载入内存，执行引导记录中的引导程序（Boot）

2. 引导程序将存储设备中的操作系统内核载入内存，进入内核入口点开始执行 3. 内核完成系统初始化，允许用户进行交互

四、典型程序执行流程

一个程序的典型执行流程：

(1) 用户通过操作系统启动程序

(2) 操作系统将程序从磁盘加载到内存中

(3) 操作系统为程序分配资源，如 CPU 时间、内存空间和输入/输出设备

(4) 程序开始执行，由 CPU 按顺序执行指令

(5) 程序可能会与操作系统交互，请求服务，如读写文件或访问网络

(6) 程序执行完毕后，操作系统回收其占用的资源

第二章 操作系统的运行环境

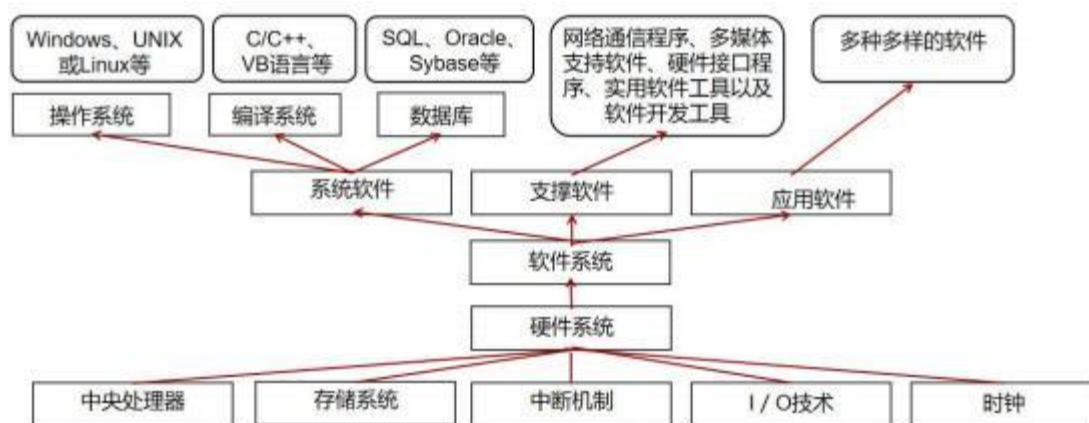
本章重难点分析 1.中央处理器

2.中断机制

3.系统调用

第一节 计算机系统的层次结构

所有的子系统都可以包括在硬件（子）系统和软件（子）系统这两个层次中



第二节 中央处理器

一、CPU 的构成与基本工作方式

CPU 一般由运算器、控制器、一系列的寄存器以及高速缓存构成。运算器实现指令中的算术和逻辑运算，是计算机计算的核心。

控制器负责控制程序运行的流程，包括取指令、维护处理器状态、处理器与内存的交互等。寄存器是一种暂时存储器件，用于处理器执行指令的过程中暂存数据、地址以及指令信息。

1.处理器中的寄存器

提供了一定的存储能力 速度比主存储器快得多

造价很高，存储容量一般都很小 寄存器一般分为二类：

- 1) 用户可见寄存器：通过一定的算法分配并使用这些寄存器，以最大限度地减少程序运行时访问内存的次数，这对程序的运行速度影响很大。
- 2) 控制和状态寄存器：用于控制处理器的操作，一般由具有特权的操作系统代码使用，以控制其他程序的执行。

常见的用户可见寄存器：

- 1) 数据寄存器：又称为通用寄存器，主要用于各种算术逻辑指令和访存指令。
- 2) 地址寄存器：用于存储数据及指令的物理地址、线性地址或者有效地址，用于某种特定方式的寻址。

3) 条件码寄存器：保存处理器操作结果的各种标记位，例如算术运算产生的溢出、符号等，这些标记在条件分支指令中被测试，以控制程序指令的流向。

常见的控制和状态寄存器：

- 1) 程序计数器 (Program Counter, PC)，它记录了将要取出的指令的地址；
- 2) 指令寄存器 (Instruction Register, IR)，包含了最近取出的指令；
- 3) 程序状态字 (Program Status Word, PSW)，它记录了处理器的运行模式信息等。

2、指令执行的过程

一个指令周期=取指令+执行指令。

取指令：处理器每次从存储器中读取一条指令，程序计数器+1。

执行指令：存储在处理器的指令寄存器，处理器于是解释并执行这条指令。指令大致分成五类：

- 1) 访问存储器指令：它们负责处理器和存储器之间的数据传送；
- 2) I/O 指令：它们负责处理器和 I/O 模块之间的数据传送和命令发送；
- 3) 算术逻辑指令：有时又称为数据处理指令，用以执行有关数据的算术和逻辑操作；
- 4) 控制转移指令：这种指令可以指定一个新的指令的执行起点；
- 5) 处理器控制指令：这种指令用于修改处理器状态，改变处理器工作方式等。

二、特权指令和非特权指令

单用户、单任务方式下使用的微型计算机系统，可使用该计算机指令系统中的全部指令。多用户或多任务的多道程序设计环境中，指令两部分：特权指令和非特权指令

特权指令：在指令系统中那些只能由操作系统使用的指令。非特权指令：大家（操作系统、普通用户）都能使用的。

三、处理器的状态 1.内核态和用户态

根据程序对资源和指令的使用权限而将处理器设置为不同状态 多数系统将处理器工作状态划分为内核态和用户态

内核态：一般指操作系统管理程序运行的状态，具有较高的特权级别，又称为管态、特权态、系统态或核心态。

用户态：一般指用户程序运行时的状态，具有较低的特权级别，又称为目态、普通态，还有些系统将处理器工作状态划分多个系统状态，例如核心状态，管理状态和用户程序状态（又称目标状态）三种。

内核态与用户态的差别：当处理器处于内核态时：

全部指令（包括特权指令）可以执行 可使用所有资源

具有改变处理器状态的能力 当处理器处于用户态时：

只有非特权指令能执行

特权级别不同，可以运行的指令集合也不相同 特权级别越高，可以运行的指令集合也越大

高特权级别对应的可运行指令集合包含低特权级 2.CPU 状态的转换

用户态到内核态的转换

其转换的唯一途径是通过中断 内核态到用户态的转换

可通过设置 PSW 指令（修改程序状态字）实现 3.限制用户程序执行特权指令

用户程序中不能使用特权指令

当用户程序占用CPU时，应让 CPU 在用户态下工作

若此刻有特权指令，则 CPU 将拒绝执行该指令，并形成“非法操作”事件

中断机制识别到事件后，转交操作系统处理，由操作系统通知用户“程序中有非法指令”

四、程序状态字

程序状态字的作用是指示处理器状态，用程序计数器来指示下一条要执行的指令，包括：CPU 的工作状态代码——指明当前 CPU 的工作状态是内核态还是用户态

条件码——反映指令执行后的结果特征 中断屏蔽码——指出是否允许中断

第三节 存储系统

中央处理器能直接访问的唯一的存储空间是主存储器

作业必须把它的程序和数据存放在主存储器中才能运行

多道程序系统，就会有若干个程序和相关的程序数据要放到主存储器中，操作系统要管理、保护这些程序和数据，使它们不至于受到破坏，互相影响，操作系统本身也要存放在主存储器中并运行

一、存储器的类型 1.类型

两类存储器：读写型的存储器，只读型的存储器 读写型的存储器

把数据存入任一地址单元，在以后把数据读出来，或者重新存入 RAM 主要用作存放随机存取的程序的数据

只读型的存储器

只能从其中读取数据，但不能随意地用普通的方法向其中写入数据 称为只读存储器（ROM：Read Only Memory）

变型：PROM 和 EPROM

PROM 一种可编程的只读存储器，可由用户使用特殊 PROM 写入器向其中写入数据

EPROM 是可用特殊的紫外线光照射擦去信息位，然后使用特殊 EPROM 写入器写入数据

在微型计算机中，一些常驻内存的模块以微程序形式固化在 ROM 中，如 BIOS 和 CBASIC 解释程序

2.存储分块

存储的最小单位称为“二进制”，它包含的信息为 0 或 1 最小编址单位是字节，一个字节一般包含八个二进制位

1024 个字节称为 1KB，1024 个 1KB 称为 1MB，1024 个 1MB 称为 1GB，等等

为简化对存储器的分配和管理，不少计算机系统中把存储器分成块，被称为一个物理页块

二、存储器的层次结构

存储系统的设计主要考虑三个问题：容量、速度和成本

容量：需求一般来说是无止境的 速度：能匹配处理器的速度

成本：其他部件相比应该在一个合适的范围之内 1.容量、速度和成本的匹配

三个目标不可能同时达到最优，需要作权衡

存取速度越快，平均每比特价格越高，容量越小

存取速度越慢，平均每比特价格越低，同时容量也增大

采用层次化的存储体系结构解决权衡问题：容量、速度和成本

沿层次下降时，每比特的价格将下降，容量将增大，速度将变慢而处理器的访问频率也将下降

较小较贵而快速的存储设备由较大较便宜而慢速的存储设备作后盾，通过对访问频率的控制来提高系统效能。



2.存储访问局部性原理

提高存储系统效能的关键在于程序存储访问局部性原理

程序执行时，有很多的循环和子调用，这样的程序段会重复存取相同的指令集合 数据存取也有类似的局部性，它们在较短的时间内能稳定在一个局部区域

处理器也主要和存储器的这个局部打交道

在经过一段时间以后，使用到的代码和数据的集合会改变

三、存储器保护

对主存中的信息加以保护，使操作系统不被破坏，是其正确运行的基本条件之一 要实现存储保护，必须要有硬件的支持

界地址寄存器（界限寄存器） 存储键

1.界地址寄存器（界限寄存器）

界地址寄存器是被广泛使用的一种存储保护技术

这种机制比较简单，易于实现 实现方法：

设置一对界限寄存器来存放作业的下限和上限地址

也可将一个寄存器作为基址寄存器，另一寄存器作为限长寄存器 访问主存时，硬件自动判断是否越界

如果未越界，则按此地址访问主存，否则将产生程序中断——越界中断或存储保护

中断

2.存储保护键

每个存储块都有与其相关的存储保护键，附加在每个存储块上

当一个用户作业进入主存时，操作系统分给它一个唯一的存储键号 将分配给该作业的各存储块的存储键，也设置成同样的键号

当该作业运行时，操作系统同时将该作业的存储键号存放到程序状态字的存储键钥匙域中 每当CPU 访问主存时，都将对主存块的存储键与 PSW 中的钥匙进行比较

如果相匹配，则允许访问，否则，拒绝并报警

第四节 中断机制

中断机制是操作系统中极为重要的一个部分，操作系统在管理输入输出设备，处理外部事件时，都需要中断机制处理，中断机制的实现必须依靠相关的硬件支持，中断的实现是硬件中断装置和相应的中断处理软件共同完成的。

一、中断的概念 1.什么是中断

中断是 CPU 对系统中或系统外发生的异步事件的响应 异步事件是指无一定时序关系的随机发生的事件 当发生某个异步事件后，中断处理器对当前程序的执行，而转去处理该异步事件，处理完了之后，处理器再转回原程序的中断点继续执行

中断的作用

解决了主机和外设并行工作的问题

消除了因外设的慢速而使得主机等待的现象 为多机操作和实时处理提供了硬件基础

充分发挥处理器的使用效率 提高系统的实时能力

2.中断的分类

强迫性中断：程序所不希望发生的，具有随机性

程序性中断：运行程序本身的中断时钟中断 输入输出（I/O）中断：由 I/O 控制器产生 控制台中断 硬件故障中断

自愿性中断：程序有意安排执行的，通常由访管指令引起，目的是要求操作系统提供系统服务

异步中断：简称为中断，对当前程序而言的外部事件激发的，属于外源性质，具有随机性 时钟中断

输入输出（I/O）中断 控制台中断

硬件故障中断

同步中断：一般称为异常由当前程序的编码决定，发生的位置可以准确预言

程序性中断

访管指令异常 二、中断系统

中断系统分为两大组成部分：中断系统的硬件中断装置和软件中断处理程序

硬件中断装置负责捕获中断源发出的中断请求，并响应中断源将处理器控制权移交给特定的 中断处理程序

中断处理程序针对中断事件的性质而执行相应的一系列操作 1.中断请求的接收

通过在计算机硬件的中断逻辑线路和中断寄存器实现

中断逻辑线路接收中断信号，把收到的中断信号寄存在硬件触发器中

中断逻辑线路中有若干个接受中断信号的触发器，触发器全体称为中断寄存器

2.中断响应 何时响应

在每条指令执行周期内的最后时刻扫描中断寄存器 如何响应

保存中断点的程序执行上下文环境在一个专门的系统堆栈中，包括程序状态字 PSW、程序计数器 PC 中的下一条指令位置和一些寄存器的值

切换到内核态

根据中断向量代号查询中断向量表，获得与该中断源相联系的中断处理程序的入口地址，并将 PC 置成该地址 控制权转移到中断处理程序 3.中断处理

典型的处理过程：

- 1) 设备给处理器发了一个中断信号
- 2) 处理器处理完当前指令后响应中断，这个延迟非常短
- 3) 处理器处理完当前指令后检测到中断，判断出中断来源并向发送中断的设备发送了确认 中断信号确认信号使得该设备将中断信号恢复到一般状态
- 4) 处理器开始为软件处理中断做准备：保存中断点的程序执行上下文环境
- 5) 处理器根据中断源查询中断向量表，获得与该中断相联系的处理程序入口地址，处理器 控制转移到中断处理程序
- 6) 中断处理程序开始工作
- 7) 中断处理结束时，恢复上下文环境。处理器状态恢复成原来的状态
- 8) 处理器开始一个新的指令周期，中断处理结束

4.典型中断的处理

(1) I/O 中断

由 I/O 设备的控制器或者通道发出

两大类：I/O 操作正常结束以及 I/O 异常

如果要继续进行 I/O 操作，则需要在准备好以后重新启动 I/O。若请求 I/O 的程序正处于等待 I/O 的状态，则应该将其唤醒

需要重新执行失败的 I/O 操作，次数有上限，次数过大的时候，系统将判定硬件故障

(2) 时钟中断

时钟中断处理程序通常要做较多的与系统运转、管理和维护相关的工作包括： 维护软件时钟
处理器调度

控制系统定时任务 实时处理

(3) 硬件故障中断

硬件故障中断处理程序需要做的工作，主要包括：

保存现场，使用一定的手段警告管理员并提供一些辅助的诊断信息

在高可靠的系统中，中断处理程序还需要评估系统的可用性，尽可能恢复系统

系统服务请求一般由处理器提供的专用指令（又称访管指令）来激发

如 x86 处理器提供 `int` 指令，用来激发软件中断

其他的不少处理器则专门提供系统调用指令 `syscall`

执行这些指令的结果是系统被切换到内核态，并且转移到一段专门的操作系统程序处开始执行

指令的格式通常是指令名加上请求的服务识别号（中断号）

操作系统利用处理器提供的这种接口建立自己的系统服务体系 处理器一般不负责定义系统调用所传递的参数格式

三、中断优先级、中断屏蔽与中断嵌套 1.多级中断与中断优先级

多级中断系统表现为有多根中断请求线从不同设备连接到中断逻辑线路上 连接在不同中断请求线上的中断信号，表示它们有不同的中断级别

中断信号的级别代表了该中断信号是否具有被优先处理的特权，以及这个特权的大小 多级中断的作用

对各类中断信号依据其紧急程度和重要性划分级别

解决如果有重要程度相当的多个中断信号同时到达时，如何选择首个被处理的中断信号的问题

2.中断屏蔽

可以允许或者禁止中断系统对某些类别中断的响应 在程序状态字 **PSW** 中设计有中断屏蔽位

主机是否允许响应或禁止某些中断，则由 **PWS** 中的中断屏蔽位决定

一旦某个 **I/O** 中断被程序状态字 **PSW** 中的中断屏蔽位加以屏蔽，即使有 **I/O** 中断信号，处理器也不予响应

还有一类中断信号是不可屏蔽的，机器故障中断，处理器要立即响应 3.中断嵌套

如果一个中断的处理过程中又发生了中断，那么将引起多个中断处理问题。对多个中断的处理策略一般有如下两种。

1) 禁止其他中断

这种处理方法可以用软件简单地实现。

2) 中断嵌套

中断按照优先度分级，允许优先级较高的中断打断优先级较低的中断处理过程

由于在中断嵌套中优先级较高的中断可以打断优先级较低的中断处理过程，因此必须把 优先级较低的中断处理过程的现场也保存起来

1.中断发生时，硬件中断装置将程序状态字 **PSW**、指令计数器 **PC** 的值，以及其他重要现场 信息压入系统堆栈

2.发生中断嵌套，被中断的处理的程序信息也被压入系统堆栈

3.每发生中断嵌套一次，就保护一次被中断的处理过程的现场，进行压栈操作 4.每结束一层嵌套的处理时，就从系统堆栈中弹出对应的现场信息，逐层恢复

第五节 I / O 技术

一、I/O 结构

早期的计算机系统中，外部设备的控制器通过 **I / O** 硬件结构与中央处理器连接 这种方法中，中央处理器定期轮询各个 **I/O** 设备控制器的状态

有 **I/O** 处理请求，中央处理器就处理 **I/O** 操作，直到处理完毕

主要缺陷是，处理器为了关注 **I/O** 设备控制器的状态，必须耗费大量的时间轮询各个外部设备，严重地降低了整个系统的性能，效率太低，已经淘汰

二、通道

通道独立于 **CPU**，专门负责数据 **I/O** 传输工作的处理单元，又称为 **I/O** 处理机 对外部设备实行统一的管理

代替 **CPU** 对 **I/O** 操作进行控制

使 **CPU** 和外部设备可以并行工作

通道可以实现中央处理器和各种外部设备并行工作 通道工作原理

当处理器执行到一条“启动外设”的指令时，就按指令中给定的参数启动指定的设备 设备启动之后，对该外部设备的控制权转移到通道

该外部设备与主存储器之间发生的信息传送，由通道控制

该外部设备工作结束后，会产生形成一个“输入输出操作结束”的 I/O 中断事件

三、直接存储访问技术

通过系统总线中的一个独立控制单元，自动地控制成块数据在内存和 I/O 单元之间的传送，处理器需要读写一整块数据的时候，它给 DMA 控制单元发送一条命令，命令中包含 I/O 设备的编址、开始读或写的主存编址、需要传送的数据长度等信息，发送完一条命令之后，就可以处理其他的事情，DMA 控制器将自动管理整块数据的传送，当这个传送过程完成后，它会给处理器发一个中断这样处理器只开始传送和传送结束时关注一下就可以了。

处理器和DMA 传送也不是完全并行

会有总线竞争的情况发生，这种总线竞争不会引起中断，也不会引起程序上下文的保存，处理器会稍稍等待。

通常这个过程只会花费一个总线周期

在 DMA 传送发生时，处理器访问总线的速度会有所变慢

对于大量数据的 I/O 传送来说，DMA 技术是很有价值 四、缓冲技术

缓冲区是硬件设备之间进行数据传输时，专门用来暂存这些数据的一个存储区域 三种情况下采用

处理器与主存储器之间

处理器和其它外部设备之间 设备与设备之间的通信

目的：为了解决部件之间速度不匹配的问题 多缓冲区（Cache）技术可提高设备利用率

Cache 是离 CPU 最近的高速内存，能使 CPU 更快速地访问经常使用的数据 CPU 首先到一级Cache 中去找数据

如果没有找到，那么CPU 接着到二级Cache 中去找 还找不到，CPU 就只好到速度较慢的系统内存中去找

第六节 时钟

时钟可以为计算机完成以下工作

在多道程序运行，时钟可以发现死循环，防止机时的浪费 在分时系统中，用时钟实现时间片轮转运行

在实时系统中，按要求的时间间隔输出信号控制设备 定时唤醒外部事件

记录用户和系统所需要的绝对时间，即年、月、日 时钟分成硬件时钟和软件时钟

硬件时钟工作原理：电路中的晶体振荡器，每隔一定间隔产生固定的脉冲频率，时钟电路中的时钟寄存器依据时钟电路所产生的脉冲数，对时钟寄存器进行加 1 的工作

软件时钟工作原理：利用内存单元模拟时钟寄存器，采用一段程序来计算相应的脉冲数，对内存时钟寄存器进行加 1 或减 1 的工作

第七节 系统调用

一、系统调用的概念与分类 1.什么是系统调用

系统调用，就是用户在程序中调用操作系统所提供的一些子功能，是一种特殊的过程调用，这种调用通常是由特殊的机器指令实现的，除了提供对操作系统子程序的调用外，这条指令还将系统转入特权方式，只能由汇编语言直接访问，是操作系统提供给编程人员的唯一接口，编程人员利用系统调用，动态请求和释放系统资源，

调用系统中已有的系统功能来完成与计算机硬件部分相关的工作以对用户屏蔽了操作系统的具体动作而只提供有关的功能，系统调用命令称“广义指令”，称为“虚处理机”

2.系统调用的分类

系统调用分为以下几类：

进程控制类系统调用 文件操作类系统调用

进程通信类系统调用 设备管理类系统调用 信息维护类系统调用

二、系统调用与一般过程调用的关系 运行在不同的系统状态

状态的转换 返回问题

嵌套调用

三、系统调用处理过程

有一个类似于硬件中断处理的中断处理机构

1. 用户使用系统调用时，产生一条相应的指令，处理机发生相应的中断，并发出有关的信号给该处理机构

2. 处理机构信号后，启动相关的处理程序去完成该系统调用所要求的功能 3. 为控制系统调用服务的机构称为陷入（TRAP）或异常处理机构

4. 由于系统调用引起处理机中断的指令称为陷入或异常指令（或称访管指令） 5. 每个系统调用都对应一个事先给定的功能号

6. 为实现各种系统调用功能的子程序编造入口地址表，对应系统子程序 7. 在进入系统调用处理前后，陷入处理机构还需保存和恢复处理机现场

第三章 进程线程模型

本章重难点分析 1. 进程的定义

2. 进程控制

3. 线程的引入及基本概念

第一节 进程基本概念

一、进程的定义

定义：进程是具有一定独立功能的程序在某个数据集合上的一次运行活动，是系统进行资源分配和调度的一个独立单位

进程分为系统进程和用户进程两类 系统进程执行操作系统程序

用户进程运行用户程序

系统进程的优先级通常高于一般用户进程的优先级 1. 进程与程序的联系和区别

1) 进程和程序的联系

程序是构成进程的组成部分之一

进程是由程序、数据和进程控制块（PCB）三部分组成的 一个进程的运行目标是执行它所对应的程序

2) 进程和程序的区别

程序是静态的，而进程是动态的 进程是程序的一个执行过程

程序的存在是永久的，进程存在生命周期，一个程序可以产生多个进程 2.可再入程序

一个能够被多个用户同时调用的程序称作是“可再入”的程序 可再入程序在执行中不会修改自身的代码

一个程序不是任何条件下都可以产生多个进程的

一个能被多个用户同时调用的程序，在执行中自身不能改变 3.进程的特征

进程的两个基本属性：

进程是一个可拥有资源的独立单位

进程同时又是一个可以独立调度和分派的基本单位 进程具有以下特性：

1) 并发性：一个进程可以同其他进程一道向前推进

2) 动态性：进程有其生命周期，且进程的状态是不断变化的

3) 独立性：一个进程是一个相对完整的资源分配单位

4) 交往性：一个进程在运行过程中可能会与其他进程发生直接的或间接的相互作用

5) 异步性：每个进程按照各自独立的、不可预知的速度向前推进

6) 结构性：一个进程由程序、数据和进程控制块三部分组成

二、进程状态及状态转换 1.三状态进程模型

运行中的进程可以处于以下三种状态之一：运行、就绪、等待 三状态模型状态定义：

（1）运行状态（Running）

进程已获得 CPU，并且在 CPU 上执行的状态

（2）就绪状态（Ready）

进程已经具备运行条件，但没有获得 CPU 的状态

（3）阻塞状态（Blocked）

进程因等待某种事件发生而暂时不能运行的状态 三状态模型的状态转换条件：

（1）就绪→运行

进程调度程序把处理机

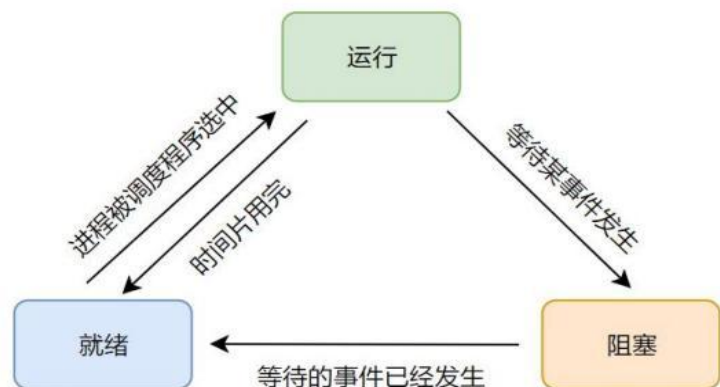
分配给某个就绪进程 （2）运行→就绪

规定的运行时间片用完 （3）运行→阻塞

运行状态的进程等待其他资源

（4）阻塞→就绪

阻塞进程在其被阻塞的原因获得时解除阻塞



2.五状态进程模型:

- (1) 运行状态 (Running): 进程正在占用 CPU 资源
- (2) 就绪状态 (Ready): 进程等待分配 CPU 资源
- (3) 阻塞状态 (Blocked): 进程因等待 I/O 操作等条件而暂停运行
- (4) 创建状态 (New): 进程正在创建过程中, 还不能运行
- (5) 结束状态 (Exit): 进程已结束运行, 回收除进程控制块之外的其他资源, 让其他进程 从进程控制块中收集有关信息

五状态状态转换条件:

- (1) 创建新进程: 进入创建状态
- (2) 提交 (Admit): 完成一个新进程的创建过程, 进入就绪状态
- (3) 调度运行 (Dispatch): 进入运行状态
- (4) 释放 (Release): 进程终止运行, 进入退出状态
- (5) 超时 (Timeout): 用完时间片, 进程暂停运行, 从运行状态进入就绪状态
- (6) 事件等待 (Event Wait): 进程要求的事件未出现而进入阻塞状态
- (7) 事件出现 (Event Occurs): 进程等待的事件出现, 进程从阻塞状态进入就绪状态

七状态模型引入原因:

五状态进程模型没有区分进程地址空间位于内存还是外存 低优先级进程对换至外存, 这种做法可得到的好处:

- (1) 提高处理机效率
- (2) 可为运行进程提供足够内存
- (3) 有利于调试

七状态模型状态定义:

与五状态进程模型相比, 增加了就绪挂起和阻塞挂起两个状态

- (1) 阻塞挂起状态 (Blocked, suspend): 进程在外存并等待某事件的出现
- (2) 就绪挂起状态 (Ready, suspend):

进程在外存, 但只要进入内存, 即可运行 七状态模型状态转换条件:

- (1) 挂起 (Suspend):

进程从内存转到外存 阻塞到阻塞挂起

就绪到就绪挂起 运行到就绪挂起

- (2) 激活 (Activate): 进程从外存转到内存 就绪挂起到就绪

阻塞挂起到阻塞

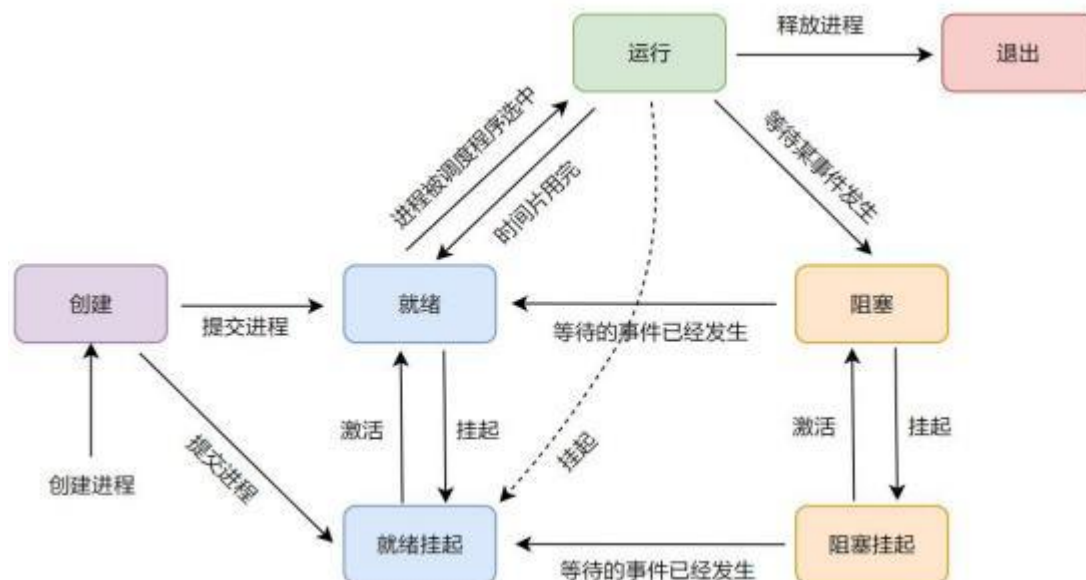
- (3) 事件出现 (Event Occurs): 等待的事件出现

阻塞到就绪

阻塞挂起到就绪挂起 (4) 提交 (Admit):

完成创建过程

进入就绪状态或就绪挂起状态



三、进程控制块

进程控制块（PCB：Process Control Block）：

描述进程的基本情况以及进程的运行变化过程 PCB 是进程存在的唯一标志

当系统创建一个进程时，为进程设置一个 PCB，再利用 PCB 对进程进行控制和管理 撤消进程时，系统收回它的 PCB，进程也随之消亡

1.PCB 的内容

PCB 的内容可以分成调度信息和现场信息两大部分

调度信息：供进程调度时使用，描述进程当前所处的状况

进程名、进程号、存储信息、优先级、当前状态、资源清单、“家族”关系、消息队列 指针、进程队列指针和当前打开文件等

现场信息：刻画进程的运行情况

程序状态字、时钟、界地址寄存器等

一旦中断进程的运行，必须把中断时刻的内容记入进程控制块的现场信息 2.进程的组成

进程由程序、数据和进程控制块三部分组成 PCB 存有进程的地址信息

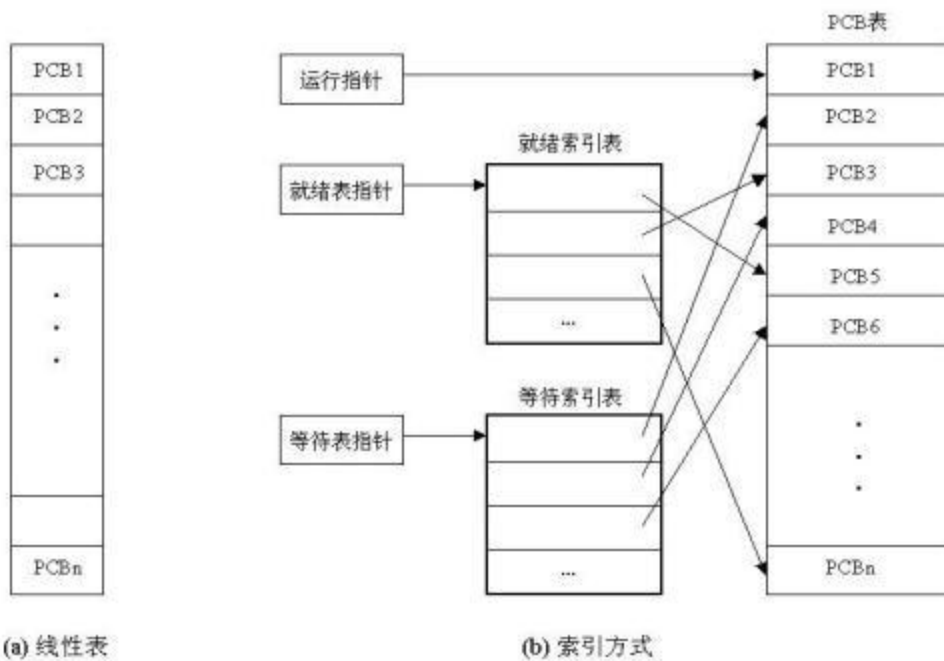
程序描述了进程要实现的功能 数据是程序操作的对象

3.PCB 组织

（1）线性方式：所有 PCB 不分状态组织在一个线性表（称 PCB 表）中 优点：简单，且不需要额外的开销

缺点：需要扫描整个 PCB 表，才能找到需要的 PCB

（2）索引方式：相同状态的进程，分别设置 PCB 索引表



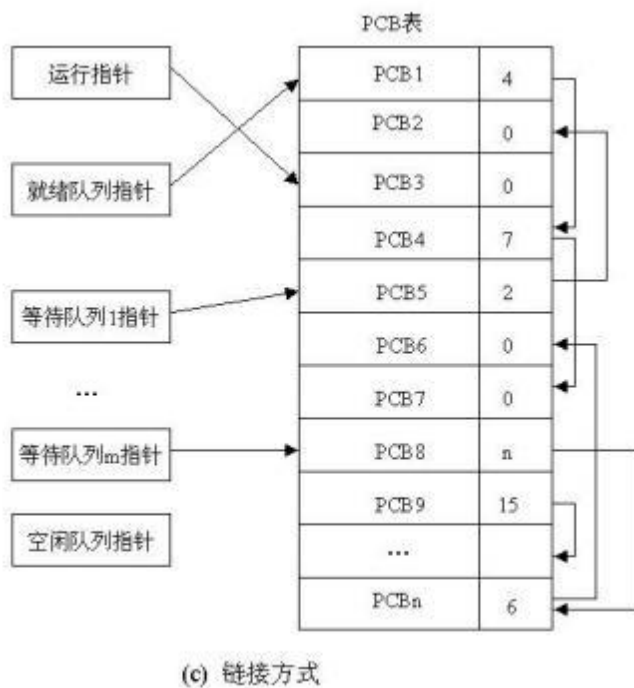
(3) 链接方式:

对于具有相同状态进程的 PCB,

通过 PCB 中的链接字构成一个队列

链接字指出本队列下一 PCB 在 PCB 表中的编号 (或地址) 编号为 0 表示队尾

队首由内存固定单元中相应的队列指针指示



4. 进程的队列

(1) 就绪队列

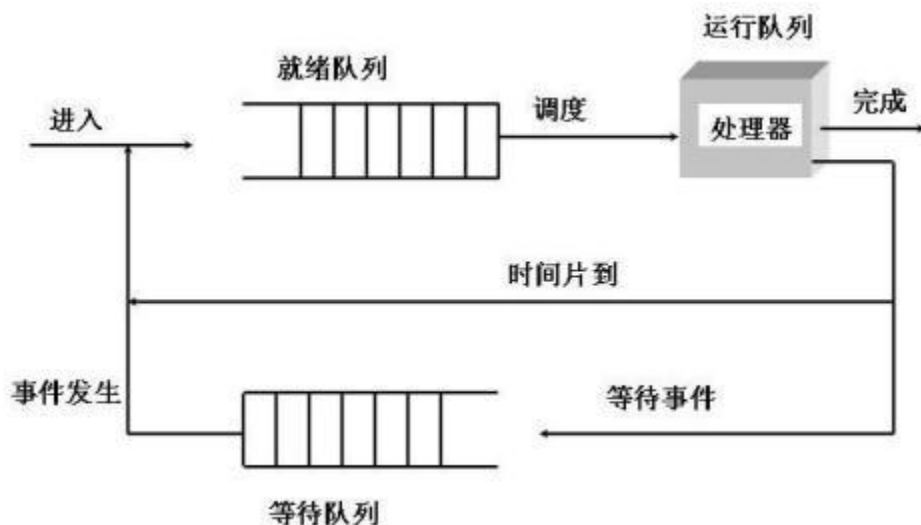
所有就绪状态的进程排在一个就绪队列中

(2) 等待队列

对每一种等待事件组织一个队列

(3) 运行队列

在单 CPU 系统中，整个系统有一个运行队列



5. 进程队列的组成 单向链接:

同一队列中的进程，通过进程控制块中的队列指针联系起来

前一进程的进程控制块中的指针值为它的下一个进程的进程控制块的地址

队列中最后一个进程的进程控制块中的指针值置为“0” 双向链接:

设置两个指针，前向指针和后向指针

分别指出它的前一个或后一个进程的进程控制块地址

系统为每个队列设置一个队首指针，指出该队列的第一个和最后一个进程的进程控制块地址，以便进行双向搜索

第二节 进程状态及状态转换

一、进程控制基本概念 进程控制:

对进程在整个生命周期中各种状态之间的转换进行有效的控制，通过进程控制原语来实现。

原语:

若干条指令所组成的一个指令序列，用来实现某个特定的操作功能，连续的，具有不可分割性，在执行时也不可间断，必须在管态下执行，并且常驻内存，用于进程控制的原语：创建进程、撤消进程、挂起进程、激活进程、阻塞进程、唤醒进程以及改变进程优先级等

1. 创建原语
创建一个新的进程，前者称为父进程，后者称为子进程 建立进程控制块 PCB

2. 撤消原语

撤消进程的 PCB

撤消属于该进程的一切“子孙进程”，释放被撤消进程所占用的全部资源

3. 阻塞原语
进程从运行状态转换为阻塞状态

中断 CPU 的执行，把 CPU 的当前状态保存在 PCB 的现场信息中 当前状态置为等待状态，并把它插入到该事件的等待队列中去 4.唤醒原语

等待状态转换为就绪状态

从等待队列中撤出并插入到就绪队列中排队，等待调度执行

二、Linux 操作系统有关进程控制的系统调用

在 Linux 操作系统中，常用的有关进程控制的系统调用有：fork，exec，wait，exit，getpid，sleep 和 nice 等 下表所示为这些系统调用的功能说明

系统调用	功能
fork	创建一个子进程
getpid	返回当前进程的PID
exec	更换进程映像，即根据指定的文件名找到可执行文件，并用它来取代调用进程的内容
exit	终止调用的程序（用于程序运行出错）
wait	等待任何要僵死的子进程
sleep	使进程挂起指定的时间
nice	改变进程的优先级

第三节 线程引入及基本概念

一、线程的引入

进程活动存在时间和空间的资源开销，数目不宜太多，切换频率不宜过高

使多个程序更好地并发执行，尽量减少系统的开销，分离调度的基本单位和独立分配资源的单位

调度和分派的基本单位不负责分配资源 拥有资源的基本单位，不频繁地切换

1.什么是线程？

线程是进程中的一个实体，是 CPU 调度和分派的基本单位，只拥有少量在运行中必不可少的资源

同属一个进程的其他线程共享进程所拥有的全部资源线程有就绪、等待和运行三种基本状态，有的系统中线程还有终止状态等

2.线程的属性

（1）唯一的标识符和一张线程描述表

线程描述表记录了线程执行的寄存器以及栈等现场状态

（2）不同的线程可以执行相同的程序

（3）同一个进程中的各个线程共享该进程的内存地址空间

（4）线程是处理器的独立调度单位，多个线程是可以并发执行的

（5）线程在生命周期内会经历等待状态、就绪态和运行态等各种状态变化 3.引入线程的好处

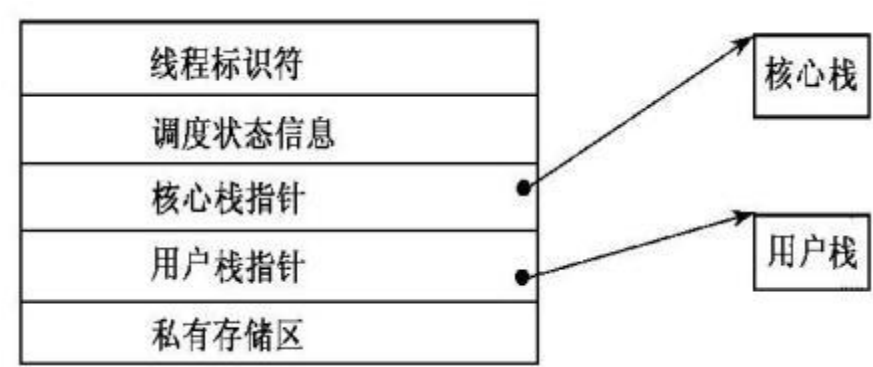
（1）创建一个新线程花费时间少

创建线程的速度比创建进程的速度快，且系统的开销也少

- (2) 线程之间的切换花费时间少
 - (3) 同一个进程内的线程共享内存和文件，通信更简便，信息传送速度也快
 - (4) 线程能独立执行，充分利用和发挥处理器与外部设备并行工作能力
- 二、线程的组成

thread 结构，即线程控制块，由以下四个基本部分组成：

- ①一个唯一的线程标识符
- ②描述处理器工作情况的一组寄存器的内容
如程序计数器、状态寄存器、通用寄存器等
- ③两个栈指针
一个指向核心栈；另一个指向用户栈
- ④一个私有存储区
用来存放现场保护信息和其他与该线程相关的统计信息等



一个进程可以包含一个线程或多个线程
当一个进程包含多个线程时，这些线程除各自私有少量资源以外，共享所属进程的全部资源

三、线程与进程的关系

线程又称为轻量级进程或者进程元
传统的进程称为重量级进程

通常一个进程都有若干个线程

1.调度
传统的操作系统中拥有资源的基本单位和独立调度、分派的基本单位都是进程引入线程的操作系统中，线程作为调度和分派的基本单位，进程作为资源拥有的基本单位

2.并发性

引入线程的操作系统中不仅进程之间可以并发执行，一个进程中的多个线程之间也可以并发执行

3.拥有资源

不论是传统的操作系统，还有线程的操作系统
进程都是拥有资源的一个独立单位

4.系统开销
在创建或撤消进程时，操作系统所付出的开销将显著地大于在创建或撤消线程时的开销，进程切换的开销远大于线程切换的开销，同一进程中的多个线程具有相同的地址空间，同步和通信的实现比较容易。

第四节 线程的实现和实例

一、线程的实现方式

1. 用户级线程、核心级线程和混合方式

线程已在许多系统中实现，但实现的方式并不完全相同 用户级线程（**User-Level Threads**），不依赖于内核
例如：数据库系统中的线程

内核级线程（**Kernel-Supported Threads**），依赖于内核 混合方式实现线程，即同时实现了以上两种类型的线程

（1）用户级线程 实现方式：

创建、撤消和切换都不利用系统调用 管理线程的工作全部由应用程序完成 每个进程都有一个私有的线程表
核心空间中有一个进程表，记载系统中各个进程的情况

优缺点 优点：

线程的切换速度快

允许采用适合自己要求的不同调度算法

可以运行在任何操作系统上 缺点：

当一个线程执行系统调用时，在同一个进程内的所有线程都被阻塞 多线程应用程序不具有多处理器的优点

（2）核心级线程 实现方式：

创建、撤消和切换都由核心实现 核心保留了一个线程控制块，

根据该控制块感知线程的存在并对线程进行控制

线程表在核心空间中，记载系统中所有线程的情况 核心空间中保存一个进程表，

记载系统所有进程的信息

核心进行调度时以线程为基本单位

优缺点 优点：

核心可以同时调度同一进程中的多个线程，真正实现并行操作

如果一个进程的某个线程阻塞了，核心可以调度同一个进程中的另一个线程

核心线程本身也可以是多线程的 缺点：

控制转移开销大

应用进程无法影响线程的切换 （3）混合方式

用户级线程和核心级线程两种实现方式结合 同一个进程内的多个线程可在多个处理器上 并行运行

阻塞式系统调用不必将整个进程阻塞 线程创建在用户空间完成

线程调度等在核心态完成 2. 线程实现方式的比较

（1）线程的调度与切换速度

用户级线程的切换速度更加快

（2）系统调用

用户级线程调用一个系统调用时，把系统调用看作是整个进程的行为 内核支持线程，则调度是以线程为单位

（3）线程执行时间

用户级线程，调度是以进程为单位进行的 核心级线程，调度是以线程为单位进行的

二、Pthread 线程库

多线程应用程序用一组用户级程序库来编写，将所有线程映射到一个单独的内核级进程中 其中最著名的是：

Pthread (POSIX thread) 库

Pthread 线程共有特性： 一个标识符

一组寄存器（包括程序计数器）

一组存储在结构中的属性，包括栈大小、调度参数以及其它线程需要的项目

三、协程 提出原因：

当线程数量非常多的时候，系统线程会占用非常多的内存空间 过多的线程切换会占用大量的系统时间

解决方法：协程机制

协程是运行在线程之上的轻量级线程，当一个协程执行完成后，让另一个协程运行在当 前线程之上，协程并没有增加线程数量，只是在线程的基础之上通过分时复用的方式运行多 个协程，协程的切换在用户态完成切换的代价比线程从用户态到内核态的代价小很多。

第四章 进程线程调度

本章重难点分析

1.进程调度的基本概念 2.经典进程调度算法

3.操作系统调度算法实例 4.实时调度算法

第一节 进程调度的基本概念

进程调度即处理机调度。进程调度的任务是控制、协调进程对 CPU 的竞争，按照一定的 调度算法，使某一就绪进程获得 CPU 的控制权，转换成运行状态。进程调度也叫低级调度

进程调度程序是操作系统的真正核心，它直接负责 CPU 的分配。系统中所有进程都是在 CPU 上运行的，进程调度程序就是它们的切换开关。

一、进程调度的主要功能

①保存现场

②挑选进程

③恢复现场

二、进程调度的时机

①创建进程

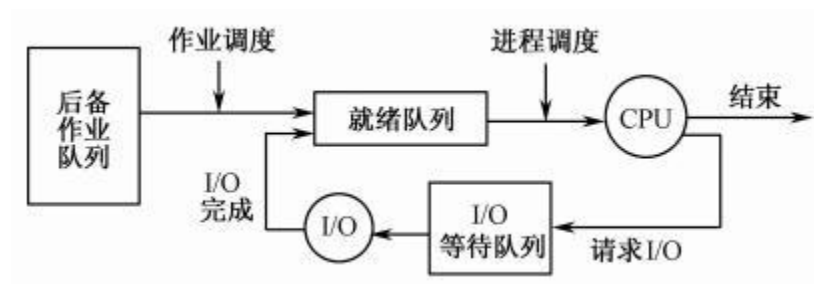
②任务完成

③等待资源

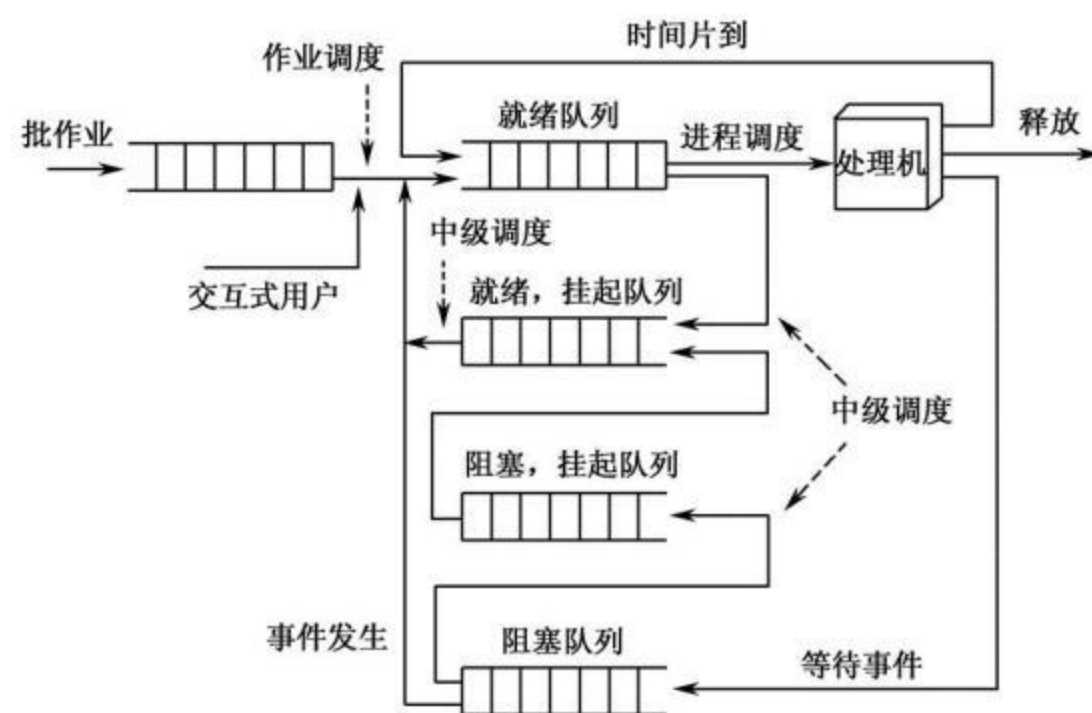
④中断发生

⑤运行到时

三、两级调度模型



四、三级调度模型



第二节 进程调度算法的设计思路

一、进程调度的主要功能

- ① 设计目标
- ② 公平性
- ③ 均衡性
- ④ 统筹兼顾
- ⑤ 优先级
- ⑥ 开销

二、性能评价标准

- ① CPU 利用率
- ② 吞吐量
- ③ 周转时间
- ④ 就绪等待时间

第三节 经典进程调度算法

一、先来先服务法

先来先服务（First-Come, First-Served, FCFS）法是最简单的一种调度算法

对于作业调度来说，每次调度从后备作业队列中选择队头的一个或几个作业，把它们调入内存，分配相应的资源，创建进程，然后把进程放入就绪队列

对于进程调度算法来说，即每次调度从就绪队列中选择一个最先进入该队列的进程，把 CPU 分给它，直至完成或者由于某些原因而阻塞，当新一个进程进入就绪队列时，它的 PCB 就链入就绪队列的末尾。每次进程调度时就把队头进程从该队列中摘下，分给它 CPU，使它运行

设有三个作业，编号分别为 1、2、3，且分别对应一个进程。各作业依次到达，相差一个时间单位。如图所示为采用FCFS 方式调度时这三个作业的执行顺序，其中，*表示作业到达的时间，实线表示作业执行过程。



FCFS 调度算法的性能如表所示。

作业	到达时间	运行时间	开始时间	完成时间	周转时间	带权周转时间
1	0	24	0	24	24	1
2	1	3	24	27	26	8.67
3	2	3	27	30	28	9.33
平均周转时间=26 平均带权周转时间=6.33						

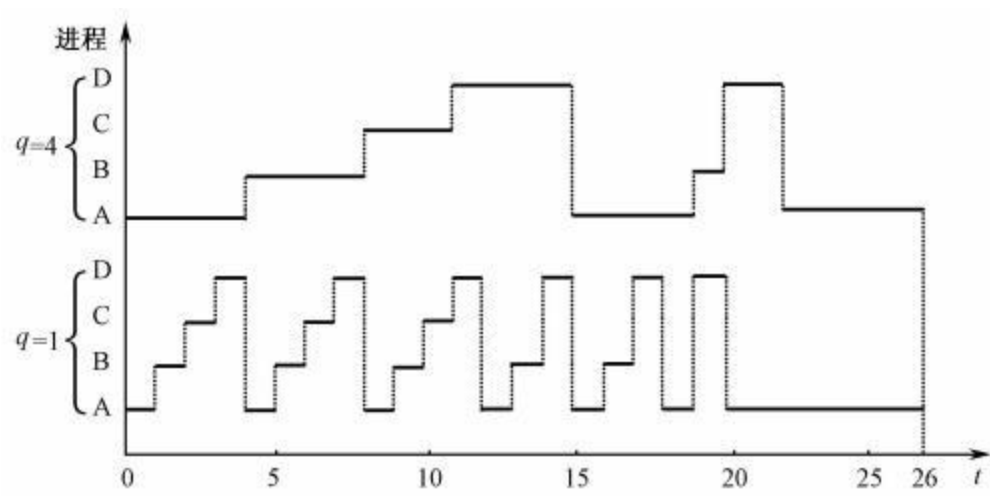
FCFS 算法比较有利于长作业（进程），而不利于短作业（进程）。因为短作业运行时间很短，如果让它等待较长时间才得到服务，它的带权周转时间就会很长。

另外，FCFS 调度算法对 CPU 繁忙型作业（指需要大量 CPU 时间进行计算的作业）较有利，而不利于 I/O 繁忙型作业（指需要频繁请求 I/O 的作业）。

FCFS 调度算法容易实现，但它的效率较低。

二、时间片轮转法

时间片轮转法（Round-Robin，RR）主要用于分时系统中的进程调度。当进程用完分给 它的时间片后，系统的计时器发出时钟中断，调度程序便停止该进程的运行，并把它放入就 绪队列的末尾；然后，把 CPU 分给就 绪队列的队首进程，同样也让它运行一个时间片，如此 往复



二、时间片轮转法

到达时间	进程名	到达时间	运行时间	开始时间	完成时间	周转时间	带 权 周 转 时间
时间片 q=1	A	0	12	0	26	26	2.17
	B	0	5	1	17	17	3.4
	C	0	3	2	11	11	3.67
	D	0	6	3	20	20	3.33
	平均周转时间=18.5 平均带权周转时间=3.14						
时间片 q=4	A	0	12	0	26	26	2.17
	B	0	5	4	20	20	4
	C	0	3	8	11	11	3.67
	D	0	6	11	22	22	3.67
	平均周转时间=19.75 平均带权周转时间=3.38						

时间片的大小对轮转法的性能有很大影响：

如果时间片太长，每个进程都在这段时间内运行完毕，那么时间片轮转法就退化为先来 先服务算法很显然，对 用户的响应时间必然加长；如果时间片太短，CPU 在进程间的切换工 作就非常频繁，从而导致系统开销增加

时间片的长短通常由以下四个因素来确定：

- ① 系统的响应时间
- ② 就绪队列进程的数目
- ③ 进程的转换时间
- ④ CPU 运行指令速度

三、优先级法

利用优先级调度算法进行进程调度时，是从就绪队列中选出优先级最高的进程，把 CPU 分给它使用如果在就绪队列中出现优先级更高的进程时，怎么办：

①非抢占式优先级法

②抢占式优先级法

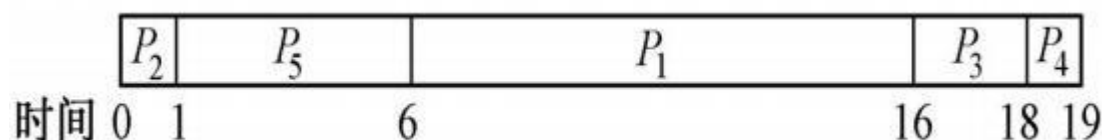
内部决定优先级是利用某些可度量的量来定义一个进程的优先级

外部优先级是按操作系统以外的标准设置的 两种确定进程优先级的方式：

①静态优先级是在创建进程时就确定下来的，而且在进程的整个运行期间保持不变

②动态优先级是随着进程的推进而不断改变的

进程	运行时间	优先数
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2



四、短作业优先法

从作业的后备队列中挑选那些需要运行时间最短的作业放入内存

短作业优先法能有效地降低作业的平均等待时间和提高系统的吞吐量 算法对长作业很不利，并且不能保证紧迫性作业会被及时处理

五、最短剩余时间优先法

最短剩余时间优先（Shortest Remaining Time First, SRTF）法是短作业优先法的变

形，它采用抢占式策略

六、多级队列法

多级队列（Multilevel Queue）调度算法是根据作业的某些特性，如占用内存大小和作业类型等，永久性地把各个作业分别链入不同的队列，每个队列都有自己的调度算法，在各个队列之间也要进行调度，通常采用固定优先级的抢占式调度

七、多级反馈队列法

多级反馈队列（Multilevel Feedback Queue）法是在多级队列法的基础上加进“反馈”措施。其实现思想是：系统中设置多个就绪队列，每个队列对应一个优先级，第一个队列的 优先级最高，第二个队列次之，以下各个队列的优先级逐个降低；各就绪队列中进程的运行 时间片不同，高优先级队列的时间片小，低优先级队列的时间片大，如从高到低依次加倍； 新进程进入系统后，先放入第一队列的末尾，各队列按 FCFS 方式排队；如某个进程在相应 时间片内没有完成工作，则把它转到下一级队列的末尾；系统先运行第一队列中的进程，

第一队列为空后，才运行第二队列中的进程，依次类推。最后一个队列（最低级）中的进程采用时间片轮转的方式进行调度。

多级反馈队列法虽然比较复杂一些，但具有较好的性能，在 UNIX 系统，Windows NT 和 OS/2 中都采用了类似的调度算法。

第四节 其他进程调度算法

一、公平共享调度

到此为止讲述的所有调度算法都是把就绪进程集合看做是单一的进程池，从这个进程池中选择下一个要运行的进程。该池可以按优先级划分成几个，但它们都是同构的。

但是，在多用户系统中，如果单个用户的应用程序或作业可以组成多个进程（或线程），就会出现传统的调度器不认识的进程集合结构。从用户的角度看，他所关心的不是某个特定的进程如何执行，而是构成应用程序的一组进程如何执行。因此，基于进程组的调度决策是非常具有吸引力的。该方法通常称作公平共享调度（fair-share scheduling）。此外，即使每个用户用一个进程表示，这个概念可以扩展到用户组。例如，在分时系统中，可能希望把某个部门的所有用户看作是同一个组中的成员，然后进行调度决策，并给每个组中的用户提供类似的服务。因此，如果同一个部门中的大量用户登录到系统，则希望响应时间效果的降低主要影响到该部门的成员，而不会影响其他部门的用户。

术语“公平共享”表明了这类调度器的基本原则。每个用户被指定了某种类型的权值，该权值定义了该用户对系统资源的共享，而且是作为在所有使用的资源中所占的比例来体现。特别地，每个用户被分配了处理器的共享。这种方案或多或少以线性的方式操作，如果用户 A 的权值是用户 B 的 2 倍，那么从长期运行的结果来看，用户 A 可以完成的工作应该是用户 B 的 2 倍。公平共享调度器的目标是监视使用情况，对那些相对于公平共享的用户占有

较多资源的用户，调度器分配以较少的资源，相对于公平共享的用户占有较少资源的用户，调度器分配以较多的资源。

二、保证调度

保证调度算法的目标是保证每个进程享用 CPU 的时间完全一样，即如果系统里一共有 n 个进程，则每个进程占用 CPU 的时间为 $1/n$ 。保障调度就是保障每个进程使用 $1/n$ 的 CPU 时间。保障就是肯定 $1/n$ 的时间运转，而不是大概 $1/n$ 时间运转。那么保障调度和轮转调度是一样吗？时间片轮转能不能达到 $1/n$ 的效果？关键就是达到 $1/n$ 不一定要靠轮转。轮转是能够达到 $1/n$ 的，但是保障调度不一定要轮转。每次给的时间片不一定要一样。

三、彩票调度

彩票调度算法是一种概率调度算法。你买过彩票就知道，你买的越多中奖的概率越大。在该算法里，给每个进程分发一定数量的彩票，而调度器则从所有彩票里随机抽取一张彩票，持有该彩票的进程就获得 CPU。这样，如果想让某个进程获得更多的 CPU 时间，我们可以给该进程多发几张彩票。彩票算法的优越性是显而易见的，通过给每个进程至少一张彩票就可以防止饥饿，因为该进程获得 CPU 的概率将大于 0。

第五节 操作系统调度算法实例

一、BSD 多级反馈队列法

BSD UNIX 系统主要用于分时交互环境中，调度算法设计成为交互用户提供好的响应时间，同时保证低优先级的后台作业不会饿死。

BSD UNIX 系统采用了多级反馈队列法，在每个优先级队列中采用了轮转的方法

$$CPU_j(i) = \frac{CPU_j(i-1)}{2}$$
$$P_j(i) = Base_j + \frac{CPU_j(i)}{2} + nice_j$$

$CPU_j(i)$ ：进程 j 在区间 i 中处理器使用情况的度量

$P_j(i)$ ：进程 j 在区间 i 开始处的优先级；值越小表示的优先级最高 $Base_j$ ：进程 j 的基本优先级

$nice_j$ ：用户可控制的调节因子

二、UNIX SVR4 调度

UNIX SVR4 中使用的调度算法是对早期 UNIX 系统所使用的调度算法的全面检查。新算法被设计成给实时进程最高的优先权，给内核模式的进程次高的优先权，给其他用户模式的进程（称作分时进程）最低的优先权。

SVR4 中实现的两个重要的修改为：

- 1.增加了可抢占的静态优先级调度器，引进了 160 种优先级，并划分到三个优先级类中。
- 2.插入了可抢占点。由于基本内核是不能被抢占的，它只能划分成许多个处理步骤，每一步都必须一直运行到完成，中间不会被中断。在处理步骤之间，有一个称作可抢占点的安全位置，在这里，内核可以安全地中断处理，并调度一个新进程。安全位置定义成一个代码区域，在这里所有的内核数据结构或者是已经更新且一致的，或者通过一个信号量被锁定。

SVR4 中定义了 160 个优先级，每个进程定义成属于这三类优先级中的一类，并被指定为具有该类中的一个优先级。这三类优先级为：

- 实时（159~100）：具有这些优先级的进程可以保证在任何内核进程或分时进程之前被选择运行。此外，实时进程可以使用可抢占点抢占内核进程和用户进程。
- 内核（99~66）：具有这些优先级的进程保证在任何分时进程之前被选择运行，但必须服从实时进程。
- 分时（59~0）：最低优先级的进程，通常是除了实时应用程序以外的用户应用程序。

每个优先级都关联着一个调度队列，在某一给定优先级的进程按循环方式执行。有一个位映像向量 $dqactmap$ ，它的每一位对应于各个优先级。

如果某个优先级上的队列不为空，则相应的位被置为 1。当一个正在运行的进程由于阻塞、时间片到期或抢占等原因离开运行状态时，调度器检查 $dqactmap$ ，并从优先级最高的非空队列中调度一个就绪进程。此外，当到达一个定义的可抢占点时，内核检查一个称作 $kprunrun$ 的标记，如果它被置位，则表明至少有一个实时进程处于就绪状态，如果当前进程的优先级低于优先级最高的实时就绪进程，则内核抢占当前进程。

在分时类中，进程的优先级是可变的。每当一个进程用完它的时间片时，调度器降低它的优先级；如果一个进程在一个事件或资源上阻塞时，则调度器提高它的优先级。分配给分时进程的时间量取决于它的优先级，其范围从给优先级 0 分配的 100ms 到给优先级 59 分配的 50ms。每个实时进程都有一个固定的优先级和固定的时间量。

三、Linux 抢占式调度

Linux 系统中的进程，既可以在用户态下运行，又可以在核心态下运行。而核心态的权限高于用户态的权限。进程每次执行系统调用时，其运行方式就会发生变化，从用户态切换到核心态

1.调度方式

Linux 系统的调度方式基本上采用“抢占式优先级”方式

Linux 系统中的调度基本上继承了UNIX 系统的以优先级为基础的调度

Linux 系统针对不同类别的进程提供了三种不同的调度策略，即 SCHED_FIFO，SCHED_RR 及 SCHED_OTHER

3.调度时机

- ① 当前进程调用系统调用 `nanosleep()` 或者 `pause()`，使自己进入睡眠状态，主动让出一段时间的 CPU 的使用权
- ② 进程终止，永久地放弃对 CPU 的使用
- ③ 在时钟中断处理程序执行过程中，发现当前进程连续运行的时间过长
- ④ 当唤醒一个睡眠进程时，发现被唤醒的进程比当前进程更有资格运行
- ⑤ 一个进程通过执行系统调用来改变调度策略或者降低自身的优先级（如 `nice` 命令），从而引起立即调度

4.调度算法

首先查找所有在就绪队列中的进程，从中选出优先级最高且在内存的一个进程。如果队列中有实时进程，则实时进程将优先运行。如果最需要运行的进程不是当前进程，则当前进程就被挂起，并且保存它的现场——所涉及的一切机器状态，包括程序计数器和CPU 寄存器等，然后为选中的进程恢复运行现场。

四、Windows 调度

Windows 实现了可抢占式调度器，它具有灵活的优先级系统，在每一级上都包括了轮转调度方法，在某些级上，优先级可以基于它们当前的线程活动而动态变化

Windows 中的优先级被组织成两段：实时和可变

在实时优先级类中，所有线程具有固定的优先级，并且它们的优先级永远不会改变，某一给定优先级的所有活动线程在一个轮转队列中

在可变优先级类中，一个线程的优先级在开始时是最初指定的值，但在它的生命周期中可能会发生变化，上升或者下降

第六节 多处理器调度算法

松耦合、分布式多处理器、集群 专门功能的处理器

紧耦合多处理 一、粒度

无约束并行性

粗粒度和非常粗粒度并行性

中等粒度并行性 细粒度的并行性 二、设计问题

把进程分配到处理器

在单个处理器上使用多道程序设计 一个进程的实际分派

三、进程调度

在大多数传统的多处理器系统中，进程并不是被指定到一个专门的处理器。不是所有处理器只有一个队列，或者使用某种类型的优先级方案，而是有多条基于优先级的队列，并且都送进相同的处理器池中。在任何情况下，都可以把系统看作是多服务器排队结构

四、线程调度

在单处理器中，线程可以用作辅助构造程序，并在处理过程中重叠执行 I/O。由于在进行线程切换时的性能损失远远小于进程切换的开销，因此可以用很少的代价实现这些优点

在多处理器系统中，线程的全部能力得到了更好的展现。在这个环境中，线程可以用于开发应用程序中真正的并行性

加载共享 组调度

专用处理器分配 动态调度

第七节 实时调度算法

一、实时调度算法概述

(1) 一个系统是否执行可调度性分析

(2) 如果执行，它是静态的还是动态的

(3) 分析结果自身是否根据在运行时分派的任务产生一个调度或计划 静态表驱动法

静态优先级驱动抢占法 基于动态规划调度法

动态尽力调度法

静态表驱动调度适用于周期性的任务

静态优先级驱动抢占调度与大多数非实时多道程序系统中的优先级驱动的抢占式调度所用的机制相同

基于动态规划调度，在一个任务已到达但未执行时，试图创建一个包含前面被调度任务和新到达任务的调度

如果新到达的任务可以按这种方式调度：满足它的最后期限，但是当前被调度的任务也不会错过它的最后期限，则修订这个调度以适应新任务

二、限期调度

大多数当代实时操作系统的设计目标是尽可能快速地启动实时任务，因此强调快速中断处理和任务分派。事实上，在评估实时操作系统时，并没有一个特别有用的度量

就绪时间

启动最后期限 完成最后期限

处理时间 资源需求 优先级

子任务结构

当考虑到最后期限时，实时调度功能可以分成许多维：下一次调度哪个任务以及允许哪 种类型的抢占另一个重要的设计问题是抢占

三、速率单调调度

为周期性任务解决多任务调度冲突的一个非常好的方法是速率单调调度 RMS

优先级最高的任务是周期最短的任务，优先级次高的任务是周期次短的任务，依次类推。当有多个任务可供执行时，周期最短的任务首先得到服务

四、优先级逆转

优先级逆转（**priority inversion**）在任何基于优先级的可抢占的调度方案中都能发生 的一种现象，但是它与实时调度的上下文关联特别大

在任何优先级调度方案中，系统应该不停地执行具有最高优先级的任务

一个更加严重的情况被称之为无界限优先级逆转（**Unbounded priority inversion**）， 在这种情况下，优先级逆转的持续时间不仅依赖于处理共享资源的时间，而且也依赖于其他 不相关任务的不可预测的行为

优先级继承协议的基本思想是优先级较低的任务继承任何与它共享同一个资源的优先 级较高的任务的优先级。

当高优先级的任务在资源上阻塞的时候，优先级立即更改。当资源被低优先级的任务释 放时这个改变结束。

第五章 存储管理

本章重难点分析 1.紧缩技术；

2.交换技术；

3.虚拟存储技术； 4.TLB；

5.页面置换算法（FIFO、LRU）、缺页率。

第一节 概述

存储器分成两类：

内存储器（简称内存） 外存储器（简称外存）

处理器可以直接访问内存，但不能直接访问外存。处理器要通过启动相应的输入/输出 设备后才能使外存与内存交换信息。

在操作系统中，管理内存的部分称为存储管理，是操作系统的重要部分之一。对内存管 理的好坏直接影响到计算机系统工作性能的高低。本章将介绍存储管理的基本思想。

一、存储体系

在计算机系统中，存储器是处理器处理的信息的来源与归宿， 占据着重要地位。



上述存储体系包含：

1.若干兆字节、中等速度、中等价格、内容易变的内存 RAM，通常是 GB 的数量级； 2.低速、价廉、内容不易变的磁盘，通常是 TB 的数量级；

3.少量的、非常快速、昂贵、内容易变的高速缓存 Cache，通常是 MB 的数量级。

快速存储设备和大容量存储设备必须构成为统一的整体，由操作系统协调这些存储器的使用。

对于内存速度和容量的要求：

- 1) 尽量快到与处理器存取速度相匹配。
- 2) 能装下当前运行的程序与数据各种速度和容量的存储器硬件，在操作系统协调之下形成了一种存储器层次结构，或称存储体系。

二、存储管理的任务

存储管理直接影响系统性能。

存储器由内存（主存）和外存组成。

所谓内存空间，是由存储单元（字节或字）组成的一维连续的地址空间。内存空间的任务：

用来存储当前正在运行程序的代码及数据，是程序中指令本身地址所指的也就是程序计数器所指的存储器。

内存空间分为两部分：

- 1) 系统区，用以存储操作系统常驻内存部分，用户不能占用这部分空间；
- 2) 用户区，分配给用户使用，用于装入并存储用户程序和数据，这部分的信息随时都在发生变化。

内存管理的要求（9 点）

- 1) 充分利用内存，为多道程序并发执行提供存储基础。
- 2) 尽可能方便用户使用。
 - ①操作系统自动装入用户程序。
 - ②用户程序中不必考虑硬件细节。
- 3) 系统能够解决程序空间比实际内存空间大的问题。
- 4) 程序的长度在执行时可以动态伸缩。
- 5) 内存存取速度快。
- 6) 存储保护与安全。
- 7) 共享与通信。
- 8) 及时了解有关资源的使用状况。
- 9) 实现的性能和代价合理。 1.内存分配和回收

内存分配和回收应该具有的功能：

- 1) 记住每个存储区域的状态。
- 2) 实施分配（静态和动态）。
- 3) 回收。

内存分配和回收采用的方法---内存分配表

①位示图表示法：将内存按序划分为固定大小的区域，称为内存块，用一位（Bit）表示一个内存块的状态（0表示空闲，称为空闲块；1表示占用，称为已用块）

②链表法：将已分配的内存空间区域（一个或多个）首地址和长度组成一条记录，并将所有已分配的记录按序组成链表，形成已分配内存链表。将未分配的内存空间区域（一个或多个）首地址和长度组成一条记录，并将所有分配的记录按序组成链表，形成空闲内存链表。分配时，操作系统查找空闲链表；回收时，操作系统合并未空闲链表

内存分配有两种方式：

- 1) 静态分配
- 2) 动态分配

动态分配的优势：

- 1) 动态存储分配具有较大的灵活性。
- 2) 对提高内存的利用率大有好处。
- 3) 反映了程序的动态性，较之静态存储分配更为合理。

2.内存共享

所谓内存共享是指两个或多个进程共用内存中相同区域。

- 1) 能使多道程序动态地共享内存，提高内存利用率；
- 2) 能共享内存中某个区域的信息。

共享的内容包括代码共享和数据共享，特别是代码共享要求代码必须是纯代码。

存储共享的目的：

- 1) 通过代码共享节省内存空间，提高内存利用率
- 2) 通过数据共享实现进程通信

保护策略：

（1）地址越界保护

（2）权限保护

①对属于自己区域的信息，可读可写。

②对公共区域中允许共享的信息或获得授权可使用的信息，可读而不可修改。

③对未获授权使用信息，不可读、不可写。

3.“扩充”内存容量

具体实现是在硬件支持下，软件、硬件相互协作，将内存和外存结合起来统一使用。借助虚拟存储技术或其他交换技术，达到在逻辑上扩充内存容量的目的。

三、地址转换

存储器以字节（每个字节为8个二进制位）为编址单位，每个字节都有一个地址与其对应。

假定存储器的容量为n个字节，其地址编号顺序为0，1，2，...，n-1。这些地址称为内存的

“绝对地址”，由绝对地址对应的内存空间称为“物理地址空间”。

用户程序中使用的地址称为“逻辑地址”，由逻辑地址对应的存储空间称为“逻辑地址空间”。

地址重定位

每个逻辑地址在内存中也没有一个固定的绝对地址与之对应。为了保证程序的正确执行，必须根据分配给程序的内存区域对程序中指令和数据的存储地址进行重定位，即要把逻辑地址转换成绝对地址。把逻辑地址转换成绝对地址的工作称“地址重定位”或“地址转换”，又称“地址映射重定位的方式可以有“静态重定位”和“动态重定位”两种。

静态重定位

地址转换工作是在程序开始执行前集中完成的，所以在程序执行过程中就无须再进行地址转换工作，这种地址转换方式称“静态重定位”。

动态重定位

在程序执行过程中，每当执行一条指令时都由硬件的地址转换机构将指令中的逻辑地址转换成绝对地址。这种方式的地址转换是在程序执行时动态完成的，故称为动态重定位。

第二节 分区管理方案

分区管理是能满足多道程序运行的最简单的存储管理方案。其基本思想是把内存划分成若干个连续区域，称为分区，每个分区装入一个运行程序。

分区的方式可以归纳成固定分区和可变分区两类

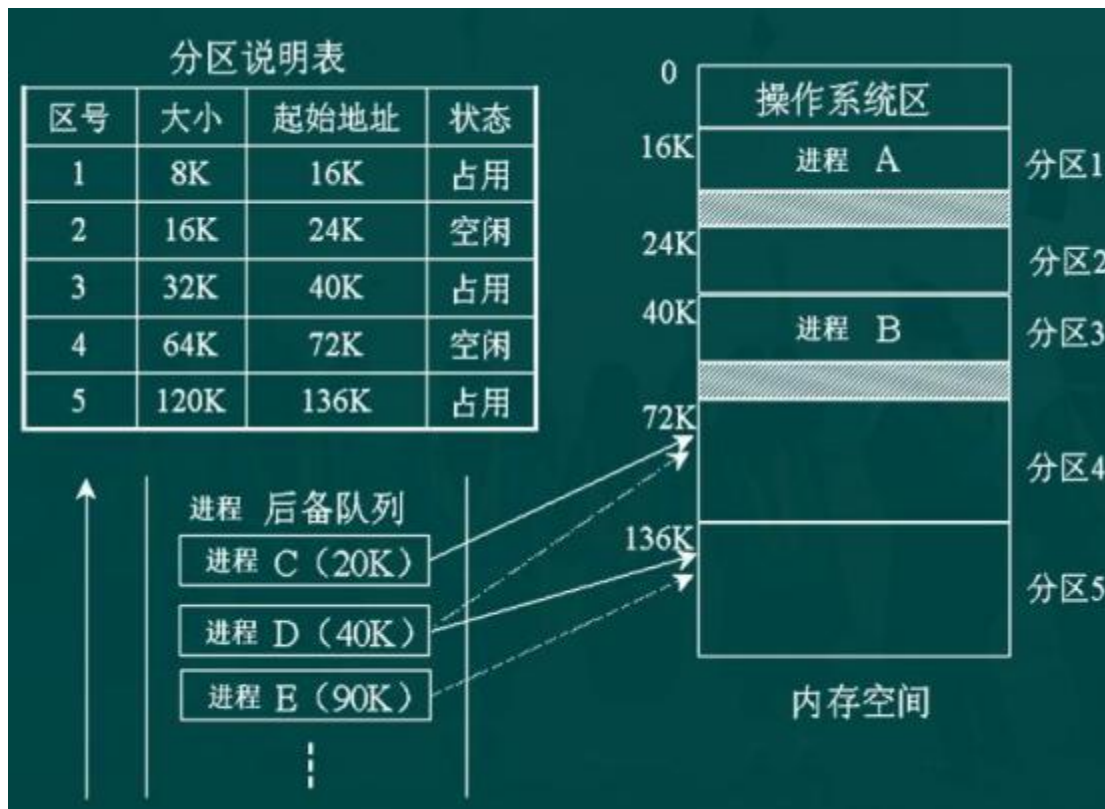
一、固定分区 1.基本思想

固定分区是指系统先把内存划分成若干个大小固定的分区，一旦划分好，在系统运行期间便不再重新划分。

为了满足不同程序的存储要求，各分区的大小可以不同。程序运行时必须提供对内存资源的最大申请量。

2.内存分配表与分区的分配、回收

内存分配表：分区序号、分区大小、分区起始地址以及使用状态（空闲或占用）。



总结：

- 1) 可以使多个程序共存于内存中。
- 2) 不管采用哪种分配策略，都不能充分利用内存。一个程序的大小，不可能刚好等于某个分区的大小。
- 3) 固定分区方案灵活性差，可接纳程序的大小受到了分区大小的严格限制。

二、可变分区 1.基本思想

可变分区是指系统不预先划分固定分区，而是在装入程序时划分。

内存分区，使为程序分配的分区的大小正好等于该程序的需求量，且分区的个数是可变的。

优点：

- 1) 可变分区有较大的灵活性。
- 2) 较之固定分区能获得较好的内存利用率。



2.紧缩技术

内存经过一段时间的分配回收后，会存在很多很小的空闲块，这些空闲块被称为碎片。

解决碎片问题的办法是在适当时刻进行碎片整理，通过移动内存中的程序，把所有空闲碎片合并成一个连续的大空闲区且放在内存的一端，而把所有程序占用区放在内存的另一

端，这一技术称为“紧缩技术”，或“压缩技术”。紧缩技术的优点：

- 1) 集中分散的空闲区；
- 2) 提高内存的利用率；
- 3) 便于进程动态扩充内存

采用紧缩技术要注意以下问题：

- 1) 紧缩技术会增加系统的开销。
- 2) 移动是有条件的。

在采用紧缩技术时，应该尽可能减少需要移动的进程数和信息量。

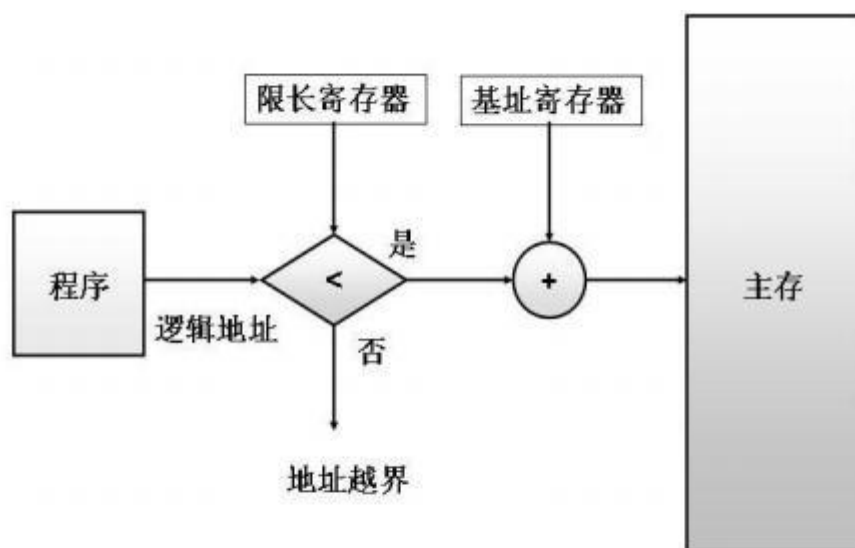
3.可变分区的实现

(1) 采用可变分区方式管理时，要有硬件的地址转换机构作支持。限长寄存器用来存储程序所占分区的长度。

基址寄存器用来存储程序所占分区的起始地址。

程序执行过程中，处理器每执行一条指令都要取出该指令中的逻辑地址，当逻辑地址小于限长寄存器中的限长值时，则逻辑地址加基址寄存器值就可得到绝对地址。

当逻辑地址大于限长寄存器中的限长值时，表示欲访问的内存地址超出了所分配的分区范围。这时就不允许访问，而形成一个“地址越界”的程序性中断事件。



已分配区表，记录已装入的程序在内存中占用分区的起始地址和长度，用标志位指出占用分区的程序名。

空闲区表，记录内存中可供分配的空闲区的起始地址和长度，用标志位指出该分区是未分配的空闲区。

4.空闲分区的分配策略

(1) 最先适应算法

最先适应算法，又称顺序分配算法，当接到内存申请时，顺序查找分区说明表，找到第一个满足申请长度的空闲区，将其分割并分配。

此算法简单，可以快速做出分配决定

(2) 最优适应算法

最优适应算法，当接到内存申请时，查找分区说明表，找到第一个能满足申请长度的最小空闲区，将其分割并分配。优点：最节约空间；

缺点：可能会形成碎片。

(3) 最坏适应算法

最坏适应算法，当接到内存申请时，查找分区说明表，找到能满足申请要求的最大的空闲区。

基本思想：在大空闲区中装入信息后，分割剩下的空闲区相对也很大，还能用于装入其他程序。

优点：可以避免形成碎片；

缺点：分割了大的空闲区后，如果再遇到较大的程序申请内存时，无法满足要求的可能性较大。

5.分区的回收

当用户程序执行结束后，系统要回收已使用完毕的分区，将其记录在空闲区表中。步骤：

首先检查是否有与归还区相邻的空闲区，即检查相邻的空闲区表中标志为“未分配”的栏目，确定是否有相邻空闲区，（上邻、下邻）应合并成一个空闲区，登记。

二、可变分区

假定进程归还的分区起始地址为 S ，长度为 L 。应考虑如下四种可能性。

1) 回收分区上邻分区是空闲的，需要将两个空闲区合并成一个更大的空闲区，然后修改空闲区表。

计算公式：起始地址不变，长度=原长度+ L

2) 回收分区下邻分区是空闲的，需要将两个空闲区合并成一个更大的空闲区，然后修改空闲区表。

计算方法：起始地址= S

长度=原长度+ L

3) 回收分区上邻和下邻分区都是空闲的，需要将三个空闲区合并成一个更大的空闲区，然后修改空闲区表。计算方法：

第 i 栏起始地址不变；

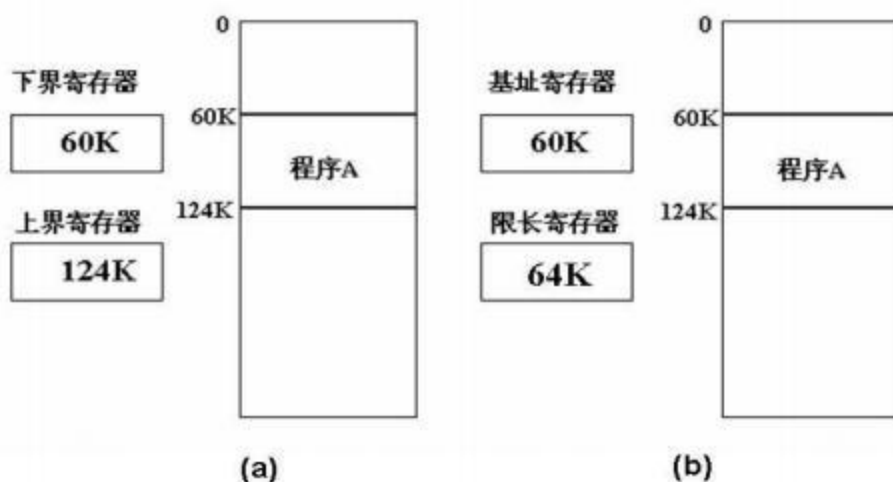
第 i 栏长度为 i 栏中原长度+ k 栏中长度+ L ；第 k 栏的标志应修改成“空”状态

4) 回收分区上邻和下邻分区都不是空闲的，则直接将空闲分区记录在空闲区表中。

找一个标志为“空”的登记栏，把归还区的始址和长度登记入表，把该栏目中的标志位修改成“未分配”。

6.分区保护

1) 一种是系统设置界限寄存器



2) 保护键方法

即为每个分区分配一个保护键，相当于一把锁。同时为每个进程分配一个相应的保护键，相当于一把钥匙，存放在程序状态字中。

每当访问内存时，都要检查钥匙和锁是否匹配，若不匹配，将发出保护性中断。

三、分区管理方案的优缺点

分区管理是实现多道程序设计的一种简单易行的存储管理技术。优点：

- 1) 算法比较简单
 - 2) 实现起来比较容易
 - 3) 内存额外开销较少
 - 4) 存储保护措施也很简单
 - 5) 在内存利用率方面，可变分区的内存利用率比固定分区高
- 缺点：

- 1) 内存使用仍不充分，并且存在着较为严重的碎片问题
- 2) 浪费了处理器时间
- 3) 不能为用户提供“虚存”，即不能实现对内存的“扩充”，受到物理存储器实际存储容量的限制
- 4) 内存中可能包含有一些实际不使用的信息

第三节 覆盖与交换技术

无论内存空间有多大，程序所需要的空间往往总是比可用的内存空间更大。

为了“扩充”内存，可以把进程地址空间中信息（指令和数据）的一部分放在外存上，而把那些当前需要的执行程序段和数据段放在内存中。

这样，在内、外存之间就会出现一个信息交换的问题。

本节所介绍的覆盖技术（**Overlay**）和交换技术（**Swapping**）就是用于控制这种交换的。覆盖技术同交换技术的主要区别是控制交换的方式不同，前者主要用在早期的系统中，而后者目前则主要用于小型分时系统。

一、覆盖技术

覆盖技术是指一个程序的若干程序段，或几个程序的某些部分共享某一个存储空间。

覆盖技术不需要操作系统的特殊支持，可以完全由用户实现，即覆盖技术是用户程序自己附加的控制。

覆盖技术可以由编译程序提供支持。覆盖可以从用户级彻底解决内存小装不下程序的问题。

覆盖技术是早期采用的简单的扩充内存的技术，对用户不透明，增加了用户的负担，要求用户清楚地了解程序的结构，并指定各程序段调入内存的先后次序，以及内存中可以覆盖掉的程序段的位置等。而且程序段的最大长度仍受内存容量的限制。通常，覆盖技术主要用于系统程序的内存管理上，因为系统软件设计者容易了解系统程序的覆盖结构。

二、交换技术

交换技术又称对换技术。

进程从内存移到磁盘，并再移回内存称为交换。

交换技术是进程在内存与外存之间的动态调度，是由操作系统控制的。交换技术多用于分时系统，大多数现代操作系统使用交换技术。

交换技术支持多道程序设计、也是虚拟存储技术（下一节）基础。

交换技术支持多道程序设计、也是虚拟存储技术（下一节）基础。使用交换技术需要考虑的相关问题：

(1) 换出进程的选择：时间片轮转法、基于优先数的调度算法。

(2) 交换时机的确定：内存空间不够或者内存空间有不够的危险时。

(3) 交换空间的分配：当进程在内存中时，不再为它分配磁盘空间。当它被换出时，必须为它分配磁盘交换空间。

(4) 换入进程换回内存时位置的确定：绝对地址：一定要在原来的位置上。

相对地址：可再进行地址重定位，可以不在原来的位置上。

缺点：交换时需要大量的处理器时间，影响对用户的响应时间。减少交换的信息量是交换技术的关键问题。

主要特点：打破了一个程序一旦进入内存便一直运行到结束的限制 与覆盖技术相比：

1) 交换技术对用户而言是透明的。

2) 交换可以发生在不同的进程或程序之间，而覆盖发生在同一进程或程序内部，而且只能覆盖那些与覆盖段无关的程序段。

因此，交换技术比覆盖技术更加广泛地用于现代操作系统。覆盖技术与交换技术的发展 导致了虚拟存储技术的出现。

第四节 虚拟页式存储管理方案

分区方式管理内存：每道程序都占用内存的一个或几个连续的存储空间。当内存中无足够大的连续空间时，程序就无法装入，必须移动已在内存中的某些程序后才能再装入新的程序。如何解决程序规模大于内存的问题？

覆盖技术是由程序员指定，系统辅助完成的，对程序员的要求甚高，而且容易出错。将这类工作全部交给操作系统完成。这就产生了虚拟存储技术。

虚拟存储技术：可以把一个逻辑地址连续的程序分散存储到几个不连续的内存区域中，并且保证程序的正确执行，则既可充分利用内存空间，又可减少移动所花费的开销。

页式存储管理就是这样一种有效的管理方式。

本节会介绍虚拟存储技术的基本思想并重点介绍将虚拟存储技术与页式存储管理方案 结合后的一种典型的、大多数操作系统采用的虚拟页式存储管理方案。

一、虚拟存储技术

虚拟存储技术的基本思想：利用大容量的外存来扩充内存，产生一个比有限的实际内存空间 大得多的、逻辑的虚拟内存空间，简称虚存，以便能够有效地支持多道程序系统的实现和大型程序运行的需要，从而增强系统的处理能力。

实现虚拟存储器需要以下的硬件支持：

1) 系统有容量足够大的外存。

2) 系统有一定容量的内存。

3) 最主要的是，硬件提供实现虚-实地址映射的机制 虚拟存储器的工作原理：

当进程开始运行时，先将一部分程序装入内存，另一部分暂时留在外存；当要执行的指令不在内存时，由系统自动完成将它们从外存调入内存的工作；当没有足够的内存空间时，系统自动选择部分内存空间，将其中原有

的内容交换到磁盘上，并释放这些内存空间供其他 进程使用。这样做的结果使程序的运行丝毫不受影响，使程序在运行中感觉到拥有一个不受 内存容量约束的、虚拟的、能够满足自己需求的存储器。

交换技术：交换到外存上的对象一般都是进程，也就是说交换技术是以进程为单位进行的。 虚拟存储技术：一般是以页为单位。

二、虚拟页式存储管理

支持页式存储管理的硬件部件通常称为“存储管理部件 ”（MemoryManagement Unit， MMU）。

存储管理部件首先把内存分成大小相等的许多区，把每个区称为“物理页面 ”，物理页 面是进行内存空间分配的物理单位。

同时，要求程序中的逻辑地址也进行分页，页的大小与物理页面的大小一致。“逻辑地 址 ”可被称为虚拟地址。

页式存储器提供编程使用的虚拟地址由两部分组成：

虚拟页号 页内地址

页式存储的地址结构确定了物理页面的大小。

用户编辑时无须考虑如何分页的问题，仍使用连续的虚拟地址即可。 三、页式存储管理中物理内存的分配与回收

内存分配表中具有三种标识： 已分配、未分配、剩余空闲物理页面数。

简单的内存分配表可以用一张“位示图 ”构成。

假设内存的可分配区域被分成 256 个物理页面，则可用字长为 32 位的8 个字作为“位 示图 ”。位示图中的每一位与一个物理页面对应，每一位的值可以是 0 或 1，0 表示对应的物 理页面为空闲，1 表示已占用。

三、页式存储管理中物理内存的分配与回收 内存分配方案：

- 1) 查看空闲物理页面数是否能满足程序要求。
- 2) 若能满足，则根据需求从位示图中找出一些为 0 的位，把这些位置成 1。
- 3) 从空闲页面数中减去本次分配的页面数。
- 4) 按照找到的位计算出对应的物理页面号。

当找到一个为 0 的位后，根据它所在的字号、位号，按如下公式就可计算出对应的块号： 物理页面号=字号×字长+位号

把程序装入到这些物理页面中，并为该程序建立页表。 内存的归还：

- 1) 根据归还的物理页面号计算出该物理页面在位示图中对应的位置。
- 2) 将占用标志修改成 0。
- 3) 再把回收的物理页面数加入到空闲物理页面数中。 假定归还块的块号为，则在位示图中对应的位置为：

字号= $\lfloor i/\text{字长} \rfloor$ ，位号= $i \bmod \text{字长}$ 四、虚拟页式存储地址转换过程

1.页式存储管理的地址转换

页表控制寄存器（页表始址寄存器、页表长度寄存器）+高速缓冲 存储器的支持。

- 1) 页表始址寄存器，用于保存正在运行进程的页表在内存的首地址；
- 2) 页表长度寄存器，用于保存正在运行进程的页表的长度。 页表：
 - 1) 指出该程序虚拟地址中的页号与所占用的物理页面号之间的对应关系。
 - 2) 页表又是硬件进行地址转换的依据，每执行一条指令时按虚拟地址中的页号查页表。 物理地址的计算公式为：

物理地址=物理页面号×块长+页内地址

需要强调的是，物理页面号又称为页帧和页框号 2.页表项

在虚拟页式存储管理中，页表项的设计如下。

物理页面号：页面在内存中时所对应的物理页面号。

有效位：又称驻留位、存在位，表示该页是在内存还是在磁盘。访问位：访问位表示该页在内存期间是否被访问过。

修改位：修改位表示该页在内存中是否被修改过。保护位：是否能读/写。

其中访问位和修改位可以用来决定置换哪个页面，具体由页面置换算法决定。 3.页表

（1）多级页表

大多数操作系统中采用二级页表，即由表页和页目录一起构成进程页表。

第一级表示页目录，保存页表页的地址；第二级表示页表页，保存物理页面号。

（2）散列页表

当地址空间大于 32 位时，一种常见的方法是使用以页号为散列值的散列页表。其中每个表项都包含一个链表，该链表中元素的散列值都指向同一个位置。

散列页表中的每个表项都包含三个字段：（a）虚拟页号；

（b）所映射的页框号；

（c）指向链表中下一个元素的指针。

（3）反置页表

每个物理页框对应一个表项。每个表项包含与该页框相对应的虚拟页面地址，以及拥有该页面进程的信息。

特点：

1）整个系统中只存在一个页表，并且每个页框对应其中一个表项。

2）需要在反置页表中存放地址空间标志符。 4.转换检测缓冲区（TLB）

按给定的虚拟地址进行读写时，必须访问两次内存。第一次按页号读出页表中对应的块号。

第二次按计算出来的绝对地址进行读写。

两次访问内存显然延长了指令的执行周期，降低了执行速度。

为了提高存取速度，有两种方法：

1）在地址映射机制中增加一组高速寄存器保存页表。需要硬件支持，经济上不可行。

2）在地址映射机制中增加一个小容量的联想寄存器（相联存储器），它由高速缓冲存储器组成。

假定访问内存的时间是 200 毫微米，访问高速缓冲存储器的时间为 40 毫微米，高速缓冲存储器为 16 个单元，查 TLB 的命中率为 90%。

按虚拟地址转换成绝对地址进行存取的平均访问时间为： $(200+40) \times 90\% + (200+200) \times 10\% = 256$ （毫微米）

不使用 TLB 需要两次访问内存的时间： $200 \times 2 = 400$ 毫微米。可见使用 TLB 访问时间下降了 36%。

5.缺页异常处理

若在页表中发现所要访问的页面不在内存，则产生缺页异常：

1）操作系统必须在内存中选择一个页面将其移出内存。

2）页面在内存期间已经被修改过，就必须把它写回磁盘。

3) 该页没有被修改过, 不需要写回, 调入的页直接覆盖被淘汰的页。整个缺页处理过程简单阐述如下。

1) 根据当前执行指令中的逻辑地址查页表的有效位, 判断该页是否在内存。

2) 该页标志为“0”, 形成缺页异常。保留现场。

3) 操作系统处理缺页异常, 寻找一个空闲的页面。

4) 若有空闲页, 则把磁盘上读出的信息装入该页面中。

5) 修改页表及内存分配表, 表示该页已在内存。

6) 如果内存中无空闲页, 则按某种算法选择一个已在内存的页面, 把它暂时调出内存。

7) 恢复现场, 重新执行被中断的指令。

6. 页面调度策略

虚拟存储器系统通常定义三种策略来规定如何(或何时)进行页面调度: 调入策略、置页策略和置换策略。

(1) 调入策略

虚拟存储器的调入策略决定了什么时候将一个页由外存调入内存之中。

①请求调页(Demand Paging): 只调入发生缺页时所需的页面。优点: 实现简单。

缺点: 容易产生较多的缺页, 造成对外存 I/O 次数多, 时间开销过大, 容易产生颠簸现象。②预调页(Prepaging): 在发生缺页需要调入某页时, 一次调入该页以及相邻的几个页。

优点: 提高了调页的 I/O 效率, 减少了 I/O 次数。

缺点: 造成浪费。这种方式常在程序装入时使用。关于调入页面来源, 通常有以下两种做法:

1) 进程装入时, 将其全部页面复制到交换区, 以后总是从交换区调入。

2) 凡是未被修改的页面, 都直接从文件区读入, 而被置换时不需调出; 已被修改的页面, 被置换时需调出到交换区, 以后从交换区调入。

(2) 置页策略

当线程产生缺页时, 内存管理器还必须确定将调入的虚拟页放在物理内存的何处。用于确定最佳位置的一组规则称为“置页策略”。

(3) 置换策略

如果缺页发生时物理内存已满, “置换策略”被用于确定哪个虚页面必须从内存中移出, 为新的页面腾出空位。在虚拟页式存储管理方案中, 可采用两种分配策略, 即固定和可变分配策略。

在进行置换时, 也可以采用两种策略, 即全局置换和局部置换。将它们组合起来, 有如下三种策略

①固定分配局部置换(Fixed Allocation, Local Replacement)。

②可变分配全局置换(Variable Allocation, Global Replacement)。

③可变分配局部置换(Variable Allocation, Local Replacement)。

7. 页面置换算法

如果刚被调出的页面又立即要用, 因而又要把它装入, 而装入不久又被选中调出, 调出不久又被装入, 如此反复, 使调度非常频繁。这种现象称为“抖动”或称“颠簸”。

常用的页面置换算法:

(1) 理想页面置换算法(Optimal replacement, OPT)——最佳 OPT 算法淘汰以后不再需要的或者在最长时间以后才会用到的页面。这一算法一般不可能实现, 但它可以作为衡量其他页面淘汰算法优劣的一个标准。

(2) 先进先出页面置换算法(First-In First-Out, FIFO) 先进先出页面置换算法总是选择最先装入内存的一页调出, 或者说是把驻留在内存中时间最长的一页调出。

优点：FIFO 算法简单，容易实现。

缺点：最先装入内存的一页，不等于说这一页是不常使用的。很少使用纯 FIFO 算法 （3）第二次机会页面置换算法

检查进入内存时间最久页面的R 位，如果是 0，那么这个页面既老又没有被使用，可以 立刻置换掉；如果是 1，就将 R 位清 0，并把该页面放到链表的尾端，修改其进入时间，然 后继续搜索。

这一算法称为第二次机会（Second Chance）算法，其基本思想是寻找一个最近的时钟 间隔以来没有被访问过的页面。

（4）时钟页面置换算法（Clock）

一个更好的办法是把所有的页面都保存在一个类似时钟面的环形链表中，一个表针指向 最老的页面。当发生缺页时，算法首先检查表针指向的页面，如果它的R 位是 0 就置换该页 面，并把新的页面插入这个位置，然后把表针前移一个位置；如果 R 位是 1 就清除 R 位并把 表针前移一个位置，重复这个过程直到找到了一个 R 位为 0 的页面为止。由于这个算法的工 作方式，就将它称为时钟（clock）算法。

（5）最近最少使用页面置换算法（Least Recently Used, LRU）

在缺页发生时，首先淘汰掉最长时间未被使用过的页面。这个策略称为 LRU 页面置 换算法。最近最少使用页面置换算法总是选择距离现在最长时间没有被访问过的页面先调出。 这种实现方法必须对每一页的访问情况时时刻刻地加以记录和更新，实现起来开销比较大， 但 LRU 算法是在效果上最接近 OPT 算法的算法。

8.缺页率

假定一个程序共有 n 页，系统分配给它的物理页面是 m 个（ m 、 n 均为正整数，且 $1 \leq m \leq n$ ）。

因此，该程序最多有 m 页可同时被装入内存。如果程序执行中访问页面的总次数为 A ， 其中有 F 次访问的页面尚未装入内存，故产生了 F 次缺页异常。

现定义： $f=F/A$ 把 f 称为“缺页率”

显然，缺页率与缺页异常的次数有关。影响缺页率的因素如下（4 点）：

（1）分配给程序的物理页面多，则同时装入内存的页面数就多，故减少了缺页异常的 次数，也就降低了缺页率。反之，缺页率就高。

（2）页面的大小

虚拟页面的大小取决于物理页面的大小，物理页面大则虚拟页面也大，每个页面大了 则 程序的页面数就少。装入程序时是按页存储在内存中的，因此，装入一页的信息量就大， 就 减少了缺页异常的次数，降低了缺页率。反之，若页面小则缺页率就高。

（3）程序编制方法

怎样编制程序也是值得探讨的，程序编制的方法不同，对缺页异常的次数有很大影响。 （4）页面调度算法
页面调度算法对缺页率的影响也很大，调度不好就会出现“颠簸”

理想的调度算法（OPT）能使缺页率最低，采用FIFO 调度算法产生的缺页率约为最佳算 法的 3 倍。

五、虚拟页式存储管理的优缺点

优点： 由于它不要求进程的程序段和数据在内存中连续存放，从而有效地解决了碎片问题。 这既提高了内存的利用率，又有利于组织多道程序执行。

缺点：存在页面空间的浪费问题。这是由于各种程序代码的长度是各不相同的，但页面的大小是固定的，所以在每个程序的最后一页内总有一部分空间得不到利用。如果页面较大，则由此引起的存储空间的损失仍然较大。

六、虚拟页式存储管理的性能问题

引入虚拟存储管理，把内存和外存统一管理，其真正目的是把那些访问概率非常高的页放入内存，减少内外存交换的次数。在虚存中，页面可能在内存与外存之间频繁地调度，有可能出现抖动或颠簸。

抖动或颠簸的原因：

- 1) 颠簸是由于缺页率高而引起的
- 2) 分配给一个进程的内存物理页面数太少

六、虚拟页式存储管理的性能问题

工作集模型，可以解决颠簸的问题：

对于给定的进程访页序列，从时刻 $(t - \Delta)$ 到时刻 t 之间所访问页面的集合，称为该进程的工作集。

其中， Δ 称为工作集窗口。

工作集是随时间而变化的，工作集大小与工作集窗口尺寸密切相关。

第六章 文件系统

本章重难点分析

- 1.文件管理的基本概念 2.文件目录
- 3.文件存储空间
- 4.实现文件系统的表目
- 5.文件系统的性能 6.文件共享保护
- 7.文件系统的性能

第一节 文件管理的基本概念

一、文件管理任务 1.文件的定义

文件：文件可以被解释为一组带标识的、在逻辑上有完整意义的信息项的序列 2.文件系统

文件系统：文件系统是操作系统中统一管理信息资源的一种软件。 文件系统的功能：

- (1) 统一管理文件的存储空间，实施存储空间的分配与回收
- (2) 实现文件从名字到外存地址空间的映射即实现文件的按名存取
- (3) 实现文件信息的共享，并提供文件的保护措施
- (4) 向用户提供一个方便使用的接口
- (5) 系统维护及向用户提供有关信息
- (6) 保持文件系统的执行效率文件系统在操作系统接口中占的比例最大
- (7) 提供 I/O 的统一接口

二、文件的存储介质及存取方式 1.外存储设备的特点

外存储设备通常由驱动部分和存储介质两部分组成。

存储介质又常被称为卷，“卷”字来自把存储介质看作“信息容器”的比喻。驱动器的作用是使计算机能够实现读写（及保存、控制、测试）存储介质上的内容。

2.外存储设备的存储介质

（1）磁带

磁带是最早使用的磁记录存储介质。显然，磁带是一种顺序存取设备，因为在磁带上，只有在前面的物理块被访问之后，才能存取后续的物理块。

磁带设备的优点是存储容量大。缺点：

- 1）磁带虽能保存大容量的信息，但存取速度太慢。
- 2）磁带只能进行顺序存取，这种设备不适合用于随机存取方式，故磁带主要用于后备存储和保存不经常使用的信息，或用作不同系统之间传递信息的一种介质。

（2）磁盘

磁盘分为软盘（已淘汰）和磁盘（又称“盘片”）。

磁盘由于其存储容量大以及成本低而得到广泛应用。

磁盘是一种典型的随机存取设备。磁盘设备允许文件系统直接存取磁盘上的任意物理块。磁盘的物理地址具有如下的形式：

柱面号（磁道号）、磁头号（盘面号）、扇区号

（3）光盘

光盘是利用在激光的作用下特性发生变化的一些材料所制成的非磁记录介质。

只读式光盘具有容量大（600MB 以上）、速度快（接近磁盘）、价格便宜等特点，但一般不可写。

（4）闪存

闪存是不易丢失存储器中的一种。优点：

- 1）电可擦除。
- 2）可随机存取的。
- 3）寿命和可靠性相当高。
- 4）闪存的读写比磁盘快而且方便。
- 5）闪存在擦除和重编程时并不需要额外的电源。

文件在存储设备中的存取方式

存储介质	磁带	磁盘		
物理结构	连续结构	连续结构	链接结构	索引结构域
存取方式	顺序存取	顺序存取	顺序存取	顺序存取
		随机存取		随机存取

三、文件的分类

为了有效、方便地管理文件，在文件系统中，常常把文件按其性质和用途的不同进行分类 1.按文件的用途分类：

(1) 系统文件：操作系统和各种系统应用程序和数据所组成的文件。

(2) 库函数文件：标准子程序及常用应用程序组成的文件允许用户对其进行读取、执行，但不允许对其进行修改。如 C 语言子程序库、FORTRAN 子程序库等。

(3) 用户文件：用户文件是用户委托文件系统保存的文件。 2.按文件的组织形式分类：

(1) 普通文件：最一般格式的文件，文件的内容是一般的数据或程序，例如由字符流组成的文件。普通文件既包括系统文件，也包括用户文件、库函数文件和用户实用程序文件等。

(2) 目录文件：目录文件是由文件的目录构成的特殊文件。

(3) 特殊文件：特殊文件是以文件形式存在和访问的设备，可进行查找目录等操作，但对文件的读写会对应于对设备的读写，并由设备驱动程序来完成具体的读写操作。

3.一些常见的文件分类方式

按文件的保护方式可划分为：只读文件、读写文件、可执行文件、无保护文件等 按信息的流向分类可划分为：输入文件、输出文件和输入输出文件等

按文件的存放时限可划分为：临时文件、永久文件和档案文件等

按文件所使用的介质类型分类可划分为：磁盘文件、磁带文件、卡片文件和打印文件等

上述种种文件系统的分类，其目的是：对不同文件进行管理，提高系统效率；同时，提高文件系统的用户界面友好性

4.UNIX 类操作系统中文件的分类

1) 普通文件。这是内部无结构的一串平滑的字符所组成的文件。

2) 目录文件。这是由文件目录项所构成的文件。

3) 特殊文件。在 UNIX 类操作系统中，把 I/O 设备也看成是一种文件——特殊文件。

第二节 文件的逻辑结构和物理结构

文件的逻辑结构就是从用户角度看文件，研究文件的组织形式。文件的物理结构，也就是文件在实际的存储空间存储时的结构。本节首先介绍文件的逻辑结构，然后介绍其物理结构

一、文件的逻辑结构

1.设计文件逻辑结构的原则

与文件的逻辑结构相联系的是逻辑文件的存取方式，即用户如何访问文件在文件系统设计时，到底选择何种逻辑结构才能更有利于用户对文件信息的操作呢？这里，我们列出在一般情况下，设计文件的逻辑结构时应遵循的一些设计原则：

(1) 易于操作

(2) 查找快捷

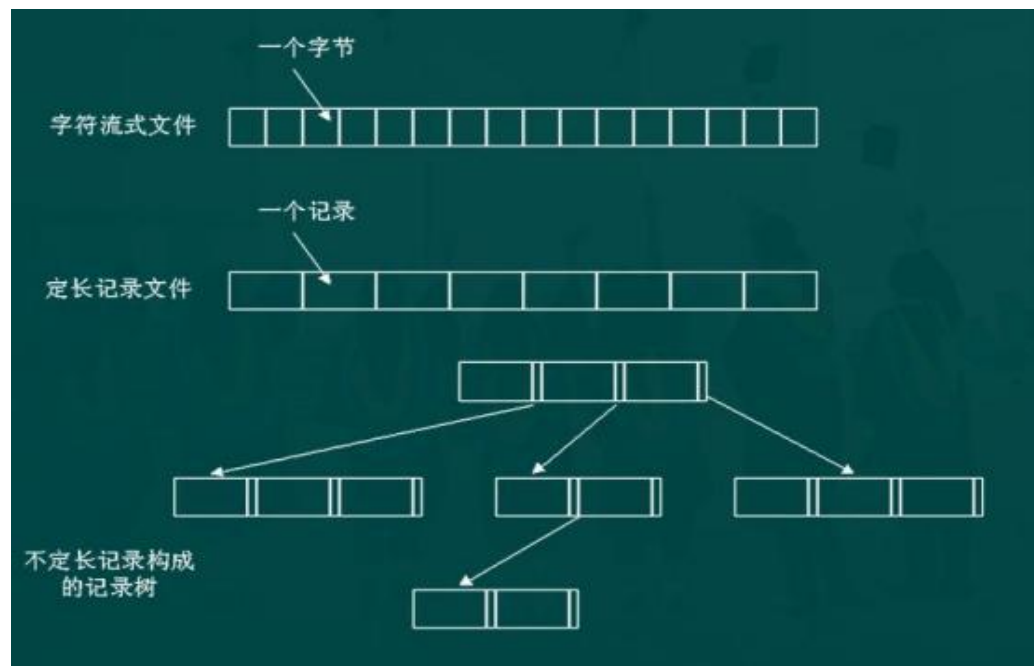
(3) 修改方便

(4) 空间紧凑

2.文件的逻辑结构

文件的逻辑结构就是用户所看到的文件的组织形式文件逻辑结构是一种经过抽象的结构，所描述的是文件中信息的组织形式，与文件在物理介质上的具体存储结构不同。

文件划分成三类逻辑结构：无结构的字符流式文件、定长记录文件和不定长记录文件构成的记录树，如图所示



定长记录文件和不定长记录文件可以统称为记录式文件

(1) 流式文件

流式文件是有序字符的集合

(2) 记录式文件

记录式文件是一组有序记录的集合

二、文件的物理结构

从研究文件管理、设计文件管理系统的角度来看，必须研究如何在物理存储器上存储文件，这是文件系统实现的物理基础

常用的文件物理结构有顺序结构、链接结构、索引结构

1.顺序结构

(1) 顺序结构原理

顺序结构又称连续结构，这是一种最简单的文件物理结构，它把逻辑上连续的文件信息依次存放在连续编号的物理块中。在顺序结构中，一个文件的目录项中只要指出该文件占据的总块数和起始块号即可。

(2) 顺序结构的优缺点

顺序结构的优点是，一旦知道了文件在文件存储设备上的起始块号和文件长度，就能很快地进行存取。这是因为从文件的逻辑块号到物理块号的变换是非常简单的。顺序结构支持

顺序存取和随机存取。

2.链接结构原理

(1) 链接结构原理

文件的链接结构的实质就是为每个文件构造所使用磁盘块的链表。

(2) 链接结构的优缺点

链接结构的优点是，存储碎片问题迎刃而解了，有利于文件动态扩充，有利于文件插入和删除，提高了磁盘空间利用率。

链接结构的主要缺点是，存取速度慢，不适于随机存取文件；磁盘的磁头移动多，效率相对较低存在文件的可靠性问题，比如指针出错，文件也就出错了；另外，链接指针需要占用一定的空间。

为了提高可靠性，可以采用双向链接，或者在每个物理块中存储文件名称和相对块号等办法改进。不过这些办法只能有限地提高可靠性，并不能从根本上解决问题，而且需要耗费更多的空间，典型的链接结构是 FAT 文件系统。

3.索引结构

（1）索引结构原理

索引结构的文件把每个物理盘块的指针字，集中存放在称为索引表的数据结构中的内存索引表中。在每个文件相应的目录条目中包括该文件的索引表地址，而索引表中的第 i 个条目指向文件的第 i 块，如图所示，要读某个文件的第 i 块，只需从该文件索引表的第 i 个条目中得到该文件块的地址即可。

（2）索引文件结构的优缺点

索引文件结构保持了链接结构的优点，又解决了其缺点索引结构文件既适于顺序存取，也适用于随机存取。索引文件可以满足文件动态增长的要求，也满足了文件插入、删除的要求索引文件还能充分利用外存空间。

索引结构的缺点是：会引起较多的寻道次数和寻道时间；索引表本身增加了存储空间开销索引表应该多大？应该定长还是变长？解决问题的办法有以下一些种类：

①索引表的链接模式：一个索引表通常就是一个物理盘块这样，读写索引表比较简单对大文件就用多个索引表并将之链接在一起

②多级索引：这是上述索引表链接模式的一种改善变种

三、UNIX 的三级索引结构

UNIX 操作系统的 i 结点是一种多级索引文件结构，是多级索引结构文件在 UNIX 中的具体实现。

i 结点的基本思想是，给每个文件赋予一张称为 i 结点的小表，在这张小表中列出了文件属性和文件中各块在磁盘上的地址。

第三节 文件目录

一、文件控制块

文件系统的一个特点是“按名存取”，文件控制块（FCB，FileControl Block），把所有文件的文件控制块有机地组织起来，就构成了文件控制块的一个有序集合，称为文件目录。

文件目录实际就是文件符号名到文件物理地址之间的一种映射机制。文件控制块 FCB 是系统为管理文件而设置的一个数据结构。

FCB 是文件存在的标志，它记录了系统管理文件所需要的全部信息。

文件控制块 FCB 是系统为管理文件而设置的一个数据结构 FCB 是文件存在的标志，它记录了系统管理文件所需要的全部信息。

FCB 包含的内容：文件名，文件代号，用户名，文件物理位置，文件长度，记录大小，文件类型，文件属性，共享说明，文件逻辑结构，文件物理结构，建立日期和时间，最后访问日期和时间最后修改日期和时间，口令，保存期限。

二、文件目录和当前目录

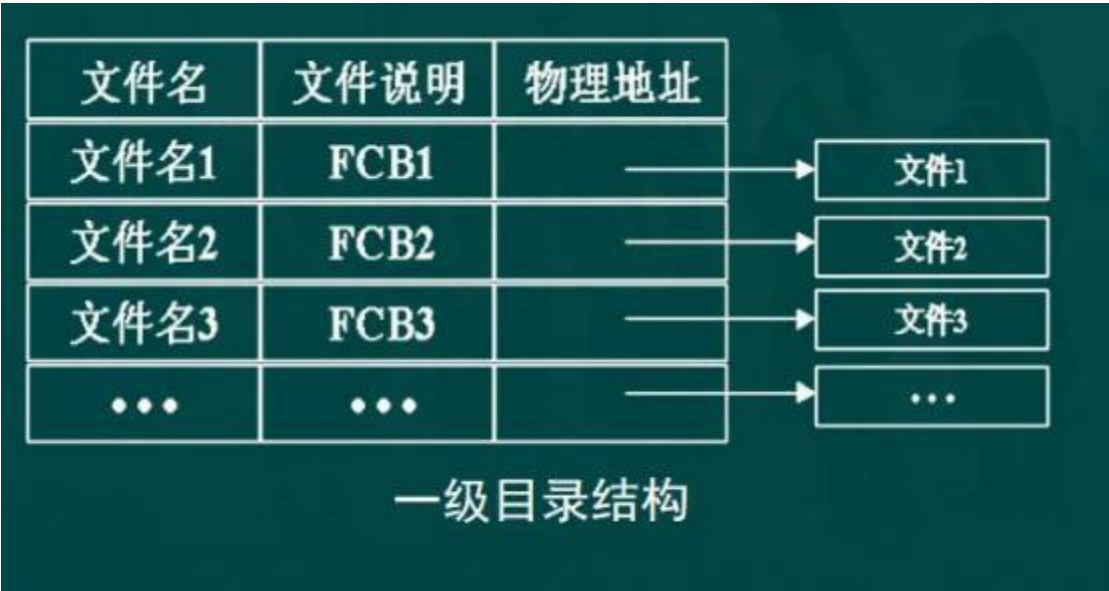
文件目录是实现用户按名存取文件的一种手段。

1. 目录结构

为了能够方便用户的检索和文件的管理，根据实际的需要，一般把文件目录设计成一级（单级）目录结构、二级目录结构和多级目录结构。

（1）一级目录结构

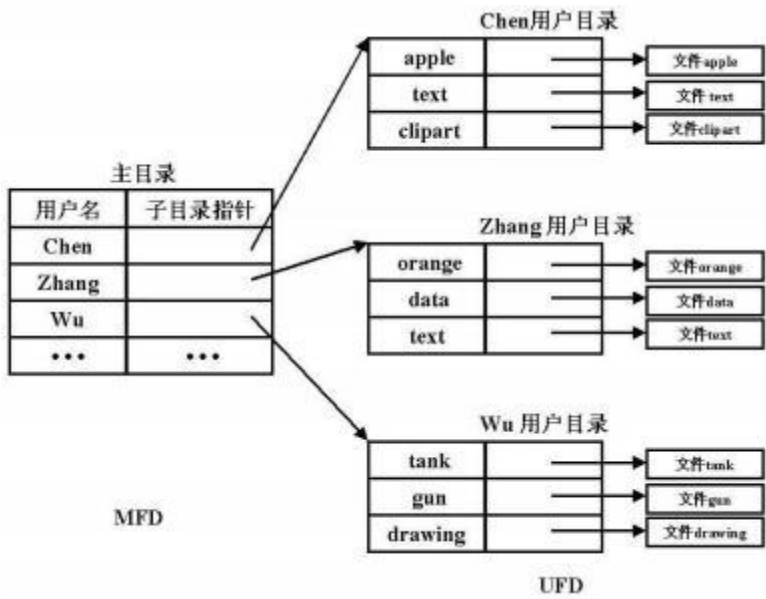
在系统中设置一张线性目录表，表中包括了所有文件的文件控制块，每个文件控制块指向一个普通文件，这就是一级目录结构。



（2）二级目录结构

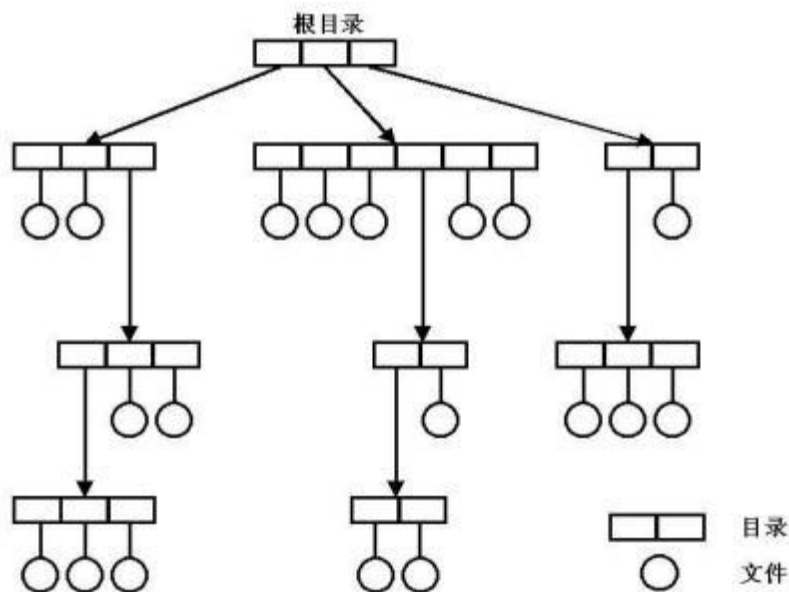
第一级称为主文件目录（MFD，Main File Directory），给出了用户名和用户子目录所在的物理位置。

第二级称为用户文件目录（UFD，User File Directory，又称用户子目录），给出了该用户所有文件的 FCB。



（3）多级目录

把二级目录的层次关系加以推广，就形成了多级目录，又称树型目录结构



多级目录结构的优点是便于文件分类，且具有下列特点：

- （1）层次清楚
- （2）解决了文件重名问题
- （3）查找搜索速度快

目前大多数操作系统如UNIX、Linux 类、Windows 等都采用多级目录结构。在右图中给出了UNIX 操作系统的一棵目录树。

2.当前目录与目录检索

文件系统向用户提供了一个当前正在使用的目录，称为“当前目录”，又称“工作目录”如果需要，用户可随意更改当前目录

用户在访问文件时，需要进行目录检索，这时用户给出文件名，系统按名寻找目录项。有两种根据路径名检索的方法：一种是全路径名，另一种是相对路径

三、 目录项和目录文件 1. 目录项

当用户建立一个新文件时，与该文件有关的一些信息与属性记录在该文件的文件控制块内。为了便于管理，通常将文件控制块做成定长数据结构的一个记录，存放在目录文件中而这样的每一个记录称为目录项。

2. 目录文件

多个文件的文件控制块集中在一起组成了文件的目录通常，文件目录以文件的形式保存起来，这个文件就被称为目录文件。

四、 目录项分解法

即把目录项（FCB）分为两部分，符号目录项（次部）和基本目录项（主部）其中，符号目录项包含文件名以及相应的文件号；而基本目录项包含了除文件名外文件控制块的其他全部信息。

五、UNIX 的文件目录实现

UNIX 目录中为每个文件保留了一项。每个目录项包含了两个域，文件名（14 个字节）

和 i 结点的编号（2 个字节）。UNIX 的 i 结点包含一些属性：

文件大小、三个时间（创建时间、最后访问时间、最后修改时间）、所有者、所在组、保护 信息以及一个计数（用于记录指向 i 结点的目录项的数量）。

当一个新的连接加到一个 i 结点上，i 结点里的计数就会加 1。

当移走一个连接时，该计数就减 1。当计数为 0 时，就收回该 i 结点，并将对应的磁盘块放 进空闲表

六、FAT 文件系统的实现

FAT 是 File Allocation Table（文件分配表）的缩写。

FAT 是一个简单的文件系统，最初为 DOS 操作系统设计，适用于小容量的磁盘，具有简 单的目录结构。

为了向后兼容，也为了方便用户升级，目前新版本的Windows仍然提供对 FAT 的支持。FAT 文件系统总共有三个版本：

FAT-12、FAT-16 和 FAT-32，取决于用多少二进制位表示磁盘块地址。

FAT 文件系统以簇为单位进行分配，所以FAT-16 文件系统表示用 16 位（2 字节）表示 簇号

FAT 文件系统三部分：

（1）引导扇区：引导扇区（Boot Sector）包含用于描述卷的各种信息，利用这些信息才可 以访问文件系统。

（2）文件分配表：每个文件都给出了它在卷上的起始簇号。起始簇号是文件所使用的第 一个簇的地址，每个簇都包含一个指针，指向文件中的下一簇，或者包含一个指示符（0xFFFF）， 表明该簇是文件的结尾。

（3）根目录

在 FAT16 文件系统中，位于根目录下的每个文件和子目录在根目录区中都包含一个目录 项。根目录与其他目 录之间的唯一区别是，根目录位于磁盘上一个特殊的位置并且具有固定 的大小，每个目录项的大小为 32 字节，其内容包括：文件名、扩展名、属性字节、最后 一次修改时间和日期、文件长度、第一簇的编号等。

第四节 文件存储空间管理

前面讨论的文件系统，主要是从用户的角度探讨问题。文件的使用者关心文件是如何命 名的、可以进行哪些文 件操作、文件目录是如何组织的、如何检索或查找文件目录等问题。

本节从设计和实现者的角度讨论文件系统如何实现。在磁盘上怎样安排文件和目录存 储，如何管理磁盘空间，以及怎样使文件系统有效而可靠地工作等。

一、磁盘空间管理

外存储设备中的空间容量比较大，对文件删除之后而不再使用的空间，必须加以回收， 然后在建立文件等操作 中重新利用。

对于只读的存储设备（如CD-ROM 光盘），无所谓回收，也无所谓动态分配，这种存储设 备在物理上就是不可重用的。

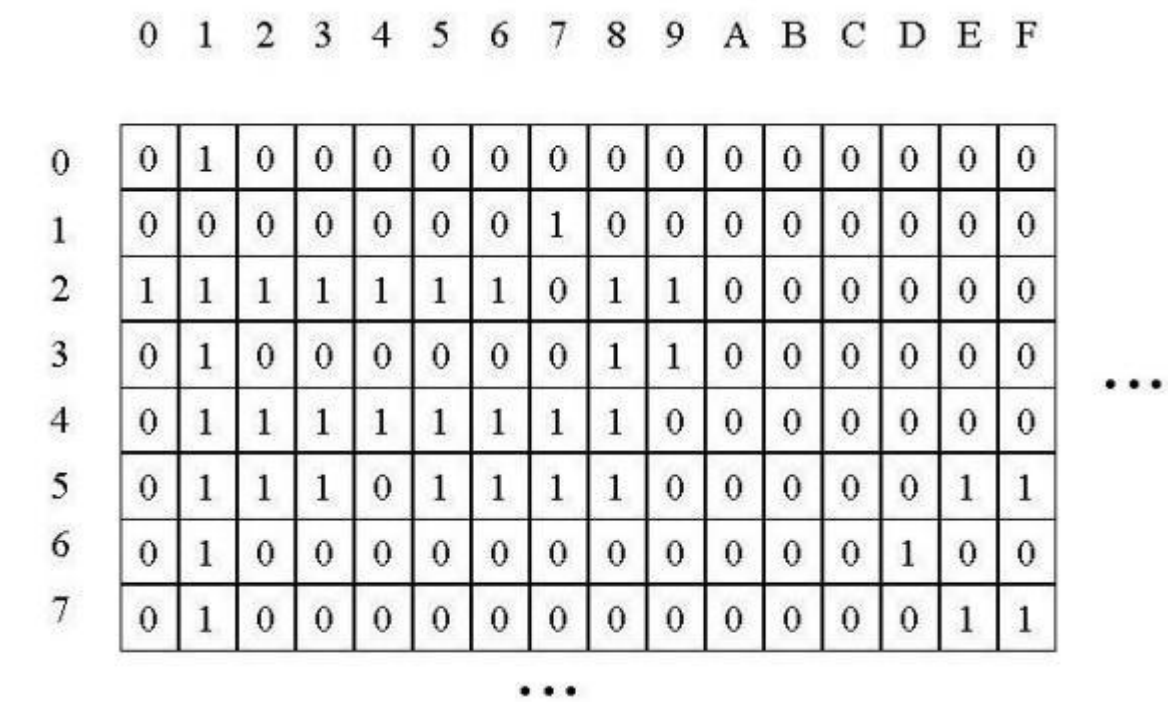
为了进行存储空间的分配与回收，在外存储设备上设置有空闲空间登记表，该表动态跟 踪该外存储设备上所有 还没有分配给任何文件的空闲块的数目和块号。

二、磁盘空间的分配与回收算法

1、位示图

- 2、空闲块表
- 3、空闲块链表
- 1、位示图

位示图法的基本思想是，利用一串二进制位（bit）的值来反映磁盘空间的分配使用情 况。每一个磁盘中物理块 用一个二进制位对应，如果某个物理块为空闲，则相应的二进制位 为 0；如果该物理块已分配了，则相应的二 进制为 1



- 位示图优点：
- 1）对空间分配情况的描述能力强，一个二进制位就描述一个物理块的状态。
 - 2）位示图占用空间较小，因此可以复制到内存，使查找既方便又快速。
 - 3）位示图适用于各种文件物理结构的文件系统。

2、空闲块表

空闲块表是专门为空闲块建立的一张表，该表记录外存储器全部空闲的物理块：每一个 空闲块的第一个空闲物 理块号和该空闲块中空闲物理块的个数。适合于文件物理结构为顺序 结构的文件系统。

序号	首空闲块号	空闲块个数
0	10a8	12
1	9002	98
2	a6003	4096
...	...	...
n	899a08	2568
...	...	...

空

3、空闲块链表

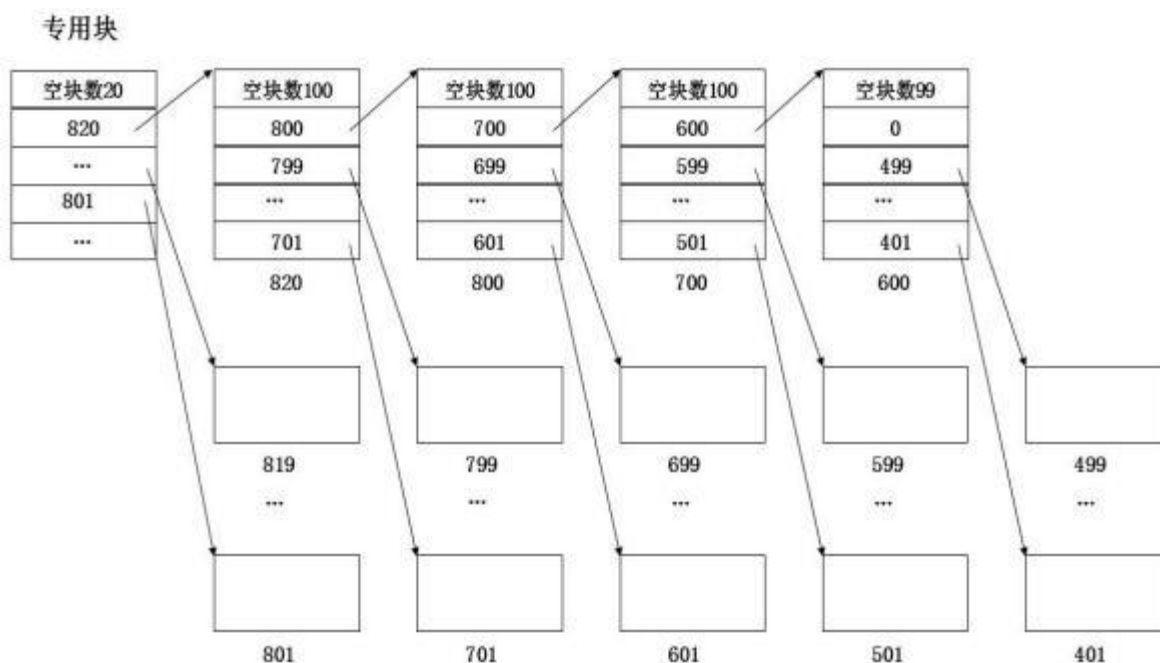
将外存储器中所有的空闲物理块连成一个链表，用一个空闲块首指针指向第一个空闲块，随后的每个空闲块中都含有指向下一个空闲块的指针，最后一块的指针为空，表示链尾，这样就构成了一个空闲块链表。



空闲块链表的特点：

- 1) 空闲块链表模式效率低，要遍历整张表，必须要读每一个物理块，需要大量 I/O 时间。
 - 2) 在空闲块链表模式中对空间的申请和释放是以块为单位的。申请空间时从链首取空闲块。空间释放时将物理块接入链尾。
 - 3) 空闲块链表法节省内存，但申请空间和回收空间的速度较慢，实现效率较低。
4. UNIX 系统的空闲块组成表

对链接表的一个改进方案是将 n 个空闲盘块的地址存储在第一个空闲块中。



假设初始化时系统已把专用块读入内存存储器 L 单元开始的区域中，分配和回收的算法如下：

(1) 分配一个空闲块

查 L 单元内容（空闲块数）：

- 当空闲块数 > 1 ， $i = L + \text{空闲块数}$ ；从 i 单元得到一空闲块号；将该块分配给申请者；空闲块数减 1
- 当空闲块数 $= 1$ ，取出 $L + 1$ 单元内容（一组的第一块块号或 0）；取值 $= 0$ ，无空闲块，申请者等待；取值 $\neq 0$ ，把该块内容复制到专用块；

该块分配给申请者。把专用块内容读到内存 L 开始的区域。

(2) 归还一个空闲块 查 L 单元的空闲块数：

- 当空闲块数<100，空闲块数加 1；j: =L+空闲块数；归还块号填入 j 单元。
- 当空闲块数=100，把内存中登记的信息写入归还块中；把归还块号填入L+1单元；将 L 单元置成 1。

第五节 实现文件系统的表目

当用户申请打开一个文件时，系统要在内存中为该用户保存一些必要的信息，这些信息以表格栏目中内容的形式出现，被称为表目。

在实现文件系统时所需要的表目有若干种，其中在内存中所需的重要表目有如下内容。

一、系统打开文件表

系统打开文件表专门用于保存已打开文件的文件控制块，通常放在内存。该表格中还保存了共享计数、修改标志等信息。

共享计数标出有几个进程打开同一个文件；修改标志是指文件控制块或 i 结点的内容是否被修改过，如果修改过，则关闭文件时要将文件控制块写回磁盘。

二、用户打开文件表

每个进程都有一个“用户打开文件表”。以 UNIX 为例，该表的内容有文件描述符、打开方式、读写指针、系统打开文件表入口等，如图所示

另外在进程的进程控制块 PCB 中，还记录了“用户打开文件表”的位置。

用户打开文件表指向了系统打开文件表。如果多个进程共享同一个文件，则一定有多个用户打开文件表目对应着系统打开文件表的同一入口。

第六节 文件及文件目录的操作

一、典型的文件操作

文件系统是提供给用户使用的、用户可以进行按名存取所需要的文件。在文件系统的实现中，为用户提供使用文件的手段是文件系统的重要任务之一。下面介绍 7 个常用的进行文件操作的系统调用：

1、建立文件

用户调用“建立文件”操作，需要提供所要创建的文件的文件名及若干参数：用户名、文件名、存取方式、存储设备类型、记录格式、记录长度等。

系统依据用户提供的文件名及若干参数，为新创建的文件分配一个文件控制块 FCB，并建立必要的存储空间，分配空的 FCB。从而建立起系统与文件的联系。

建立文件系统调用的一般格式为：

create（文件名，访问权限，（最大长度））。建立文件的具体步骤如下：

- 1) 检查参数的合法性：文件名是否符合命名规则。
- 2) 检查同一目录下有无重名文件。
- 3) 在目录中有无空闲位置。
- 4) 填写目录项内容：文件名、用户名、存取权限、长度置零，首地址等。
- 5) 返回。

2、打开文件

任何一个文件使用前都要先打开，即把文件控制块 **FCB** 送到内存。

打开文件系统调用的一般格式为：**fd=open**（文件路径名，打开方式）。

打开文件时，系统主要完成以下工作。

- 1) 根据文件路径名查目录，找到 **FCB** 主部。
- 2) 根据打开方式、共享说明和用户身份检查访问合法性。
- 3) 根据文件号查系统打开文件表，看文件是否已被打开。如果是，共享计数加 1；否则，将外存中的 **FCB** 主部等信息填入系统打开文件表空表项，共享计数置为 1。
- 4) 在用户打开文件表中取一空表项，填写打开方式等，并指向系统打开文件表对应表项。返回信息：**fd**：文件描述符，是一个非负整数，用于以后读写文件。

3、读文件

读文件系统调用的一般格式为：

read（文件名，（文件内位置），要读的长度，内存目的地址）。隐含参数：文件主。

读写方式可为读、写和既读又写等。读文件时，系统主要完成以下工作。

- 1) 检查长度是否为正整数。
- 2) 根据文件名查找目录，确定该文件在目录中的位置。
- 3) 根据隐含参数中的文件主和目录中该文件的存储权限数据，检查是否有权读。
- 4) 由文件内位置与要读的长度计算最末位置，将其与目录中的文件长度比较，超过否？
- 5) 根据参数中的位置、长度和目录中的映射信息，确定物理块号、需要读出的块数等读盘参数。
- 6) 根据下一块号读块至内存缓冲区。
- 7) 取出要读的内容，也许要进行成组的分解，将取出的内容送至参数中的内存目的地址。
- 8) 根据块内长度或起始块号+块数，确定是否还读下一块，同时确定下一块块号。
- 9) 正常返回。
- 10) 错误返回，返回相应错误号。

4、写文件

写文件系统调用的一般格式为：**write**（文件名，记录键，内存位置）。

把内存中指定单元的数据作为指定的一个记录写入指定文件中，系统还将为其分配物理块，以便把记录信息写到外存上。

5、关闭文件

关闭文件系统调用的一般格式为：**close**（文件名）。

系统根据用户提供的文件名或文件描述符，在该文件的文件控制块上做修改。例如，将该文件的共享用户数减 1，减 1 后若值为 0，则将文件控制块置上“非活跃”标志，若该文件控制块内容被修改过，则要写回外存。

6、删除文件

删除文件系统调用的一般格式为：**delete**（文件名）。系统根据用户提供的文件名或文件描述符，检查此次删除的合法性，若合法，则收回该文件所占用的文件控制块及物理块等资源。

7、指针定位

指针定位的一般格式为：**seek**（**fd**，新指针的位置）。指针定位时，系统主要完成以下工作。

- 1) 由 W 检查用户打开文件表，找到对应的入口。
- 2) 将用户打开文件表中文件读写指针位置设为新指针的位置，供后继续写命令存取该指针处文件内容。

二、典型的目录操作

对目录的操作与文件类似，通常由系统调用实现。

UNIX 的目录操作（“.”代表当前目录，“..”代表根目录）

- 1) **create**，创建目录。
- 2) **delete**，删除目录。只有当一个目录为空时，该目录方可删除。
- 3) **opendir**，打开目录。使内容可被读取。
- 4) **closedir**，关闭目录。
- 5) **readdir**，打开目录的下一目录项。缺点：程序员须了解和处理目录的内部结构。
- 6) **rename**，文件可换名。目录也可换名。
- 7) **link**，链接技术允许在多个目录中出现同一文件。
- 8) **unlink**，删除目录项。

第七节 文件系统的性能

文件系统的物理基础是磁盘设备，显然，磁盘存储器的服务效率，其速度和可靠性，就成为文件系统性能和可靠性的关键。

设计文件系统时应尽可能减少磁盘访问次数，这样，可以适当减少磁盘存储器性能对文件系统性能的影响。

其他采取有效的措施，提高文件系统的性能。常见的技术措施有如磁盘高速缓存、RAID 技术。

一、磁盘高速缓存

基本思想：系统在内存中保存一些磁盘块，这些磁盘块在逻辑上属于磁盘，内存的这一区域被称为块高速缓存。

运行时，系统检查所有的读请求，所需文件块如在块高速缓存中则直接在内存中进行读操作；否则先启动磁盘，将所需块读到高速缓存中，再复制到其他内存区域。如果内存中的高速缓存已满，则需要按照一定的算法淘汰一些较少使用的磁盘块，让出块高速缓存空间。块高速缓存的内容需要定期写回到磁盘上，以保存对磁盘块的修改。

1、记录的成组

缓存的一种应用是记录的成组。

把若干个逻辑记录合成一组存储一物理块的工作称“记录的成组”，每块中的逻辑记录个数称“块因子”。

记录的成组在不同存储介质上进行信息转储是很有用的。

由于信息交换以块为单位，所以，要进行成组操作时必须使用内存的缓冲区。该缓冲区的长度=成组的最大逻辑记录长度 X 成组的块因子。成组转储操作如图所示。

记录式文件中，有“定长记录格式”和“不定长记录格式”

2、记录的分解

对应前述记录成组的操作，有必要考虑从一组逻辑记录中把一个逻辑记录分离出来的操作，这种操作称为“记录的分解”。

记录的分解操作也要使用内存缓冲区。

定长记录格式：只要知道逻辑记录的长度就能够很容易地进行分解。

不定长记录格式：要根据说明逻辑记录长度的控制信息，计算出用户所指定的记录在内存缓冲区中的位置，然后把记录分解出来。

二、RAID 技术

1、RAID 技术主要是解决两个问题：

- 1) 磁盘速度慢
- 2) 容易出现故障

2、几种常见的组成 RAID 结构

- 1) RAID0 采用多个磁盘并行以提高读写速度；
- 2) RAID1 用磁盘镜像的方法来提高存储的可靠性；
- 3) RAID2 和 RAID3 以“位”或“字节”作为并行单位；
- 4) RAID4 的并行单位是“块”，为提高可靠性加入了校验，校验码是独立存储的；
- 5) RAID5 与 RAID4 相同，但是检验码以“块”为单位与数据块一起随机存储在磁盘块中；
- 6) 其他的 RAID 结构都是上述各种方法的组合或扩展，最常用的是 RAID1+0 以及 RAID5。

第八节 文件共享、保护

一、文件共享

文件的共享是指一个文件可以允许多个用户共同使用文件共享不仅是完成共同任务所必需，而且还带来许多好处：节省文件所占用的存储空间；免除系统复制文件的工作；减少用户大量重复性劳动；减少实际输入输出文件的次数此外，利用文件共享可以实现进程间相互通信。

在允许文件共享的系统中，必须对共享文件进行管理从共享的时间段上看，共享文件的使用有两种情况：

- (1) 文件可以同时使用
- (2) 文件不允许同时使用

在文件共享的具体方式上，有三种文件的共享形式：

- (1) 文件被多个用户使用，由存取权限控制
- (2) 文件被多个程序使用，但分别用自己的读写指针
- (3) 文件被多个程序使用，但共享读写指针

二、文件存取控制

文件系统必须有防止硬、软件的各种意外可能破坏文件的能力为此，文件系统经常采用

建立副本和定时转储的方法来保护文件

1.建立副本

对文件建立副本，是保护文件不受破坏的有效方法

2.定时转储

定时转储的含义是，每隔一定的时间就把文件转储到其他的存储介质上

三、UNIX 的文件使用权限管理方案

1.存取控制矩阵

在存取控制矩阵方式中，系统以一个二维矩阵来实施文件的存取控制在这个二维矩阵中，其中一维代表所有的用户，另一维代表所有的文件两维交叉点所对应的矩阵元素则是某一个用户对一个文件的存取控制权限，包括读 R、写 W 和执行 E。

2.二级存取控制

对文件实施存取控制的另一种方法是二级存取控制。二级存取控制方法中设立两个存取级别。在第一级，把用户按某种关系划分为若干用户组，进行对访问者的识别；在第二级，进行对操作权限的识别。这样，所有用户组对文件权限的集合就形成了对该文件的存取控制。

3.UNIX 中的文件存取权限

下面以 UNIX 系统为例，进一步说明对文件的存取权限的控制设计。UNIX 中对文件的存取权限划分为两级。在第一级中对访问者或者用户进行分类识别：

- (1) 文件属主 (owner)；
- (2) 文件属主的同组用户 (group)；
- (3) 其他用户 (other)。

在第二级中，对文件操作权限的权限设置，则根据不同的操作内容进行权利限定，把对文件的操作分成如下的类别：

- (1) 读 (read) 操作 (r)；
- (2) 写 (write) 操作 (w)；
- (3) 执行 (execute) 操作 (x)；
- (4) 不能执行任何操作 (-)

第七章 设备管理

本章重难点分析

1.I/O 设备的控制方式

2.设备硬件和设备管理软件的组成 3.缓冲技术

第一节 设备管理的基本概念

输入输出设备 (I/O 设备) 也称为外部设备 (peripheral)，包括计算机系统中除 CPU 和内存储器以外的所有的硬件设备和装置。

广义的输入输出设备即上述定义，狭义的输入输出设备不包括外存储设备。



一、设备管理的任务

1) 解决 I/O 设备性能同 CPU 性能不匹配

CPU 性能越强，I/O 设备性能同 CPU 性能不匹配的反差也就越大。

操作系统主要通过缓冲技术、中断技术和虚拟技术来解决这一问题。

2) 对多种 I/O 设备实现统一的管理, 从而方便用户使用, 是 I/O 设备管理的另一项重要任务。

操作系统需要在 I/O 设备管理和系统的其他部分之间提供简单而易于使用的接口, 这个接口对于所有设备都应该是相同的, 这就是所谓的设备独立性。

3) 如何保证用户安全正确的使用 I/O 设备。

对于设备的使用者而言, 由设备传送或管理的数据应该是安全和保密的, 不能破坏或泄露;

对于设备的拥有者而言, 多用户多任务环境中的设备使用应该通过协调避免冲突, 设备不能被破坏。

二、输入输出设备的分类 1.按设备的使用特性分类

(1) 输入设备

(2) 输出设备

(3) 交互式设备

特点: 用户命令信息通过各种输入设备进入计算机系统, 系统同步地在显示器上显示用户的命令信息以及执行命令后所得到的处理结果。

(4) 存储设备等特性:

1) 可写入性

2) 可读出性

3) 可保存性

2.按设备的信息组织方式分类

(1) 字符设备 (Character Device)

键盘、终端、打印机等以字符为单位组织和处理信息的设备。

(2) 块设备 (Block Device)

磁盘、磁带等以字符块为单位组织和处理信息的设备被称为块设备。 3.按设备使用可共享性分类

(1) 独占设备:

在一个程序 (作业、用户) 的整个运行期间都必须由单个程序 (作业、用户) 独占直至该程序 (作业) 完成的设备。

例如: 打印机、磁带驱动器。

问题: 低效率的, 有可能造成死锁。

(2) 共享设备:

能够同时让许多程序 (作业、用户) 使用的设备。例如: 磁盘。

广义的共享设备是指非并发共享, 几乎所有的设备都是广义的共享设备, 可以把独占设备解释为只能顺序共享 (即非并发共享)。

狭义的共享设备是指并发共享, 即操作系统中的共享设备的真正定义。

(3) 虚拟设备:

在一类设备上模拟另一类设备, 被模拟的设备称为虚拟设备。

通常用共享设备模拟独占设备 (如用磁盘的固定区域模拟打印机), 用高速设备模拟低速设备。引入虚拟设备的目的是为了提高设备利用率。

三、设备管理与文件管理的区别:

设备管理是对计算机系统中所有输入输出设备硬件的管理, 为用户提供标准的接口来使用设备

文件管理对设备里面存储的数据和信息，提供一整套管理规则，以文件及其配套概念来

具体实施 联系：

将物理的设备资源抽象为逻辑的文件资源，使得用户可以用统一、透明的方式访问物理 设备和设备上的数据和信息

在 UNIX 系统中将所有的设备都当作文件对象来管理，通过提供给用户一个简单易用的 接口平台，提高设备利用率，降低设备使用难度

第二节 设备硬件和设备管理软件的组成

一、设备硬件的组成

从硬件的角度看，设备由物理设备和电子部件两部分组成——操作系统而言更注重的是 其电子部件的控制方式

典型的计算机系统其硬件结构：中央部分是 CPU 和主存，通过总线与第二层的接口（适 配器）部件相连，第三层是各种外部设备控制器，最外层是外部设备

每一种外部设备在它自己的设备控制器（一种电子部件）的控制下工作，而设备控制器 则通过适配器和主机连接。

（1）写入控制寄存器：操作系统可以控制设备发送数据、接收数据、开启或关闭。

（2）读取状态寄存器：操作系统可以获悉设备的状态，如是否准备好接收新的数据。

（3）数据寄存器：通常作为操作系统发出或接收数据的缓冲区。为了使处理器能够访问设 备控制器中的寄存器，必须为每个寄存器分配唯一的地址，该地址称为 I/O 端口地址或 I/O 端口号。

I/O 端口地址主要有两种编址方式： 内存映射编址和 I/O 独立编址。

（1）内存映射编址：分配给系统中所有端口的地址空间与内存地址空间统一编址，处理器 把设备控制器中的寄存器看作一个存储单元，对 I/O 的读写操作等同于对存储器的操作，这 样的系统称为内存映射 I/O（memory-mapped I/O）。

大部分处理器采用内存映射 I/O。

（2）I/O 独立编址：分配给系统中所有端口的地址空间与内存地址空间是完全独立的。采 用 I/O 独立编址的处理器需要使用专门的 I/O 指令对 I/O 端口进行操作。

输入输出设备对设备控制器中的 I/O 控制方式（第三节）：

1）程序直接控制方式

2）中断控制方式

3）DMA 方式

4）通道控制方式

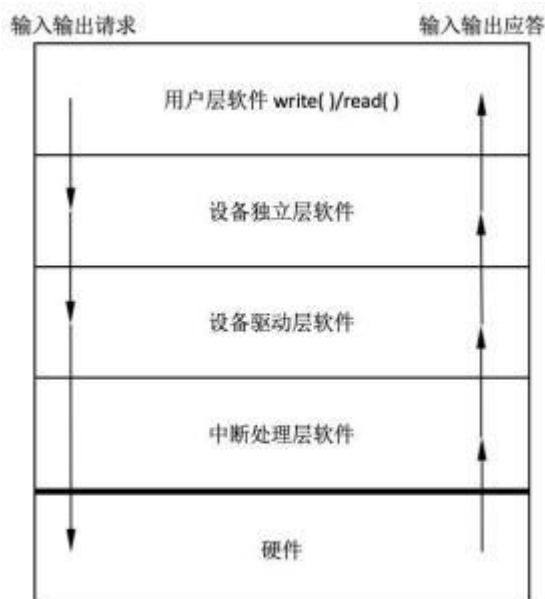
二、设备软件的组成

设计 I/O 软件结构的基本思想：

较低的层处理与硬件有关的细节，并将硬件的特征与较高的层隔离；而较高的层则向用 户提供一个友好的、清晰而规整的 I/O 接口。一般的 I/O 软件结构分为四层：

1）中断处理程序；

- 2) 设备驱动程序;
- 3) 设备独立的操作系统软件;
- 4) 用户级软件。



从功能上看，设备独立层是 I/O 软件的主要部分；

从代码量上看，设备驱动层是 I/O 软件的主要部分。

中断处理程序负责控制输入输出设备和内存与CPU 之间的数据传送设备独立层起到承上启下的作用：

将种类繁多的输入输出设备的驱动屏蔽起来，对用户呈现出一个标准的调用接口，将用户使用输入输出设备的需求按照设备驱动的要求配置参数并传递数据。

大部分 I/O 软件都包含在操作系统中，但是用户层软件仍有部分是与库函数连接在一起的，甚至还有在内核之外运行的程序构成。

三、设备独立性

设计 I/O 软件的一个最关键的目标是设备独立性（**device independence**），即除了直接与设备硬件打交道的低层软件之外，其他部分的软件并不依赖于硬件。

设备需要的 I/O 软件功能可以在与设备独立的软件中实现，这类软件面向用户应用层并提供一个统一的接口

设备独立层软件的功能：设备驱动程序的统一接口，设备命名，设备保护，提供一个与设备无关的逻辑块缓冲，存储设备的块分配，独占设备的分配和释放，错误处理。

1.设备命名

在操作系统的 I/O 软件中，对 I/O 设备借用了与文件统一命名的方法，即采用文件系统路径名的方法来命名设备。设备独立层的软件负责把设备的符号名映射到相应的设备驱动程序上。

例如：在 UMINX 系统中，`/dev/tty00`，唯一确定了一个特殊文件，其中包含了主设备号和次设备号。主设备号用于寻找对应的设备驱动程序，次设备号提供了设备驱动程序的有关参数，用来确定要读写的具体设备。

2.设备保护

对设备进行必要的保护，防止无授权的应用或用户的非法使用。设备保护是与设备命名的机制密切相关的：

- MS-DOS：操作系统根本没有对设备设计任何的保护机制。
- 大型的计算机系统：用户进程对 I/O 设备的直接访问是完全禁止的。

• **UNIX 系统：**采用一种存取权限的模式，系统管理员可以根据需要为 每一个设备设置适当的“读、写、运行”权限。

3.提供一个与设备无关的逻辑块

与设备无关的软件就有必要向较高层软件屏蔽各种 I/O 设备空间大小、处理速度和传输 速率各不相同的这一事实，而向上层提供大小统一的逻辑块尺寸。

较高层的软件只与抽象设备打交道，不考虑物理设备空间和数据块大小而使用等长的逻辑块。

4.缓冲

对于常见的块设备和字符设备，一般都使用缓冲区。块设备：如果用户进程只写了半块 数据，则操作系统通常将数据保存在内部缓冲区，等到用户进程写完整块数据才将缓冲区的数据写到磁盘上。

字符设备：当用户进程把数据写到设备的速度快于系统输出数据的速度时，必须使用缓冲。

5.存储设备的块分配

在创建一个文件并向其中填入数据时，通常在硬盘中要为该文件分配新的存储块。 操作系统需要为每个磁盘设置一张空闲块表或位图。

6.独占设备的分配和释放 例如：打印机驱动器

操作系统对设备使用请求进行检查，并根据申请设备的可用状况决定是接收该请求还是

拒绝该请求。

7.错误处理

一般来说错误处理是由设备驱动程序完成的。大多数错误是与设备密切相关的，因此， 只有驱动程序知道应如何处理（比如，重试、忽略或放弃）。

第三节 I/O 设备的控制方式

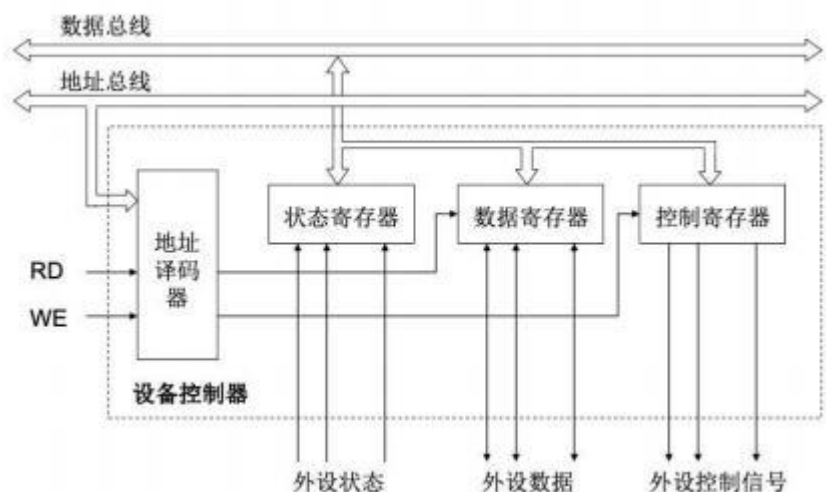
I/O 设备的控制方式取决于 I/O 设备硬件与处理器和内存的联结方式以及其相应的设备 驱动程序。

主要有以下四种方式：程序控制方式、中断控制方式、DMA 控制方式和通道控制方式。

一、程序直接控制方式

程序控制方式也称为 PIO（Programmed I/O，程控 I/O）方式，是指由用户进程直接控 制处理器或内存和外围设备之间进行信息传送的方式，也称为“忙-等”方式、轮询方式或 循环测试方式。

这种方式的控制者是用户进程。程序直接控制方式的传送结构如图：



优点：

- 1) 处理器和外设的操作能通过状态信息得到同步；
- 2) 硬件结构比较简单。

缺点：

- 1) 处理器效率较低；
- 2) 对外部出现的异常事件无实时响应能力。

程序直接控制方式适合处理器执行速度较慢，而且外围设备较少的系统，如单片机系统。

二、中断控制方式

中断是一种在发生了一个异常事件时，调用相应处理程序（通常称为中断服务程序）进行服务的过程

- 1) 中断源向处理器发出请求，处理器执行完当前指令且状态为允许中断的情况下响应该请求。
- 2) 硬件自动关中断（防止在保留断点和程序转移过程中又有新的中断请求发生）、保留断点、转到相应的中断服务程序入口处。
- 3) 执行中断服务程序，由软件完成中断服务。
- 4) 中断服务程序结束时，执行中断返回指令返回断点处，继续执行原程序。

采用中断控制方式，可以做到以下内容：

- 1) 处理器与外设在大部分时间内并行工作，有效地提高了计算机的效率。
- 2) 具有实时响应能力，可适用于实时控制场合。
- 3) 及时处理异常情况，提高计算机的可靠性。

如要采用中断方式进行数据传送，则处理器和设备控制器就应具备中断机构。

三、DMA 控制方式

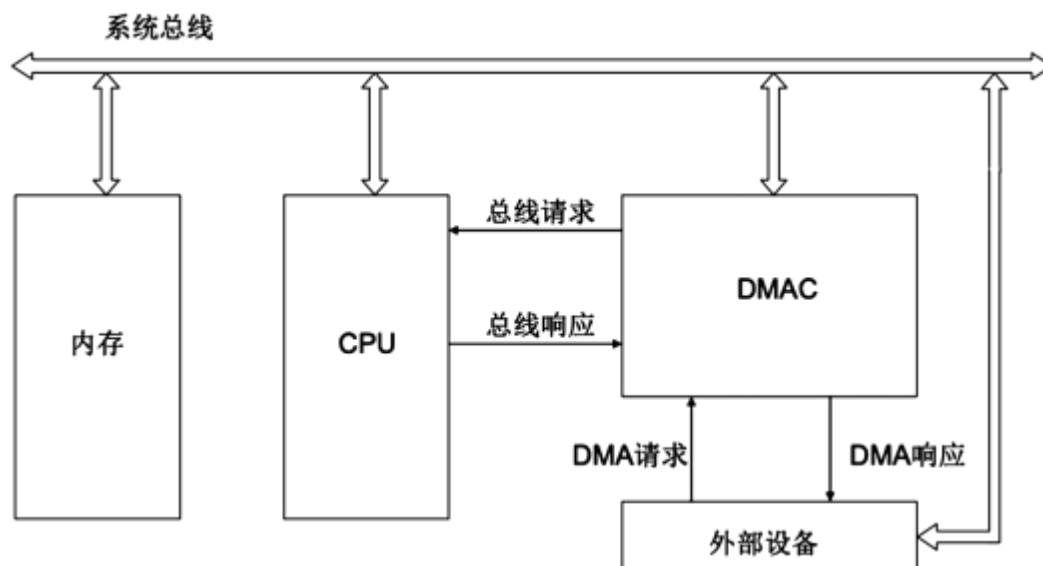
DMA 是直接内存访问（Direct Memory Access）的缩写，它是一种完全由硬件执行 I/O 数据交换的工作方式。

DMA 控制器（DMA Controller, DMAC）从 CPU 完全接管对总线的控制，数据交换不经过 CPU，而直接在内存和外部设备之间进行。

由 DMA 控制器占用系统总线并向内存发出地址和控制信号，对传送信息的个数计数，并且以中断方式向 CPU 报告传送操作的结束。

DMA 方式的传送结构如图所示，可分为三个阶段：传送前预处理、数据传送、传送后处理

DMA 方式一般用于高速传送成组的数据。



四、通道控制方式

通道（**channel**）是一个特殊功能的处理器，它有自己的指令和程序，可以实现对外部设备的统一管理和外部设备与内存之间的数据传送。

与 **DMA** 方式相比，通道方式增加了 **CPU** 与通道操作的并行能力；增加了通道之间以及同一通道内各设备之间的并行操作能力；为用户提供了灵活增加外设的可能性。

按照信息交换方式的不同，一个系统中可以设立三种类型的通道：

- 1.选择通道，优点是以数据块为单位进行传输，传输率高；缺点是通道利用率低。
- 2.数组多路通道，优点是同选择通道一样，以数据块为单位进行传输，传输率高，又具有多路并行操作的能力，通道利用率高，缺点是控制复杂。
- 3.字节多路通道，各设备与通道之间的数据传送是以字节为单位交替进行的，各设备轮流占用一个很短的时间片，多路并行操作能力与数组多路通道相同。

通道具有的功能：

接受 **CPU** 的指令，按指令与外部设备进行通信，从内存读取通道指令，执行通道程序，向设备控制器和设备发送各种命令，组织外部设备和内存之间进行数据传送，并根据需要提供数据缓存的空间

从外部设备得到设备的状态信息，形成并保存通道本身的状态信息，根据要求将这些状态信息送到内存的指定单元，供 **CPU** 使用，将外部设备的中断请求和通道本身的中断请求按序及时报告 **CPU**

第四节 设备分配与回收

进程必须首先向设备管理程序提出资源申请，然后，由设备分配程序根据相应的分配算法为进程分配。如果申请进程得不到它所申请的资源，将被放入资源等待队列中等待，直到所需要的资源被释放。

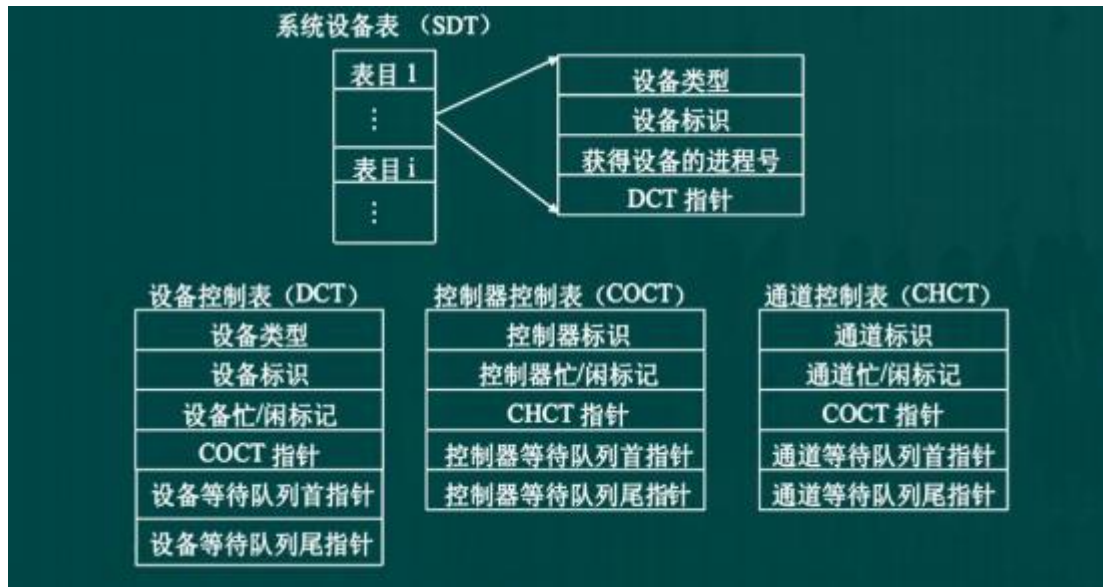
为了合理、高效、安全地分配和回收设备，我们从数据结构、分配原则、分配策略以及分配算法进行分析。

一、设备分配的相关数据结构和策略

1、数据结构

设备分配算法的实现中，常采用的数据结构主要含四张表：

- 系统设备表（**SDT**）：每个接入系统中的设备都占有一个表目项。
- 设备控制表（**DCT**）：系统中的每台设备都有一张设备控制表 **DCT**。
- 控制器控制表（**COCT**）：每个控制器都有一张控制器控制表 **COCT**，用于登录某控制器的使用分配情况及与该控制器相连的通道的情况。
- 通道控制表（**CHCT**）：**CHCT** 表反映了通道的情况，系统中的每个通道一张 **CHCT**。



2、分配原则

设备分配的总原则是：要充分发挥设备的使用效率，尽可能地让设备忙碌，但又要避免由于 不合理的分配方法造成进程死锁。

设备分配方式有两种，即静态分配和动态分配。

(1) 静态分配

基本思想：在用户作业开始执行前，由系统一次分配该作业所要求的全部设备、控制器（和 通道）。一旦分配之后，这些设备、控制器（和通道）就一直为该作业所占用，直到该作业 被撤销。

优点：使安全分配方式不会出现死锁。 缺点：设备的使用效率低。

因此，静态分配方式并不符合分配的总原则。

(2) 动态分配

基本思想：在进程执行过程中根据执行需要进行。当进程需要设备时，通过系统调用命令向 系统提出设备请求，由系统按照事先规定的策略给进程分配所需要的设备、I/O 控制器（和 通道），一旦用完之后，便立即释放。

优点：动态分配方式有利于提高设备的利用率。

缺点：如果分配算法使用不当，则有可能造成进程死锁。

3.分配策略

考虑的因素有：I/O 设备的固有属性，I/O 设备的分配算法，设备分配的安全性以及设备独 立性。

与进程调度相似，设备的分配策略通常采用先来先服务（FIFS）和高优先级优先等。

对于独占设备或共享设备，会根据其特性采用不同的分配策略，以期达到合理、高效和 安全的目的。

二、独占设备的分配

1.设备的绝对号和相对号

系统为每一台设备确定一个编号，以便区分和识别，这个确定的编号称为设备的“绝对 号”。

有时用户可能要求同时使用几台同类设备，为了避免使用时的混乱，用户可以把自己要 求使用的若干台类设备给出编号，由用户在程序中定义的设备编号称设备的“相对号”。

2.设备的指定方式

指定设备的方式可以有两种：一种是指定设备的绝对号，另一种是指定设备类、相对号。

(1) 指定设备的绝对号，系统应该把与绝对号对应的设备分配给作业。如指定的这台设备 占用或者故障，则作业就阻塞不能执行。

(2) 用户程序中通过“设备类、相对号”提出使用设备的要求这种方式使设备分配的适应 性好、灵活性强。

1) 系统只要从指定的某一类设备中提供“好的且尚未分配的”设备来进行分配。

2) 如果分配给用户的设备在使用中出了故障，系统可以从同类设备中找另一台“好的且尚 未分配的”设备来替换。

3.独占型设备的分配和释放

独占型设备通常是指打印机、磁带机、扫描仪、绘图仪等设备，这类设备在一段时间内 只能由一个进程所占有。在申请命令和释放命令期间，用户独占该设备。

操作系统设置“设备分配表”用来记录计算机系统所配置的独占设备类型、台数以及分 配情况等。

设备分配表=“设备类表”+“设备表”。

用户作业申请某类外部设备的过程如下（6 步）：

1) 用户作业提出某类外部设备申请。

2) 系统首先检查“设备类表”

3) 依次检查该类设备在设备表中的登记项。

4) 找出符合条件的设备。

5) 进行设备分配，修改表：

①修改“设备类表”中该类设备的现存台数。

②把该台设备在“设备表”中的“分配状态”标志改成“已分配”

③在该“设备表”中填上拟占用该设备的作业名和作业程序中定义的该设备相对号。

6) 把已分配的该设备的绝对号与相对号的对应关系通知用户程序，以便用户在分配到的设 备上进行相关的操作。

三、共享设备分配

共享设备可被多个进程所共享，但在每个 I/O 传输的单位时间内只由一个进程所占有。

在每一个使用命令之前都隐含有一个申请命令，在每一个使用命令之后都隐含有一个释放命 令

共享设备使用的具体方法如下：

申请设备：如设备被占用，进入设备等待队列，否则分配设备 启动设备：I/O 传输

释放设备：当设备结束，发出中断信号时，系统唤醒一个等待设备的进程

由于独占设备的分配和回收必须遵守“独占”的要求，设备利用率低，死锁几率增大， 不利于调度，使用能将独占设备转变为共享设备的技术称为“SPOOLing”系统可有效解决这 个问题

第五节 磁盘驱动调度

本节讨论磁盘驱动的调度策略

几乎所有计算机都使用磁盘来存储信息

从存储角度看，与内存相比，磁盘有三个主要的优点： 可用的存储容量非常大

每位的价格非常低

电源关掉后信息不会丢失

磁盘的结构决定了对磁盘的访问需要通过移动磁头所在的磁臂，驱动磁盘旋转以及选择 适当磁头来确定数据存放在磁盘面上的空间位置

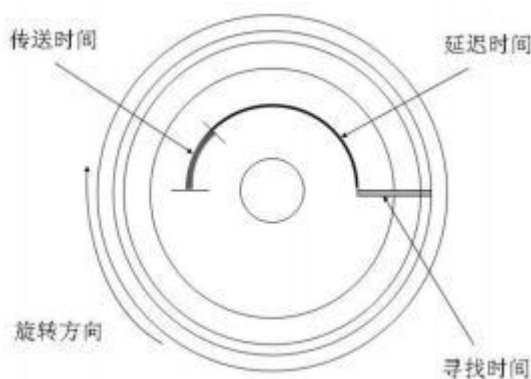
如何快速找到上述位置便是磁盘驱动调度要做的工作

一、信息传输时间

执行一次输入输出所花的时间如下：

- 1) 寻找时间——磁头在移动臂带动下移动到指定柱面所花的时间。
- 2) 延迟时间——指定扇区旋转到磁头下所需的时间。
- 3) 传送时间——由磁头进行读写完成信息传送的时间。 传送信息所花的时间，是在硬件设计就固定的。

寻找时间和延迟时间是与信息在磁盘上的位置有关。



各磁盘的编号按柱面顺序（从 0 号柱面开始），每个柱面按磁道顺序，每个磁道又按扇区顺序进行排序

假定用 t 表示每个柱面上的磁道数，用 s 表示每个盘面上的扇区数，则第 i 柱面， j 磁头， k 扇区所对应的块 b 由如下公式确定： $b = k + s \times (j + i \times t)$

每个柱面上有 $s \times t$ 个磁盘块，为了计算第 p 块在磁盘上位置，可以令：

$$d = s \times t$$

$$m = [p/d]$$

$$n = p \bmod d$$

于是，第 p 块在磁盘上的位置为 柱面号= m

$$\text{磁头号} = [n/s]$$

$$\text{扇区号} = [n \bmod s]$$

二、移臂调度及调度算法

磁盘是一种高速旋转的存储设备。移动磁臂上的磁头在沿着直径盘片的直径方向移动，同时对指定的磁道上的扇面中的数据进行读写操作。

一系列访问的读写请求来到时，如何决定各等待访问的执行次序，使寻找和延迟时间都尽可能小的那个访问者可以优先得到服务，并降低若干个访问者的总访问时间，增加磁盘单位时间内的操作次数？

根据访问者指定的柱面位置来决定执行次序的调度，称为“移臂调度”。移臂调度的目的是尽可能地减少操作中的寻找时间。

常用的移臂调度算法有先来先服务算法、最短寻找时间优先算法、电梯调度算法和单向扫描算法。

1.先来先服务算法

最简单的移臂调度算法是“先来先服务”调度算法，这个算法实际上不考虑访问者要求访问的物理位置，而只是考虑访问者提出访问请求的先后次序。

采用先来先服务算法决定等待访问者执行输入输出操作的次序时，移动臂来回地移动。先来先服务算法花费的寻找时间较长，所以执行输入输出操作的总时间也很长。

2.最短寻找时间优先调度算法

最短寻找时间优先调度算法总是从等待访问者中挑选寻找时间最短的那个请求先执行的，而不管访问者到来的先后次序。

采用最短寻找时间优先算法决定等待访问者执行操作的次序时，读写磁头总共移动了 200 多个柱面的距离，与先来先服务算法比较，大幅度地减少了寻找时间，因而缩短了为各访问者请求服务的平均时间，也就提高了系统效率。

3.电梯调度算法

乘电梯，如果电梯已向上运动到4层时，依次有 3 位乘客陈生、伍生、张生在等候乘电梯。

他们的要求是：陈生在 2 层等待去 10 层；伍生在 5 层等待去底层；张生在 8 层等待 15 层。由于电梯目前运行方向是向上，所以电梯的行程是先把乘客张生从 8 层带到 15 层，然后电梯换成下行方向，把乘客伍生从 5 层带到底层，电梯最后再调换方向，把乘客陈生从 2 层送到 10 层。

前述的同一例子来讨论采用“电梯调度”算法的情况，由于磁臂的初始方向有两个，而该算法是与磁臂的方向有关，所以分成两种情况来讨论：

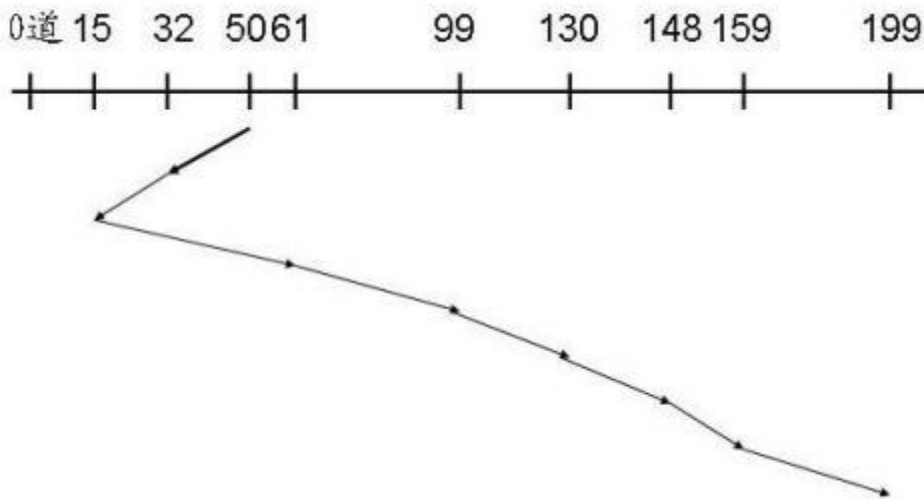
（1）磁臂由里向外移动

（2）磁臂由外向里移动

二、移臂调度及调度算法

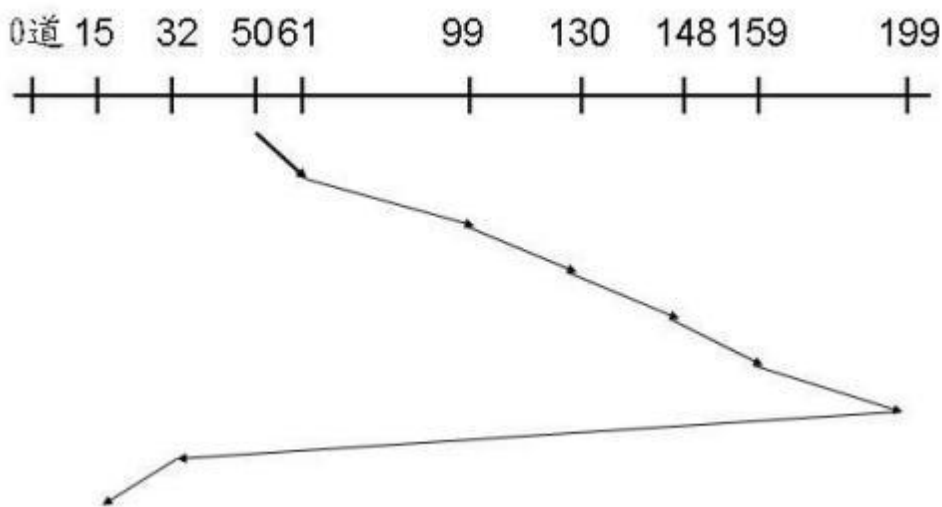
（1）磁臂由里向外移动

开始时，在 50 号柱面执行操作的读写磁头的磁臂方向是由里向外，趋向 32 号柱面的位置，当访问 50 号柱面的操作结束后，沿臂移动方向最近的柱面是 32 号柱面，所以应先为 32 号柱面的访问者服务，然后是为 15 号柱面的访问者服务，由于在向外移方向已无访问等待者，故改变磁臂的方向，由外向里依次为各访问者服务，这种情况下为等待访问者服务的次序是 61，99，130，148，159，199。



（2）磁臂由外向里移动

开始时，正在 50 号柱面执行操作的读写磁头的磁臂是由外向里（即向柱面号增大的内 圈方向）趋向 61 号柱面的位置当访问 50 号柱面的操作结束后，沿臂移动方向最近的柱面是 61 号柱面，所以应先为 61 号柱面服务，然后按磁臂由外向里移动的方向，依次为 99，130，148，159，199 柱面的访问者服务当 201 号柱面的操作结束后，向里移动的方向已经无访问 等待者，所以改变磁臂的前进方向， 由里向外依次为 32、15 柱面的访问者服务。



“电梯调度”与“最短寻找时间优先”都是要尽量减少磁臂移动时所花的时间 区别：

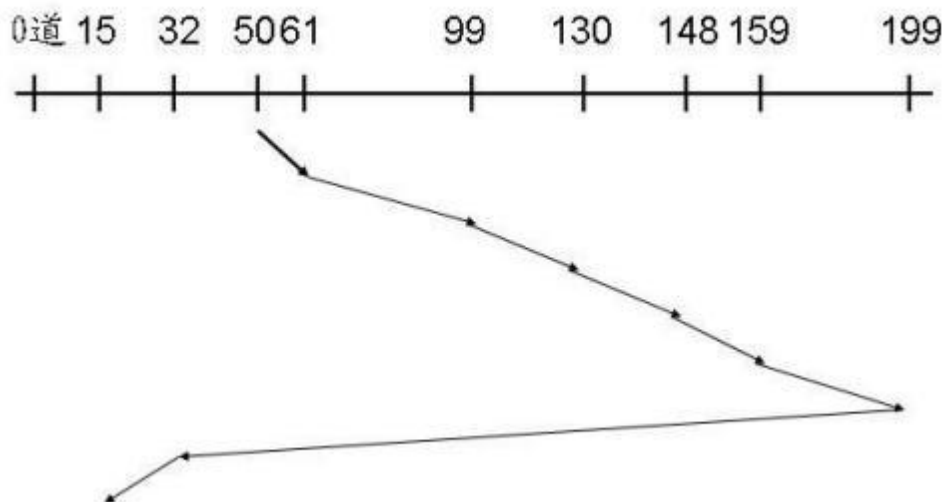
“最短寻找时间优先”不考虑磁臂的移动方向，总是选择离当前读写磁头最近的那个柱 面，这种选择可能导致磁臂来回改变移动方向

“电梯调度”是沿着磁臂的移动方向去选择离当前读写磁头最近的那个柱面的访问者， 仅当沿磁臂的前进移动方向无访问等待者时，才改变磁臂的前进方向，由于磁臂改变方向是 机械动作，速度相对较慢，因此，电梯调度算法是一种简单、实用且高效的调度算法，但是，“电梯调度”算法在实现时，不仅要记住读写磁头的当前位置，还必须记住磁臂的当前前进 方向。

4、单向扫描调度算法

单向扫描调度算法的基本思想是，不考虑访问者等待的先后次序，总是从 0 号柱面开始 向里道扫描，按照各自所要访问的柱面位置的次序去选择访问者。在移动臂到达最后一个一个 柱面后，立即快速返回到 0 号柱面，返回时不为任何的访问者等待服务。在返回到 0 号柱面 后，再次进行扫描。

例如，如果现在读写磁头正在 50 号柱面上执行输出操作，而等待访问者依次要访问的柱面 为 130，199，32，159，15，148，61，99。



5.调度算法比较

除了“先来先服务”调度算法外，其余三种调度算法都是根据欲访问的柱面位置来进行调度的。在调度过程中可能有新的请求访问者加入，新的请求访问者加入时如果读写已经超过了它们所要访问的柱面位置则只能在以后的调度中被选择执行。

在多道程序设计系统中，在等待访问磁盘的多个访问者请求中，可能要求访问的柱面号相同但在不同磁道或不同扇区，因此在进行移臂调度时，在按照某种算法把磁臂定位到某个柱面后，应该在等待访问这个柱面的各个访问者的输入输出操作都完成之后再改变磁臂的位置。

三、旋转调度优化

在移动臂定位后有若干个访问者等待访问该柱面的情况下，若从减少输入输出操作总时间为目标出发，显然应该优先选择延迟时间最短的访问者去执行。根据延迟时间来决定执行次序的调度称为“旋转调度”。

进行旋转调度时应分析下列情况：

- 1) 若干访问等待者请求访问同一磁道上的不同扇区。
- 2) 若干访问等待者请求访问不同磁道上的不同编号的扇区。
- 3) 若干访问等待者请求访问不同磁道上的具有相同编号的扇区。

对于前两种情况，旋转调度总是为首先到达读写磁头位置下的扇区进行读写操作。

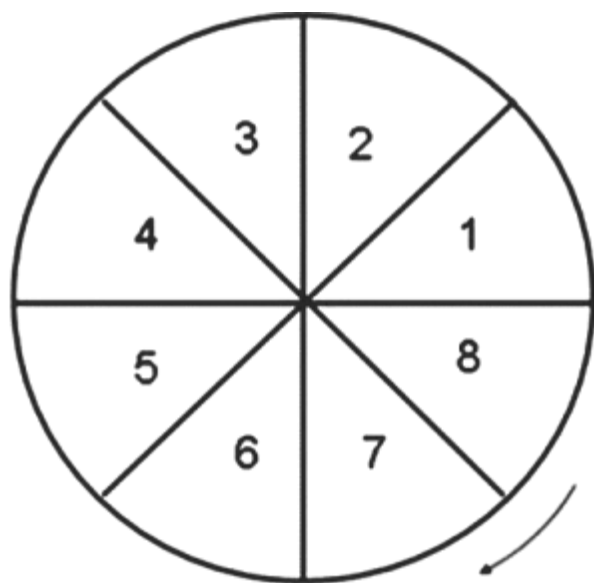
对于第三种情况，由于这些扇区编号相同，又在同一个柱面上，所以它们同时到达读写磁头的位置下。这时旋转调度可任意选择一个读写磁头进行读写操作。

四、信息的优化分布

记录在磁道上的排列方式也会影响磁盘的输入输出操作的时间。

假设某个系统在磁盘初始化时把磁盘的盘面分成 8 个扇区，今有 8 个逻辑记录被存储在同一个磁道上的这 8 个扇区中，供处理程序使用。处理程序要求顺序处理这 8 个记录，从 1 至 8。每次处理程序请求从磁盘上读出

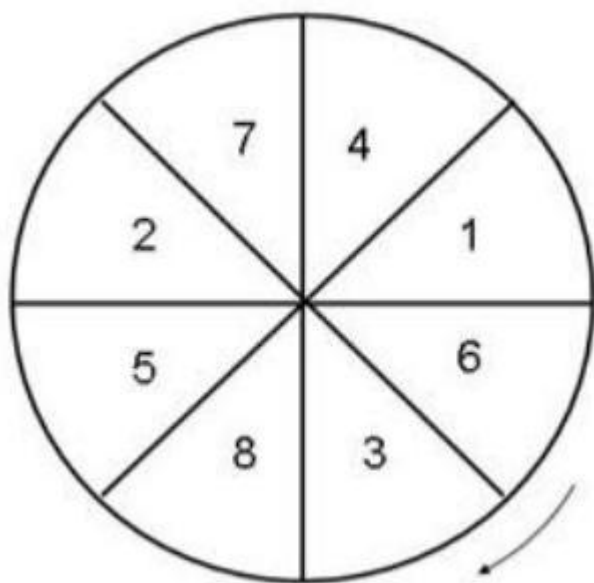
一个逻辑记录，然后程序对每个读出的记录花费 10ms 的时间进行运算处理，接着再读出下一个记录进行类似的处理，直至这 8 个记录都处理结束。假定磁盘转速为 40ms/周，8 个逻辑记录依次存储在磁道上。程序对每个读出的记录花费 10ms 的时间进行运算处理，接着再读出下一个记录进行类似的处理，假定磁盘转速为 40ms/周处理这 8 个记录所要花时间为： $8 \times (5+10) + 7 \times 30 = 330$ (ms)



(a)

上述 8 个逻辑记录在磁道上的位置重新进行优化安排，使得当读出一个记录并对之处理完毕之后，读写磁头正好处于需要读出的下一个记录位置上，于是可立即读出该记录，这样就不必花费那些延迟时间。

程序处理这 8 个记录所要花费的时间变为： $8 \times (5+10) = 120$ (ms)



(b)

第六节 缓冲技术

1.缓冲的引入

中断、DMA 和通道控制技术使得系统中各 I/O 设备之间、I/O 设备和CPU 之间可以并行 工作，但 I/O 设备和 CPU 的处理速度不匹配的问题是客观存在的，中断方式时容易造成数据 丢失。

引入缓冲区的作用：

解决 I/O 设备与处理机速度不匹配的问题 减少中断次数

根据 I/O 控制方式的不同，实现缓冲区的方法有两种：

- （1）采用专用的硬件设置数据缓冲区。这种方法常常应用在外设设备的 I/O 控制器中。打 印机、键盘。
- （2）在内存划出一定容量的专用数据缓冲区，以便存储输入/输出的数据，这种设置在内存 的缓冲区又称为“软件缓冲”。

2.缓冲的种类

根据系统设置的缓冲区的个数，可把缓冲技术分为以下几种：

- 1) 单缓冲：在 I/O 设备和处理器之间设置一个缓冲区。
- 2) 双缓冲区：解决两台 I/O 设备或者打印机和终端之间的并行操作问题。
- 3) 多缓冲：一种具有多个缓冲区，其中一部分缓冲区专门用于输入，另一部分缓冲区专门 用于输出的缓冲结构。
- 4) 缓冲池：把多个缓冲区连接起来统一管理，在缓冲池中的每个缓冲区既可用于输入又可 用于输出的一种缓冲结构。

3.缓冲池管理

缓冲区是一种临界资源，所以在使用缓冲区时都有一个申请、释放和互斥的问题需要考 虑。

缓冲池由多个缓冲区组成，而一个缓冲区由两部分组成：

- （1）用来标识和管理该缓冲器的缓冲首部；
- （2）用于存储数据的缓冲体。

这两部分有一一对应的映射关系。

缓冲首部包括：设备号、设备上的数据块号（使用块设备时）、互斥标识位以及缓冲队 列连接指针和缓冲器号 等。

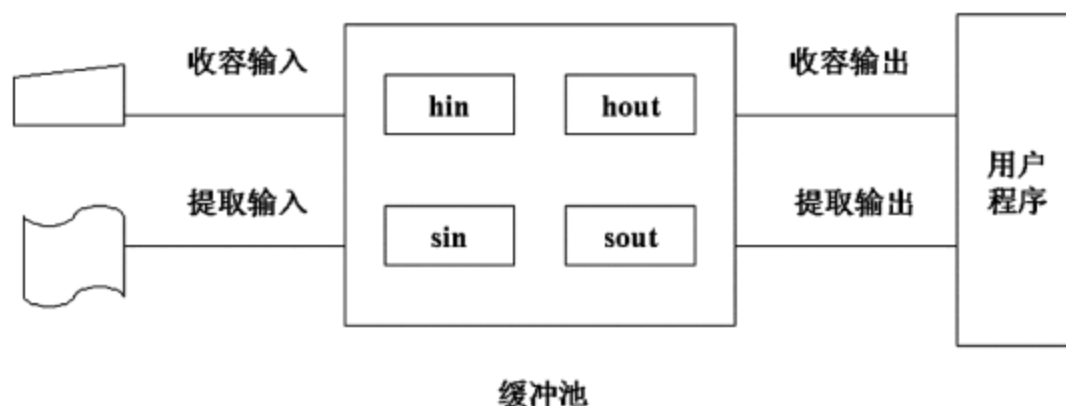
系统把各缓冲区按其使用状况连成三种队列：

- （1）空闲缓冲队列 **em**，其队首指针为 **F (em)**，队尾指针为 **L (em)**。
- （2）装满输入数据的输入缓冲队列 **in**，其队首指针为 **F (in)**，队尾指针为 **L (in)**。
- （3）装满输出数据的输出缓冲队列 **out**，其队首指针为 **F (out)**，队尾指针为 **L (out)**。在缓冲池中，有四 种工作缓冲区：

- 1) 用于收容设备输入数据的收容输入缓冲区 **hin**。
- 2) 用于提取设备输入数据的提取输入缓冲区 **sin**。

3) 用于收容处理器输出数据的收容输出缓冲区hout。

4) 用于提取处理器输出数据的提取输出缓冲区 sout。



对缓冲池的管理由如下几个操作组成：

- 1) 从三种缓冲区队列中按一定的选取规则取出一个缓冲区的过程 `take_buf (type)`。
- 2) 把缓冲区按一定选取规则插入相应的缓冲区队列的过程 `add_buf (type, number)`。
- 3) 供进程申请缓冲区用的过程 `get_buf (type, number)`。
- 4) 供进程将缓冲区放入相应缓冲区队列的过程 `put_buf (type, work_buf)`。

其中，参数 `type` 表示缓冲队列类型，`number` 为缓冲区号，而 `work_buf` 则表示工作缓冲区类型。

第七节 虚拟设备技术

虚拟设备技术，又称为 **SPOOLing** 技术，是多道程序设计系统中处理独占 I/O 设备的一种方法，它可以提高设备利用率并缩短单个程序的响应时间。

SPOOLing 技术之所以被称为虚拟设备技术，是因为它可以使进程在所需的外部设备不存在或被占用的情况下使用该设备。

一、虚拟设备的实现原理—SPOOLing 系统工作原理

SPOOLing 系统，全称为 **Simultaneous Peripheral Operations On-Line**，其含义是同时的外部设备联机操作，也称假脱机技术。

SPOOLing 系统主要包括输入程序模块、输出程序模块、作业调度程序三部分。工作原理如下：

- (1) 利用输入程序模块，在作业执行前就利用慢速设备将作业预先输入到后援存储器（如磁盘、磁鼓，此时，这些磁盘、磁鼓称为输入井）中去，称为预输入。
- (2) 作业进入内存运行后，使用数据时，直接从输入井中取出。
- (3) 作业执行时不必直接启动外部设备输出数据，只需将这些数据写入输出井（专门用于存储将要输出信息的磁盘、磁鼓）中去，称为缓输出。
- (4) 待作业全部运行完毕，再由外部设备输出全部数据和信息。

按照上述工作方式，就实现了对作业的输入、组织调度和输出管理的统一进行。同时，使外部设备在处理器直接控制下，又与处理器并行工作（故称为假脱机）。

二、SPOOLing 系统的实现—打印机的值班进程

SPOOLing 系统通常分为输入 SPOOLing 和输出 SPOOLing，二者工作原理类似。以常见的打印机共享为例，说明输出 SPOOLing 的基本原理。

打印机是一种典型的独占设备，引入 SPOOLing 技术后，用户的打印请求传递给 SPOOLing 系统，而并不是真正把打印机分配给用户。

SPOOLing 系统的输出管理模块在磁盘上申请一个空闲区，把需要打印的数据传送到里面，再把用户的打印请求挂到打印队列上。

如果打印机空闲，后台打印程序就会从打印队列中取出一个请求，再从磁盘上的对应输出并取出数据，执行打印操作。

由于磁盘是共享的，SPOOLing 系统可以随时响应打印请求并把数据缓存起来。独占设备改造成了共享设备，从而提高了设备的利用率和系统效率。

第八章 进程同步机制与死锁

本章重难点分析

1.进程同步与互斥 2.死锁

第一节 进程（线程）的同步与互斥

多道程序系统中并发进程通常有多个。在逻辑上具有某种联系的进程称为相关进程

如果一个进程的执行不影响其他进程的执行，且不受其他进程的影响，则这些并发进程相互之间是无关的

如果一个进程的执行影响到其他进程的执行，或者受其他进程的影响，则这些并发进程是相关的

一、与时间有关的错误

例如，两个并发程序 A 和 B 共享一个公共变量 n，程序 A 每执行一次循环都要作 $n := n+1$ 操作，程序 B 则在每一次循环中打印出 n 的值并将 n 重新置 0。

程序 A.

```
While (true) { n=n+1;  
.... }; ... ..
```

程序 B.

```
While (true) { print (n) ;  
n=0;  
.... };
```

相同的程序在可能的三种情况下，分别产生了三组不同的结果。

根本原因在于：在并发程序中共享了公共变量，使得程序的计算结果与并发程序执行的速度有关。

这种错误的结果又往往是与时间有关的（如上例中的三种情形，其结果时错时对，随执行速度的不同而异），所以，把它称为“与时间有关的错误”

二、进程的同步与互斥

解决进程同步与互斥的做法有两种：一是由竞争各方平等协商；二是引入进程管理者

临界资源是指计算机系统上的需要互斥使用的硬件或软件资源，如外设、共享代码段、共享数据结构等。多个进程在对临界资源进行写入或修改操作时，必须互斥地进行

计算机系统上也有一些可以同时访问的共享资源不是临界资源，如只读数据

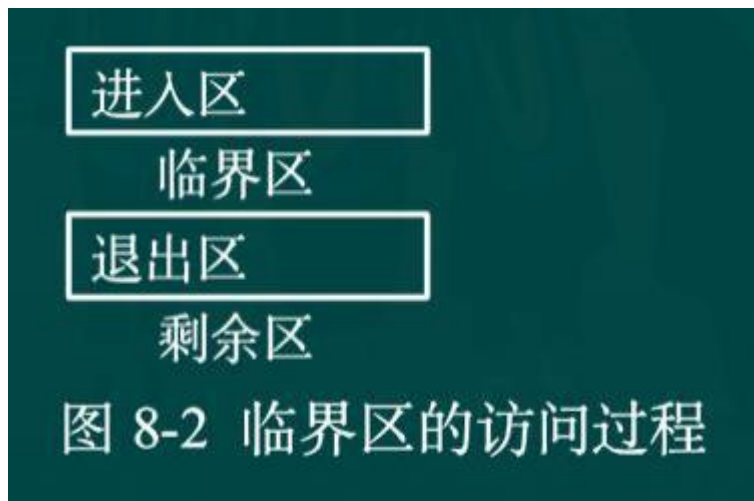
进程间的相互制约关系按相互感知程度分为如表 8-1 所列的三种类型。资源共享的程度 分成三个层次：互斥（**mutual exclusion**）、死锁（**deadlock**）和饥饿（**starvation**）

保证资源的互斥使用是指多个进程不同时使用同一个资源。避免死锁是指避免多个进程 互不相让而得不到足够的资源的情况出现。避免饥饿是指避免某些进程一直得不到资源

进程间的相互制约关系：

相互感知的程度	交互关系	一个进程对其他进程潜在的控制问题的影响	
相互不感知（完全不了解其他进程的存 在）	竞争 （ competition ）	一个进程的操作对其 他进程的结果无影响	互斥，死锁，饥饿
间接感知（双方都与 第三方交互，如共享 资源）	通过共享进行协作	一个进程的结果依赖 于从其他进程获得的 信息	互斥，死锁，饥饿
直接感知（双方直接 交互，如通信）	通过通信进行协作	一个进程的结果依赖 于从其他进程获得的 信息	死锁，饥饿

- 临界资源的正确访问过程分成如图8-2 所示的四个部分：
- 1）进入区（**entry section**）：在进入区设置相应的“正在访问临界区”标志，检查可否进 入临界区； 以防止其
他进程同时进入临界区
 - 2）临界区（**critical section**）：进程中访问临界资源的一段代码
 - 3）退出区（**exit section**）：将“正在访问临界区”的标志清除
 - 4）剩余区（**remainder section**）：代码中的其余部分 同步机制应遵循以下 4 条准则
- 1）空闲则入：任何时刻最多只有一个进程位于临界区。当有进程位于临界区时，其他进程 不能进入临界区
 - 2）忙则等待：当已有进程处于临界区时，后到达的进程只能在进入区等待
 - 3）有限等待：等待进入临界区的进程不能无限期地“死等”
 - 4）让权等待：因在进入区等待而不能进入临界区的进程，



应释放处理机，转换到阻塞状态 1.进程互斥的软件方法

通过平等协商方式实现进程互斥的最初方法是软件方法。其基本思路是在进入区检查和 设置一些标志，如果已有进程在临界区，则在进入区通过循环检查进行等待；在退出区修改 标志

彼得松算法（Peterson 's Algorithm）是一种软件实现的临界区访问算法，其做法是： 先修改临界区标识、后检查、后修改者等待

以两个进程为例，假设有两个进程 P_i 和 P_j ，其中一个公用整型变量为 $turn$ ，描述允 许进入临界区的进程标识； $turn$ 为 i 时，进程 P_i 可进入，否则不可进入；标志 $flag[i]$ 表 示进程 i 想进入临界区，若 $flag[i]$ 为 $TRUE$ ，则进程 i 准备进入临界区，反之则不想进入 彼得松算法的基本思想是：每个进程都在进入区先将 $flag$ 修改为本进程为 $TRUE$ ，而将 $turn$ 设为另一个进程，所以当另一个进程想进临界区时就可以进入。如果两个进程同时希望进入 临界区，那么 $turn$ 会在几乎同时被设成 i 和 j ，但是只有后一个赋值语句的结果会被保持， 因此 $turn$ 的最终值决定了先到的（先赋值的）进程能进入临界区。

彼得松算法可实现四条准则中的前二条：空闲则入和忙则等待。但 P_i 和 P_j 会形成乒乓 效应，也就是当 P_i 希望进入临界区而 P_j 无响应时， P_i 会出现“饥饿”现象。另外对于三 个以上进程间的互斥其标识设计更为复杂

这里的根本问题就是修改标志和检查标志不能作为一个整体来执行 2.进程互斥的硬件方法

软件方法实现进程互斥不适用于数目很多的进程间的互斥

利用硬件方法的主要思路是用一条指令完成读和写两个操作，因而保证读操作与写操作 不被打断。依据所采用的指令的不同硬件方法分成 TS 指令和 $Swap$ 指令两种

（1）Test-and-Set 指令

TS 指令的功能是读出指定标志后把该标志设置为 $TRUE$ 。 TS 指令互斥算法是，每个临 界资源设置一个公共布尔变量 $lock$ ，表示资源的两种状态： $TRUE$ 表示正被占用， $FALSE$ 表 示空闲。初值为 $FALSE$ 。在进入区利用 TS 进行检查和修改标志 $lock$ 。有进程在临界区时， 重复检查；直到其他进程退出时，检查通过。

（2）Swap 指令（或 Exchange 指令）

$Swap$ 指令的功能是交换两个字（字节）的内容。 $Swap$ 指令互斥算法是，每个临界资源 设置一个公共布尔变量 $lock$ ，初值为 $FALSE$ ；每个进程设置一个私有布尔变量 key ，用于与 $lock$ 间的信息交换。在进入区利用 $Swap$ 指令交换 $lock$ 与 key 的内容，然后检查 key 的状 态；有进程在临界区时，重复交换和检查过程，直到其他进程退出时，检查通过。

硬件方法把修改和检查操作合成为整体而具有明显的优点，体现在以下几个方面：

- 1) 适用范围广：硬件方法适用于任意数目的进程，在单处理器和多处理器环境中完全相同
- 2) 简单：硬件方法的标志设置简单，含义明确，容易验证其正确性
- 3) 支持多个临界区：在一个进程内有多个临界区时，只需为每个临界区设立一个布尔变量 硬件方法的主要缺点包括：
 - 1) 进程在等待进入临界区时，处理机为“忙等”，不能实现“让权等待”
 - 2) 进入临界区的进程是从等待进程中随机选择的，可能导致“饥饿”

三、信号量（semaphore）和 P、V 原语

平等进程间的协商机制存在有些问题是平等协商无法解决的，需要引入一个地位高于进程的管理者来解决公有资源的使用问题

操作系统可以从进程管理者的角度来处理互斥的问题，信号量就是由操作系统提供的管理公有资源的有效手段

信号量是荷兰学者 Dijkstra 于 1965 年提出的一种卓有成效的进程同步机制

信号量机制所使用的 P、V 原语就来自荷兰语的 test（proberen）和 increment（verhogen）所谓“原语”即指执行中不受进程调度和执行的打断，恰如一条指令

每个信号量 s 除一个整数值 s.count（计数）外，还有一个进程等待队列 s.queue，其中存放的是阻塞在该信号量的各个进程的标识

信号量只能通过初始化和两个标准的原语即 P 原语和 V 原语来访问。P、V 原语的执行很好地解决了操作的整体性

信号量的初始化可指定一个非负整数值，表示空闲资源总数。若信号量为非负整数值，表示当前的空闲资源数；若为负值，其绝对值表示当前等待临界区的进程数

信号量机制中 P 原语相当于进入区操作，V 原语相当于退出区操作。

信号量机制可实现临界资源的互斥访问。如图所示，mutex（MUTual Exclusion）为互斥信号量，其初值为 1；临界区代码置于 P（mutex）和 V（mutex）原语之间。在使用信号量时，必须成对使用 P 和 V 原语。遗漏 P 原语则不能保证互斥访问，遗漏 V 原语则不能在使用临界资源之后将其释放给其他等待的进程。P、V 原语的使用不能次序错误、重复或遗漏



信号量机制可实现进程间的同步。如图所示前趋关系是指并发执行的进程 P1 和 P2 中， 分别有代码 C1 和 C2， 要求 C1 在 C2 开始前完成执行。我们可设置一个互斥信号量 S12， 其 初值为 0。这样， 只有在 P1 执行到 V（S12）后， P2 才会结束 P（S12）的执行

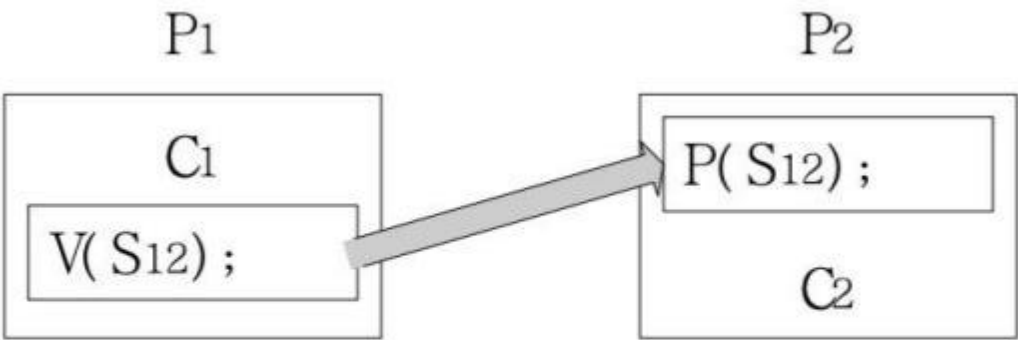


图 8-7 利用信号量描述前趋关系

第二节 经典的进程同步问题

Dijkstra 把同步问题抽象成一种“生产者-消费者关系 ”

生产者-消费者问题是计算机中各种实际的同步、互斥问题的一个抽象模型。计算机系 统中的许多问题都可被 归结为生产者和消费者关系，例如，处理并生成数据是生产者进程， 打印结果是消费者进程；在输入时输入进 程是生产者进程，处理并生成数据是消费者进程。

一、简单生产者—消费者问题

简单生产者-消费者问题：设有一个生产者进程 P，一个消费者进程 Q，它们通过一个缓冲区 联系起来，如图 所示。缓冲区只能容纳一个产品，生产者不断地生产产品，然后往空缓冲区 送产品；而消费者则不断地从缓冲 区中取出产品，并消费掉。

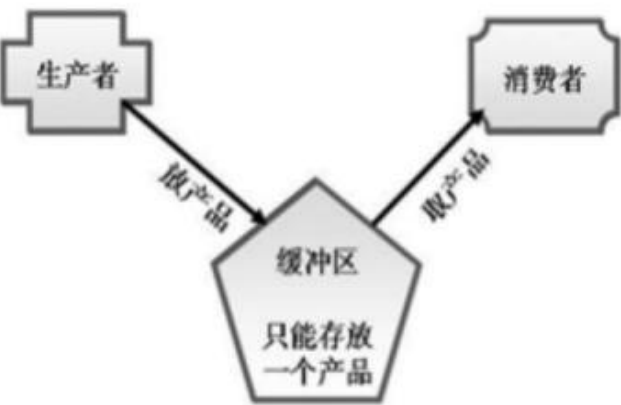


图 8-8 简单的生产者-消费者问题

P 进程不能往已经“满”的缓冲区中放入产品，设置信号量 **empty**，其初值为 1，用于 指示空缓冲区数量；同样，Q 进程也不能从已经“空”的缓冲区中取出产品，设置信号量 **full**， 初值为 0，用于指示满缓冲区数量。

一、简单生产者—消费者问题 P:

```
while (true) { P (empty) ;
```

生产一个产品；

送产品到缓冲区； V (full) ;

```
};
```

Q:

```
while (true) { P (full) ;
```

从缓冲区取产品； V (empty) ;

消费产品； };

二、多个生产者—消费者问题

设有若干个生产者进程 P1, P2, ... , Pn, 若干个消费者进程 Q1, Q2, ... , Qn, 如图所 示。生产者进程不断地生产产品并把它们放入缓冲池内，消费者进程不断地从缓冲池内取产 品并消费之。这里既存在同步问题，也存在互斥问题

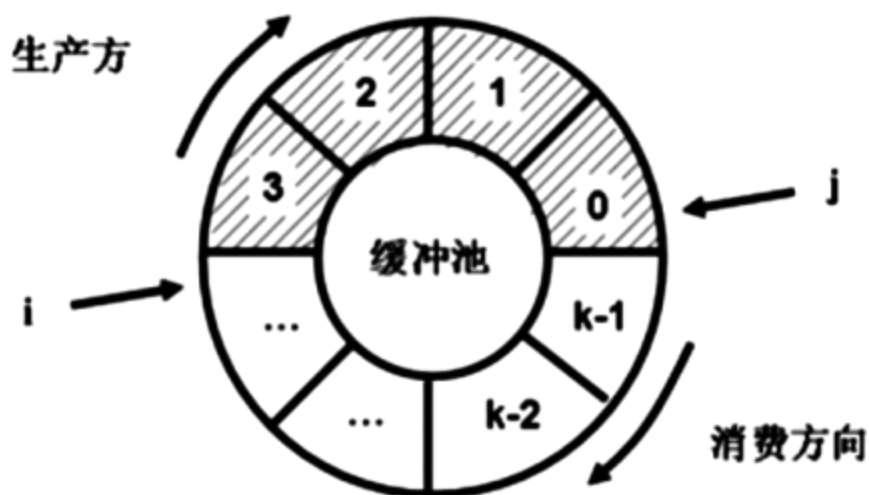


图 8-8 环形缓冲池

(1) 同步问题

生产者进程不能往“满”的缓冲区中放产品，设置信号量 **empty**，初为k，用于指示缓 冲池中空缓冲区数目。消 费者进程不能从“空”的缓冲区中取产品，设置信号量 **full**，初 值为 0，用于指示缓冲池中满缓冲区数目。

(2) 互斥问题

设置信号量**mutex**，初值为 1，用于实现临界区（环形缓冲池）的互斥。另设整型量 i 和 j，初值均为 0。i 用于 指示空缓冲区的头指针；j 用于指示有产品的满缓冲区的头指针。

（3）算法

生产者进程 P1, P2, ..., Pn: i=0;

```
while (true) {
```

```
    生产产品 P(empty); P(mutex);
```

```
    往 Buffer[i]中放产品; i= (i+1) mod k;
```

```
    V(mutex);
```

```
    V(full); };
```

消费者进程 Q1, Q2, ..., Qn: j=0;

```
while (true) { P(full);
```

```
    P(mutex);
```

```
    从 Buffer[j]中取产品; j= (j+1) mod k;
```

```
    V(mutex);
```

```
    V(empty); 消费产品;
```

三、读者—写者问题

（1）读者——写者问题的描述

假定有某个共享文件 F，系统允许若干进程对文件 F 进行读或写。这里把要读文件的进程称为读者，把要写文件的进程称为写者。

读者和写者必须遵守如下的规定。

- 1) 多个进程可以同时读文件 F。
- 2) 任一个进程在对文件 F 进行写时，不允许其他进程对文件进行读或写。
- 3) 当有进程正在读文件时不允许任何进程去写文件。

设 read_count 记录当前正在读的读者进程个数，由于多个读者都对 read_count 进行修改，所以 read_count 是一个共享变量，需要互斥使用，故设置信号量mutex。

再设置信号量write，用于写者之间互斥，或第一个读者和最后一个读者与写者的互斥。这里对上面的解决方案做一些具体的说明。

read_count 是一个计数器，初值为 0。而 mutex 和write 都是信号量，它们的初值都是 1。

read_count 是一个共享变量，需要互斥使用，在每个读者进入时，对 read_count 的计数不能出错。另外，如果第一个读者进入，就不能再允许写者进入，这就是 if (read_count=1) P(write) 语句的作用；第一个读者负责禁止任何写者进入。以上这两个操作都要放在互斥区内。而其他的读者可以随着第一个读者进入陆续进入。

当有读者完成读操作之后，相应地要对read_count 进行减一操作。而且如果 read_count=0，表明已经没有读者了，写者可以随时进入。

对于读者来说，只要有一个写者已经在临界区执行写操作，所有的读者就必须等待。

(2) 读者——写者问题的解决方案 写者进程:

```
while (true) { P (write) ;
```

写文件;

```
V (write) ; }
```

读者进程:

```
while (true) {
```

```
P (mutex) ;
```

```
read_count=read_count+1;
```

```
If (read_count=1) P (write) ; V (mutex) ;
```

读文件;

```
P (mutex) ;
```

```
read_count=read_count-1;
```

```
If (read_count=0) V (write) ; V (mutex) ;
```

```
}
```

第三节 死锁

一、死锁的定义

死锁现象并不是计算机操作系统环境下所独有，在日常生活乃至各个领域是屡见不鲜的。例如，没有交通灯的十字路口出现了死锁。

死锁是指在多道程序系统中的一种现象：一组进程中的每一个进程均无限期地等待被该组进程中的另一个进程所占有且永远不会释放的资源。系统发生这种现象称为系统处于死锁状态，简称死锁。处于死锁状态的进程称为死锁进程。

当死锁发生后，死锁进程将一直等待下去，除非有来自死锁进程之外的某种干预。系统发生死锁时，死锁进程的个数至少为两个。

系统发生死锁不仅浪费了大量的系统资源，甚至会导致整个系统崩溃，带来灾难性的后果。

二、死锁产生的原因

产生死锁的原因主要有两个：一是竞争资源，系统资源在分配时出现失误，进程间对资源的相互争夺而造成僵局；二是多道程序运行时，进程推进顺序不合理

1.资源的概念

系统中的资源有两类：永久性资源（可重用资源），如内存、外部设备、CPU 等硬件资源，以及各种数据文件、表格、共享程序代码等软件资源

临时性资源（消耗性资源），是指由某个进程所产生、只为另一个进程使用一次、或经过短暂时间后便不再使用的资源，如 I/O 和时钟中断信号、同步信号、消息等

永久性资源和临时性资源都可能导致死锁发生。 2.死锁的例子

进程 P1 和 P2 在运行中都使用输入、输出设备，假定系统中只有一台输入设备，一台输出设备，则进程 P1 和 P2 可有如下形式

P1
①申请一台输入设备； ②申请一台输出设备； 使用各设备；
释放输入设备； 释放输出设备； P2
I 申请一台输出设备； II 申请一台输入设备； 使用各设备；
释放输出设备； 释放输入设备；
当进程 P1 申请并获得了该输入设备后（运行完成①），由于进程切换或某种原因，停止 前进。此时 P2 到达，P2 进程完成了对输出设备的申请（运行完成②），接下来再申请输入设备（运行④），由于输入设备被 P1 占用必将导致 P2 被阻塞且进入等待队列等待
若进程 P1 重新获得运行机会，接下来便要申请输出设备（运行③），同样，也被阻塞进入等待输出设备的等待队列。进程 P1 和 P2 彼此无限地等待对方释放资源从而唤醒自己，于是形成了僵局，如图所示

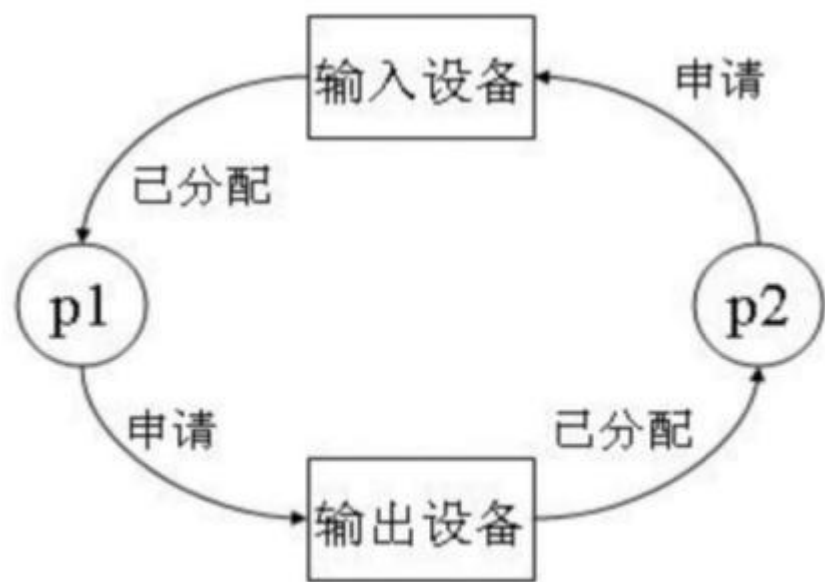


图 8-10 申请不同类资源产生死锁

假设有一类可重用资源 R，如主存（或磁盘），它包含有 m 个页面（或扇区），由 n 个进程 P1, P2, ..., Pn ($2 \leq m \leq n$) 共享。假定每个进程按下述顺序依次申请和释放页面（或扇区）：

- ①申请一个页面 ...
- ②申请一个页面 ...
- 释放一个页面 ...
- 释放一个页面

这里每次申请和释放只涉及 R 的一个分配单元（页面或扇区）。因此，当所有单元全部分配完毕时，很容易发生死锁；占有 R 的单元的所有进程（前 m 个进程）会永远封锁在第二次申请②上，而有些进程（n-m 个进程）类似地会封锁在它们的第一次申请①上。图 8-11 说明了 n= 3、m=2 时系统的状态

这类死锁是相当普遍的。例如，在若干输入和输出进程竞争磁盘空间的 SPOOLing 系统中，就可能发生这类死锁。如果磁盘空间完全分配给等待装入的作业输入文件和已部分运行 的作业输出记录，则系统就有可能发生死锁

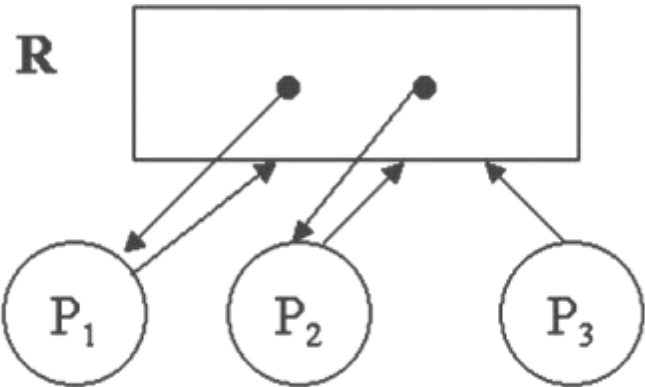


图 8-11 同类资源共享时的死锁现象

对于第四章描述的生产者和消费者问题，若把生产者和消费者程序中前面两个 P 操作顺 序颠倒，则形成如下所示的两个序列

生产者进程： P（mutex）； P（empty）；

消费者进程： P（mutex）； P（full）；

当进行了n 次生产后（或 n 个生产者，每人都送了缓冲区一个产品后），缓冲区被全部 占满，此时 empty=0。若生产者执行 P（mutex）后（此时 mutex=0），又继续执行 P（empty）， 此刻 empty=-1，使生产者因无可用缓冲区而在 empty 上等待。若又有一个消费者进程到达， 并执行 P（mutex），结果使 mutex=-1

因此，消费者也阻塞，并且在 mutex 上等待。此时，生产者、消费者都彼此等待对方来 唤醒自己，处于循环等待状态，这样便形成了死锁局面

对临时性资源的使用不加限制而引起的死锁

信件可以看作是一种临时性资源，也可能引起死锁。比如，进程 P1 等待进程 P3 的信件 s3 到来后再向进程 P2 发送信件 s1，P2 又要等待 P1 的信件 s1 到来后再向P3 发送信件 s2， 而 P3 也要等待 P2 的信件 s2 来到后 才能发出信件 s3。在这种情况下就形成了循环等待，产生 死锁。

进程 P1

receive（s3）；

.....

send（s1）；

.....

进程 P2

receive（s1）；

.....

send（s2）；

.....

进程 P3

receive (s2) ;

.....

send (s3) ;

.....

三、死锁产生的必要条件

产生死锁有四个必要条件 (Coffman 等, 1971) 1.互斥条件

资源是独占的且排他使用 2.不可剥夺条件

又称不可抢占或不可强占。进程所获得的资源在未释放前, 不能被其他进程强行剥夺 3.请求和保持条件

又称部分分配或占有申请。进程在申请新的资源的同时, 继续占用已分配到的资源 4.循环等待条件

又称环路等待。在发生死锁时, 必然存在一个进程等待队列{P1, P2, ..., Pn}, 其中 P1 等待 P2 占有的资源, P2 等待 P3 占有的资源, ..., Pn 等待 P1 占有的资源, 形成一个进程等待环路

四、死锁发生后的处理方法 1.死锁预防

死锁预防通过设置某些严格限制, 破坏产生死锁的必要条件以防止死锁发生

若死锁是由于资源数量不足而造成的, 可以通过增加资源数量的方法来进行预防, 例如 内存

临时性的资源如系统中的共享写数据, 各种信号量等, 不能通过增加资源数量来解决, 所以这种情况下通过破坏互斥条件来预防死锁不现实

在预防死锁的静态分配策略中, 资源分配的原则是

一个进程在申请新资源的要求不能立即得到满足时, 便处于等待状态。而一个处于等待状态的进程的全部资源可以被剥夺。被剥夺的资源重新加入到资源表中。仅当该进程重新获得它原有的资源以及得到新申请的资源时, 才能重新启动执行

这种方法破坏了不可剥夺条件 具体实施方法

(1) 若一个进程已占用了某些资源, 又要申请一个新的资源, 在申请新的资源时, 不能立即得到满足, 在变为等待状态之前, 该进程必须释放已占有的所有资源, 即阻塞前释放资源

(2) 若一个进程申请某些资源, 首先系统应检查这些资源是否可用, 如果可用, 就分配给该进程。否则, 系统检查这些资源是否分配给另外某个等待进程。若是, 则系统将剥夺所需资源, 分配给这个进程。如果资源没有被等待进程占有, 那么, 该进程必须等待。在其等待过程中, 其资源也有可能被剥夺

破坏不可剥夺条件以预防死锁的方法适用于这样一些资源, 它们的状态是容易保存和恢复的, 例如 CPU、内存等

在预防死锁的静态分配策略中, 还可以采用以下方法, 该方法破坏了请求和保持条件 (3) 每个进程必须在开始执行前就申请它所需要的全部资源, 仅当系统能满足进程的资源申请要求且把资源一次性分配给进程后, 该进程才能开始执行

采用该方法后, 进程在执行过程中不再申请资源, 故不可能出现占有了某些资源再等待其他资源的情况, 即“请求和保持”的条件不成立, 从而预防死锁的发生

这种方法其缺点是严重浪费系统资源

静态分配资源策略还有一个问题是大部分进程在运行开始时不能确定所需要的全部资源, 例如堆栈所需的内存

(4) 当进程没有占用资源时才允许它去申请资源，如果进程已经占用了某些资源而又要再 申请资源，则它应先归还所占的资源后再申请新资源

这种资源分配策略仍会使进程处于等待状态，同时也存在着资源利用率低的缺点 采用资源有序分配策略打破了循环等待的条件

其基本思想是将系统中所有资源顺序编号。进程申请资源时，必须严格按照资源编号的顺序进行，否则系统不予分配。释放资源时，应按编号逆序进行

2.死锁避免
在资源的动态分配过程中，采取某种方法防止系统进入不安全状态，从而避免死锁的发 生。可获得较高的资源利用

死锁避免的基本思想是：系统对进程发出的每一个申请进行动态检查，并根据检查结果 决定是否分配资源；如果分配后系统可能发生死锁，则不予分配，否则予以分配

死锁避免和死锁预防的区别在于，死锁预防是设法破坏产生死锁的四个必要条件之一， 严格地防止死锁的出现。而死锁避免则是在系统运行过程中注意避免死锁的最终发生

由于在避免死锁的策略中允许进程动态地申请资源，因而，系统需提供某种方法在进行 资源分配之前，先分析资源分配的安全性。当估计到可能有死锁发生时就应设法避免死锁的 发生

如果操作系统能保证所有的进程在有限时间内得到需要的全部资源，则称系统处于“安 全状态”，否则说系统是不安全的

所谓安全状态是指，如果存在一个由系统中所有进程构成的安全序列{P1, ..., Pn}，则 系统处于安全状态。一个进程序列{P1, ... , Pn}是安全的，如果对于其中每一个进程Pi (1 ≤i≤n)，它以后尚需要的资源量不超过系统当前剩余资源量与所有进程 Pj (j <i) 当前占 有资源量之和。则系统处于安全状态

如果不存在任何一个安全序列，则系统处于不安全状态

不安全状态不一定导致死锁，但死锁状态一定是不安全状态

最著名的死锁避免算法是由 Dijkstra[1965]和 Habermann[1969]提出来的银行家算法。这一名称的来历是基于该算法将操作系统比作一个银行家，操作系统的各种资源比作资金， 申请资源的进程比作向银行贷款的顾客 操作系统的资源分配问题就如同银行家利用其资金贷款的问题，一方面银行家能贷款给 若干顾客，满足顾客对资金的要求；另一方面，银行家可以安全地收回其全部贷款而不至于 破产

银行家算法规定

- (1) 当一个顾客对资金的最大需求量不超过银行家现有的资金时就可接纳该顾客
- (2) 顾客可以分期贷款，但贷款的总数不能超过最大需求量
- (3) 当银行家现有的资金不能满足顾客尚需的贷款数额时，对顾客的贷款可推迟支付，但 总能使顾客在有限的时间里得到贷款
- (4) 当顾客得到所需的全部资金后，一定能在有限的时间里归还所有的资金

假定某系统有三类资源 A、B、C，A 类资源共有 10 个资源实例，B 类资源共有 5 个资源 实例，C 类资源共有 7 个资源实例，现有 5 个进程 P1、P2、P3、P4、P5，它们对各类资源的 最大需求量和第一次分配后已占有的资源量如图所示。

资源申请进程	目前占有量			最大需求量			尚需要量		
	A	B	C	A	B	C	A	B	C

P1	0	1	0	7	5	3	7	4	3
P2	2	0	0	3	2	2	1	2	2
P3	3	0	2	9	0	2	6	0	0
P4	2	1	1	2	2	2	0	1	1
P5	0	0	2	4	3	3	4	3	1
系统剩余 资源量	A	B	C						
	3	3	2						

应用银行家算法，找到一个进程安全序列 P2，P4，P5，P3，P1，可以得出结论：图 8-13 中的系统状态是安全状态 在此状态下，进程 P2 提出新的资源申请：A 类 1 个，B 类 0 个，C 类 2 个，进行试探性分配后，系统状态如图所示

资源申请进程	目前占有量			最大需求量			尚需要量		
	A	B	C	A	B	C	A	B	C
P1	0	1	0	7	5	3	7	4	3
P2	3	0	2	3	2	2	0	2	0
P3	3	0	2	9	0	2	6	0	0
P4	2	1	1	2	2	2	0	1	1
P5	0	0	2	4	3	3	4	3	1
系统剩余 资源量	A	B	C						
	2	3	0						

应用银行家算法，仍然可以找到一个进程安全序列 P2，P4，P5，P1，P3，表明该系统 状态是安全状态，可以实施资源分配

在图所示状态下，进程 P5 又申请资源：A 类 3 个，B 类 3 个，C 类 0 个，此时系统不能 实施分配，因为该申请超过了系统当前剩余的资源量

若进程 P1 提出新的资源申请：A 类 0 个，B 类 2 个，C 类 0 个，也不能实施资源分配， 因为根据银行家算法，若进行了分配，则在新的系统状态下找不到进程安全序列，这将导致 系统进入不安全状态

不安全状态并非是死锁状态，只是有死锁的可能性 3.死锁的检测与解除

死锁的检测与解除是在系统运行过程中，通过在系统中设置检测机构，定时检测死锁是 否真的发生，并精确地确定与死锁有关的进程与资源，然后采取措施解除死锁。

操作系统可定时运行一个“死锁检测”程序，该程序按一定的算法去检测系统中是否存 在死锁，即是否存在“循环等待”条件。

通常，死锁检测可以在任何一次资源分配后，也可以在每次调度后，或者利用定时器定 时运行检测，或某个进程长期位于阻塞态或阻塞进程过多时，启动死锁检测程序。

死锁检测的算法过程

- (1) 为每个进程和每个资源指定唯一编号
- (2) 设置一张资源分配状态表，每个表目包含“资源号”和占有该资源的“进程号”两项， 资源分配表中记录了每个资源正在被哪个进程所占有

(3) 设置一张进程等待分配表, 每个表目包含“进程号”和该进程所等待的“资源号”两项

(4) 算法规则: 当任一进程 P_j 申请一个已被其他进程占用的资源 r_i 时, 进行死锁检测。检测算法通过反复查找资源分配表和进程等待表, 来确定进程 P_j 对资源 r_i 的请求是否导致

形成环路, 若是, 便确定出现死锁 解除死锁的方法

剥夺资源法解除死锁

(1) 选择一个牺牲进程, 即要剥夺哪个进程的哪些资源

(2) 被选中的进程重新运行或回退到某一点开始继续运行

(3) 保证不发生“饿死”现象

(4) “最小代价”

进程重新运行的代价包括

①进程的优先级

②进程已经运行了多长时间, 该进程完成其任务还需要多长时间

③该进程使用的资源种类和数量? 这些资源能简单地剥夺吗

④为完成其任务, 进程还需要多少资源

⑤有多少进程要被撤销

⑥该进程被重新启动运行的次数

(2) 撤销进程

撤销死锁进程, 将它们占有的资源分配给另一些死锁进程, 直到死锁解除为止 撤销进程的代价

①进程优先数, 即被撤销进程的优先数

②进程类的外部代价

③运行代价 4.忽略死锁

对于那些死锁发生的几率极低, 而解决死锁的方法代价极大或暂时没有办法解决的死锁 问题暂时不予理睬

第四节 哲学家就餐问题

一、哲学家就餐问题

哲学家就餐问题是操作系统中关于进程同步与互斥的经典问题, 也是涉及到死锁的关键问题

问题的描述是这样的, 有 5 个哲学家以思考、用餐交替进行的方式生活, 他们坐在一张圆桌边, 桌子上有 5 个盘子和 5 只筷子

当一个哲学家思考时, 他与邻座的哲学家没有任何联系。当一个哲学家感觉到饿了, 他就试图拿起他左右两边的筷子用餐。如果他的邻座已经拿到筷子, 则他可能只拿到一只甚至一只筷子也拿不到。当一个饥饿的哲学家得到了两只筷子, 他就可以用餐。当他用餐完毕, 他就放下筷子并再次开始思考。

更多资料 自考真题考研资料库

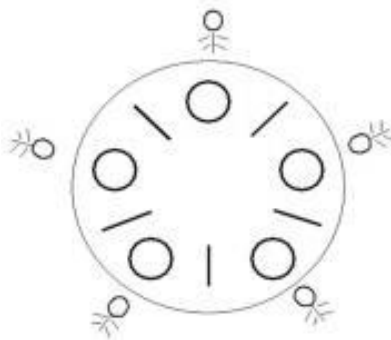


图8-16 哲学家就餐问题

一个简单的解决方案是为每只筷子设置一个信号量，一个哲学家通过在相应信号量上执行 P 操作拿起一只筷子，通过执行 V 操作放下一只筷子。

5 个信号量构成一个的数组：VAR chopstick ARRAY[0..4]OF

semaphore; 每个信号量都置初值为 1 算法设计：

P (chopstick[i]) ; {拿左筷子}

P (chopstick[(i+1) MOD 5]) ; {拿右筷子} 用餐；

... ..

V (chopstick[i]) ; {放左筷子}

V (chopstick[(i+1) MOD 5]) ; {放右筷子} Until false;

改进算法： Repeat

思考；

... ..

P (chopstick[1]) ; {拿右筷子} P (chopstick[5]) ; {拿左筷子}

用餐；

V (chopstick[1]) ; {放右筷子} V (chopstick[5]) ; {放左筷子} Until false;

当每个哲学家都想就餐时，可能有 4 个哲学家都已拿到了自己左边的一支筷子，而剩下的第 5 个哲学家拿不到编号小的筷子（一定被某个哲学家拿走）而等待，该哲学家不可能去拿另一支筷子。因此，4 个哲学家中的一个哲学家就有机会拿起其右边的筷子而可以用餐，就餐后放下一双筷子，以使另一个哲学家又可得到右边的筷子去用餐，同样地，其他哲学家也都先后可吃顿饭，从而防止了死锁。

为了提高资源的利用率，通常应当按照大多数进程使用资源的次序对资源进行编号。先使用者编号小，后使用者编号大。

1) 这种硬性规定申请资源的方法，会给用户编程带来限制，按照编号顺序申请资源增加了资源使用者的不便；

2) 如何合理编号是一件困难的事情，特别当系统添加新设备类型时，会造成不灵活、不方便；如果有进程违反规定，则仍可能发生死锁。资源有序分配法与资源静态分配策略相比，显然提高了资源利用率，进程实际需要申请的资源不可能完全与系统所规定的统一资源编号一致，为遵守规定，暂不需要的资源也要提前申请，仍然会造成资源的浪费。